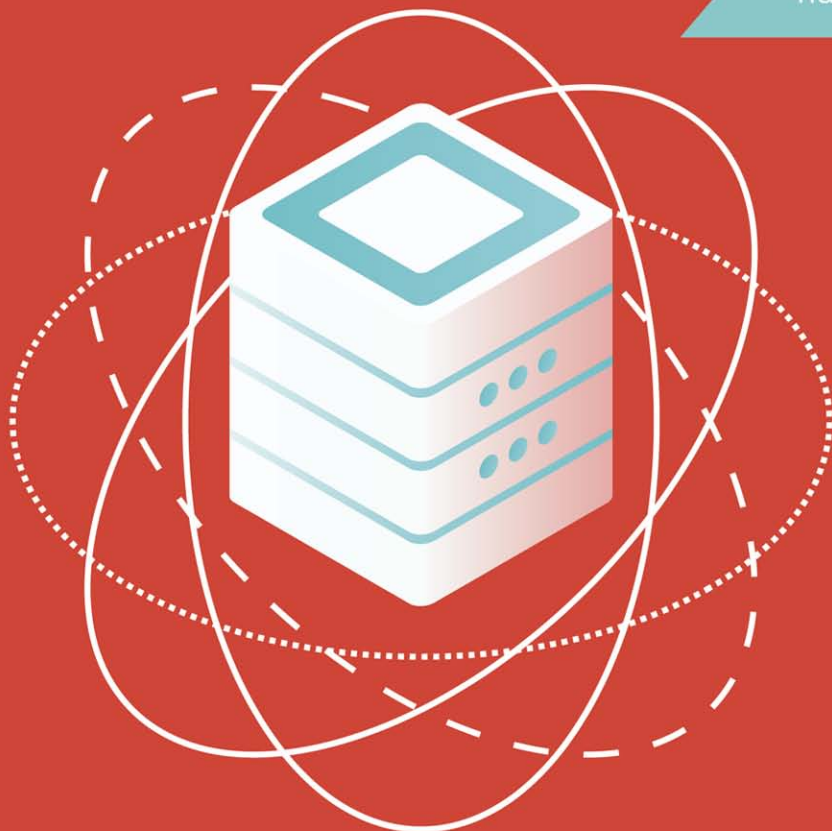


АНТОН ЖИЯНОВ

ОКОННЫЕ ФУНКЦИИ SQL

анализ данных
на практике



АНТОН ЖИЯНОВ

ОКОННЫЕ ФУНКЦИИ SQL



Издательство АСТ
Москва

УДК 004.43
ББК 32.973
Ж66

Жиянов, Антон.

Ж66 Оконные функции SQL. Анализ данных на практике / Антон Жиянов. — Москва: Издательство АСТ, 2024. — 254 с. — (Программирование для всех)

ISBN 978–5–17–158845–8

«Оконные функции SQL» — книга о мощном инструменте для анализа данных, который позволяет выполнять сложные вычисления и получать информацию о группах строк или результатах окон, но если вкратце — как делать классные аналитические отчеты без участия «экселя».

Вместе с Антоном Жияновым разберемся в основах SQL:

- что такое «окно» в SQL и как оно работает;
- про фреймы и как с ними работать;
- и конечно же выполним практические задания в песочнице.

Книга будет полезна как начинающим разработчикам, так и опытным специалистам, желающим расширить и закрепить свои знания в области оконных функций SQL.

УДК 004.43
ББК 32.973

ISBN 978–5–17–158845–8

© А. Жиянов, текст
© ООО Издательство «АСТ»

Введение

О книге

Кто-то однажды сказал, что если освоить оконные функции в SQL — жизнь уже никогда не будет прежней. В хорошем смысле. Задача книги помочь вам освоить «окна» без напряжения всех сил и головной боли от мудреных объяснений.

Помогут в этом три инструмента:

1. Картинки.
2. Картинки.
3. Картинки.

Шучу. Но картинок правда будет много, как и практических задачек. Не пропускайте их, чтобы надежно усвоить материал. Это важно. Если не разобраться в запросах с «окошками», выглядят они как-то так:

Как я читаю сложные запросы

Оывлар олырв алвыра ловырпоар алповщ
select воаржо воарв лоарыовор врыа ыр
аов аорпоы влоипв ловуфлм from ваоры
выдорыл лырлрва олваопр кызари where
аор вора ыдоарв апоры ыдовад дыова ловр
оарова овраовра вораопр order by ов авра
орвао враопрао шыл враова.



Книга состоит из трех частей:

1. В первой части мы научимся использовать оконные функции.
2. Во второй части погрузимся в нюансы оконных фреймов.
3. В третьей части дополнительно попрактикуемся.

Если полностью изучите первую часть — уже освоите оконные функции на более глубоком уровне, чем большинство ваших коллег и знакомых. И сможете спокойно применять в рабочих задачах. Вторая и третья части — для тех, кого «окошки» увлекут по-настоящему.

Базы данных

Оконные функции в той или иной степени поддерживаются во всех современных реляционных СУБД. Книга тестировалась на трех:

- MySQL 8.0.2+ (MariaDB 10.2+)
- PostgreSQL 11+
- SQLite 3.28+

Полнее всего окошки реализованы в PostgreSQL и SQLite. MySQL поддерживает основные возможности, но лишен некоторых более продвинутых. Oracle 11g+, MS SQL 2012+ и Google BigQuery поддерживают «окошки» примерно так же, как MySQL. Так что если вы используете одну из них — книгу тоже будет полезна.

Вы можете использовать любую из перечисленных СУБД, если она у вас под рукой. Если нет — в следующей главе я дам ссылку на онлайн-песочницу.

Вопросы и задания

Я стараюсь объяснять материал просто и наглядно. Но окна в SQL — сложная тема, поэтому важный момент: *задания* — неотъемлемая часть книги. Половину знаний вы получите именно из заданий. Поэтому старайтесь не пропускать их. Не бойтесь ошибаться.

Версия книги

Версия книги: 2023.05.29

Авторские права

© 2023 Антон Жиянов. Копирование, распространение, переработка, адаптация, перевод или любое преобразование материалов без письменного разрешения автора — запрещены.

Зачем нужны оконные функции

Если вкратце — оконные функции помогают делать классные аналитические отчеты без участия «экселя».

Проще всего объяснять на конкретных примерах. Будем работать с игрушечной таблицей сотрудников, вот такой:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Рассмотрим некоторые задачи, которые удобно решать с помощью «окошек» в SQL. Как именно их решать — разберемся в следующей главе. Пока просто оцен им возможности.

Если вам не терпится перейти к практике — эту главу можно пропустить.

Ранжирование

Ранжирование — это всевозможные рейтинги, начиная от призеров чемпионата мира по плаванию и заканчивая Forbes 500.

Мы будем ранжировать сотрудников.

Общий рейтинг зарплат

Составим рейтинг сотрудников по размеру заработной платы:

Столбец `rank` показывает позицию сотрудника в рейтинге.

Видно, что у некоторых коллег одинаковая зарплата (Леонид и Марина, Вероника и Григорий) — поэтому они получили один и тот же ранг.

rank	name	department	salary ↓
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Рейтинг зарплат по департаментам

Тот же рейтинг, только не для всей компании, а по каждому департаменту в отдельности:

rank	name	department	salary ↓
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Ксения	it	90
4	Елена	it	84
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96

Столбец rank показывает позицию сотрудника в рейтинге конкретного департамента.

Группы по зарплате

Разобьем сотрудников на три группы в зависимости от размера зарплаты:

- высокооплачиваемые,
- средние,
- низкооплачиваемые.

tile	name	department	salary ↓
1	Иван	it	120
1	Леонид	it	104
1	Марина	it	104
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96
2	Ксения	it	90
3	Елена	it	84
3	Борис	hr	78
3	Дарья	hr	70

Столбец `tile` показывает, к какой группе относится каждый сотрудник.

Самые «дорогие» коллеги

Найдем самых высокооплачиваемых людей по каждому департаменту:

id	name	department	salary
12	Bob	hr	78
25	Frank	it	120
33	Alice	sales	100

Что ж, этим зарплату больше не повышать.

Сравнение со смещением

Сравнение со смещением — это когда мы смотрим, в чем разница между соседними значениями. Например, сравниваем страны, которые занимают 5 и 6 место в мировом рейтинге ВВП — сильно ли отличаются? А если сравнить 1 и 6 место?

Сюда же попадают задачи, в которых мы сравниваем значение из набора с границами набора. Например, есть 100 лучших теннисисток мира. Мария Саккари занимает в рейтинге 10 место. Как ее показатели соотносятся со спортсменкой, которая занимает первое место? А с той, кто занимает последнее?

Мы будем сравнивать сотрудников.

Разница по зарплате с предыдущим

Упорядочим сотрудников по возрастанию зарплаты и проверим, велик ли разрыв между соседями:

name	department	salary ↑	diff
Дарья	hr	70	
Борис	hr	78	11%
Елена	it	84	8%
Ксения	it	90	7%
Вероника	sales	96	7%
Григорий	sales	96	0%
Анна	sales	100	4%
Леонид	it	104	4%
Марина	it	104	0%
Иван	it	120	15%

Столбец `diff` показывает, на сколько процентов зарплата сотрудника отличается от предыдущего коллеги. Видно, что больших разрывов нет. Самые крупные — между Дарьей и Борисом (11%) и Мариной и Иваном (15%).

Диапазон зарплат в департаменте

Посмотрим, как зарплата сотрудника соотносится с минимальной и максимальной зарплатой в его департаменте:

name	depart	salary ↑	low	high
Дарья	hr	70	70	78
Борис	hr	78	70	78
Елена	it	84	84	120
Ксения	it	90	84	120
Леонид	it	104	84	120
Марина	it	104	84	120
Иван	it	120	84	120
Вероника	sales	96	96	100
Григорий	sales	96	96	100
Анна	sales	100	96	100

Для каждого сотрудника столбец `low` показывает минимальную зарплату родного департамента, а столбец `high` — максимальную. Видно, что разброс значений в HR и продажах невелик, а у айтишников — значительный.

Агрегация

Агрегация — это когда мы считаем суммарные или средние показатели. Например, среднюю зарплату по каждому региону или количество золотых медалей у каждой страны в зачете Олимпийских игр.

Мы будем агрегировать зарплаты сотрудников.

Сравнение с фондом оплаты труда

У каждого департамента есть фонд оплаты труда — денежная сумма, которая ежемесячно уходит на выплату зарплат сотрудникам. Посмотрим, какой процент от этого фонда составляет зарплата каждого сотрудника:

name	depart	salary ↑	fund	perc
Дарья	hr	70	148	47%
Борис	hr	78	148	53%
Елена	it	84	502	17%
Ксения	it	90	502	18%
Леонид	it	104	502	21%
Марина	it	104	502	21%
Иван	it	120	502	24%
Вероника	sales	96	292	33%
Григорий	sales	96	292	33%
Анна	sales	100	292	34%

Столбец `fund` показывает фонд оплаты труда отдела, а `perc` — долю зарплаты сотрудника от этого фонда. Видно, что в HR и продажах все более-менее ровно, а у айтишников есть заметный разброс зарплат.

Сравнение со средней зарплатой

Интересно, велик ли разброс зарплат в департаментах. Проверим — посчитаем отклонение зарплаты каждого сотрудника от средней по департаменту:

name	depart	salary ↑	savg	diff
Дарья	hr	70	74	-5%
Борис	hr	78	74	5%
Елена	it	84	100	-16%
Ксения	it	90	100	-10%
Леонид	it	104	100	4%
Марина	it	104	100	4%
Иван	it	120	100	20%
Вероника	sales	96	97	-1%
Григорий	sales	96	97	-1%
Анна	sales	100	97	3%

Результат подтверждает предыдущие наблюдения: у ай-тишников зарплаты колеблются от -16% до +20% от среднего, а у остальных департаментов отклонение в пределах 5%.

Скользящие агрегаты

Скользящие агрегаты — это те же сумма и среднее. Только рассчитывают их не по всем элементам набора, а более хитрым способом.

Поясню на примере. Здесь возьмем другую таблицу — с доходами и расходами компании за 9 месяцев 2020 года:

year	month	income	expense
2020	1	94	82
2020	2	94	75
2020	3	94	104
2020	4	100	94
2020	5	100	99
2020	6	100	105
2020	7	100	95
2020	8	100	110
2020	9	104	104

Скользящее среднее по расходам

Судя по данным, доходы растут: 94 в январе → 104 в сентябре. А вот растут ли расходы? Сходу сложно сказать, месяц на месяц не приходится. Чтобы сгладить эти скачки, используют «скользящее среднее» — для каждого месяца рассчитывают средний расход с учетом предыдущего и следующего месяца. Например:

- скользящее среднее за февраль = (январь + февраль + март) / 3;

- за март = (февраль + март + апрель) / 3;
- за апрель = (март + апрель + май) / 3;
- и так далее.

Рассчитаем скользящее среднее по всем месяцам:

year ↑	month ↑	expense	roll_avg
2020	1	82	79
2020	2	75	87
2020	3	104	91
2020	4	94	99
2020	5	99	99
2020	6	105	100
2020	7	95	103
2020	8	110	103
2020	9	104	107

Теперь хорошо видно, что расходы стабильно растут.

Прибыль нарастающим итогом

Благодаря скользящему среднему, мы выяснили, что растут и доходы, и расходы. А как они соотносятся друг с другом? Хочется понять, находится ли компания «в плюсе» или «в минусе» с учетом всех заработанных и потраченных денег.

Причем важно понимать не на конец года, а на каждый месяц. Потому что если по итогам года все ОК, а в июне компания ушла в минус — это потенциальная проблема (такую ситуацию называют «кассовым разрывом»).

Поэтому посчитаем доходы и расходы по месяцам нарастающим итогом (кумулятивно):

- кумулятивный доход за январь = январь;
- за февраль = январь + февраль;

- за март = январь + февраль + март;
- за апрель = январь + февраль + март + апрель;
- и так далее.

month ↑	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75	188	157	31
3	94	104	282	261	21
4	100	94	382	355	27
5	100	99	482	454	28
6	100	105	582	559	23
7	100	95	682	654	28
8	100	110	782	764	18
9	104	104	886	868	18

Теперь видно, что дела у компании идут неплохо. В некоторых месяцах расходы превышают доходы, но благодаря накопленной «денежной подушке» кассового разрыва не происходит.

Резюме

Вот задачи, которые непринужденно решаются с помощью оконных функций в SQL:

- Ранжирование (всевозможные рейтинги).
- Сравнение со смещением (соседние элементы и границы).
- Агрегация (сумма и среднее).
- Скользящие агрегаты (сумма и среднее в динамике).

Конечно, это не исчерпывающий список. Но, надеюсь, теперь понятно, как пригодятся оконные функции в аналитике данных. В следующей главе разберемся, что такое «окна» и как их применять.

Песочница

Если будете работать в локальной базе — вот скрипт, который создает таблицы и данные:

```
https://antonz.ru/to#empsql
```

Если базы под рукой нет — используйте песочницу:

```
https://antonz.ru/to#empdb
```

Это SQLite, который работает прямо в браузере. Можете прямо сейчас попробовать выполнить в ней запрос:

```
select * from employees;
```

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

В книге много упражнений, где вы будете писать запросы — это можно делать в песочнице. После того как успешно решите задачу, сравните с эталонным решением.

Часть 1

Окна и функции

Ранжирование

Нет более обманчивого раздела SQL, чем «оконные функции». Когда слышишь эти слова, думаешь «наверно, просто придумали какие-то дополнительные функции». А вот и нет! «Оконные функции» — это отдельный язык, встроенный в обычный SQL. И он сложнее, чем все, что вы знали о «селектах», вместе взятое.

Вот как мы поступим:

- Разберемся, что такое «окно» в SQL и как оно работает.
- Посмотрим, как решать самые распространенные задачи с помощью оконных функций.

В этой главе будем решать задачу *ранжирования*. Ранжирование — это всевозможные рейтинги, начиная от призеров чемпионата мира по плаванию и заканчивая Forbes 500.

Мы же будем ранжировать сотрудников в игрушечной таблице `employees`:

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Оконная функция

Составим рейтинг сотрудников по размеру заработной платы:

было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
23	Леонид	it	104
24	Марина	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

стало

rank	name	department	salary ↓
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Обратите внимание — сотрудники с одинаковой зарплатой получили один и тот же ранг (Леонид и Марина, Вероника и Григорий).

Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по убыванию зарплаты:

```

select
  null as "rank",
  name, department, salary
from employees
order by salary desc, id;

```

Теперь пройдем от первой строчки до последней и поставим ранг каждой записи. Начнем с 1 и будем увеличивать ранг каждый раз, когда значение salary меньше, чем у предыдущей записи:

①

rank	name	department	salary ↓
1	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

②

rank	name	department	salary ↓
1	Иван	it	120
2	Леонид	it	104
	Марина	it	104
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

③

rank	name	department	salary ↓
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

④

rank	name	department	salary ↓
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

⑤

rank	name	department	salary ↓
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

и так далее...

Чтобы проставить ранг, достаточно на каждом шаге смотреть только на значения из столбца `salary`, выделенные синей рамкой. Назовем эти значения *окном*.

Попробуем описать содержимое окна словами:

- Это значения столбца `salary`.
- Они упорядочены от большего значения к меньшему.

Сформулируем то же самое на SQL:

```
window w as (order by salary desc)
```

- `window` — ключевое слово, которое показывает, что дальше будет определение окна;
- `w` — название окна (может быть любым);
- `(order by salary desc)` — описание окна («значения столбца `salary`, упорядоченные по убыванию»).

Задача — посчитать ранг по окну `w`. На SQL это записывается как `dense_rank() over w`.

`dense_rank()` — это оконная функция, которая считает ранг по указанному окну. Логика `dense_rank()` такая же, как была у нас при ручном подсчете — начать с 1 и увеличивать ранг каждый раз, когда очередное значение из окна отличается от предыдущего.

Добавим окно и оконную функцию в исходный запрос:

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as (order by salary desc)
order by "rank", id;
```


rank	name	department	salary
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Вот как движок выполняет такой запрос:

1. Берет таблицу, указанную в `from`.
2. Выбирает из нее все записи.
3. Для каждой записи рассчитывает значение `dense_rank()` с помощью окна `w`.
4. Сортирует результат как указано в `order by`.

Вот как отрабатывает шаг 3, на котором назначается ранг:

rank	name	department	salary
Ранжирование: ранг			

Смотреть: <https://antonz.ru/to#w22rnk>

Конструкция `window` сама по себе ничего не делает с результатами запроса. Она только определяет окно, которое можно использовать (или не использовать) в запросе. Если убрать вызов `dense_rank()`, запрос отработает, как будто нет никаких окон:

```
select
  null as "rank",
  name, department, salary
from employees
window w as (order by salary desc)
order by salary desc, id;
```

rank	name	department	salary
	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

Окно начинает работать только тогда, когда в `select` появляется оконная функция, которая его использует.

Оконные запросы в СУБД Oracle и MS SQL Server

Ни Oracle, ни SQL Server не поддерживают конструкцию `window`. Чтобы заставить работать оконный запрос в этих СУБД, перенесите описание окна внутрь инструкции `over`.

Не так:

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as (order by salary desc)
order by "rank", id;
```

А так:

```
select
  dense_rank() over (
    order by salary desc
  ) as "rank",
  name, department, salary
from employees
order by "rank", id;
```

Сортировка окна и сортировка результатов

При первом взгляде на «окошки» у людей часто возникают вопросы. Давайте их разберем.

Вот запрос, который считает рейтинг зарплат:

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as (order by salary desc)
order by "rank", id;
```

rank	name	department	salary
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Оставим `order by` в окне, но уберем в основном запросе:

select

`dense_rank() over w as "rank",
name, department, salary`

from employees

window w **as** (**order by** salary **desc**);

Ничего не изменилось:

rank	name	department	salary
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Получается, `order by` в основном запросе не нужен? Не совсем так.

`order by` в окне задает сортировку окна, а `order by` в основном запросе — сортировку результатов уже после того, как отработало и окно, и все прочие части запроса.

Допустим, мы хотим проставить ранг по убыванию зарплаты, а отсортировать — наоборот, по возрастанию:

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as (order by salary desc)
order by salary asc;
```

rank	name	department	salary
8	Дарья	hr	70
7	Борис	hr	78
6	Елена	it	84
5	Ксения	it	90
4	Вероника	sales	96
4	Григорий	sales	96
3	Анна	sales	100
2	Леонид	it	104
2	Марина	it	104
1	Иван	it	120

Видно, что ранг проставился по сортировке окна (`salary desc`), а результат упорядочен по сортировке основного запроса (`salary asc`).

Если вообще не указать `order by` запроса — порядок записей будет не определен. Иногда может повезти, как в запросе

с рангом по возрастанию зарплаты, а иногда и нет. Полагаться на везение не стоит, поэтому всегда указывайте явно сортировку результатов.

Однозначность сортировки

Еще один частый вопрос — зачем в сортировке результата столбец `id`:

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as (order by salary desc)
order by "rank", id;
```

rank	name	department	salary
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Почему `order by rank, id`, а не `order by rank`? Чтобы знать, как сортировать сотрудников с одинаковым рангом. Без `id` порядок записей Леонид–Марина и Вероника–Григорий не

определен, и СУБД может расположить их в любом порядке. А с `id` все однозначно: «Леонид, затем Марина» и «Вероника, затем Григорий».

Несколько окон

Еще один частый вопрос (не связанный с сортировкой) — как задать несколько окон в одном запросе.

Просто перечислите их через запятую в разделе `window`. Например, отранжируем сотрудников по зарплате в прямом и обратном порядке:

```
select
  dense_rank() over w1 as r_asc,
  dense_rank() over w2 as r_desc,
  name, salary
from employees
window
  w1 as (order by salary asc),
  w2 as (order by salary desc)
order by salary, id;
```

r_asc	r_desc	name	salary
1	8	Дарья	70
2	7	Борис	78
3	6	Елена	84
4	5	Ксения	90
5	4	Вероника	96
5	4	Григорий	96
6	3	Анна	100
7	2	Леонид	104
7	2	Марина	104
8	1	Иван	120

Ранг по имени

Символ  обозначает задания. Это неотъемлемая часть книги. Старайтесь не пропускать их — половину знаний вы получите именно из заданий.

Предположим, мы хотим ранжировать сотрудников по имени (по алфавиту от А к Я):

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as ...
order by "rank", id;
```

Что должно быть вместо ... ?

Выберите один вариант:

1. (order by salary desc)
2. (order by name desc)
3. (order by name)
4. (order by rank)

Песочница: <https://antonz.ru/to#empdb>

☒ Ранг по имени

Символ ☒ обозначает решения заданий.

Ранжирование по name от А к Я требует сортировки по name по возрастанию.

```
(order by name)
```

Запрос:

```
select
  dense_rank() over w as "rank",
  name, department, salary
```



```

from employees
window w as (order by name)
order by "rank", id;

```

Результат:

rank	name	department	salary
1	Анна	sales	100
2	Борис	hr	78
3	Вероника	sales	96
4	Григорий	sales	96
5	Дарья	hr	70
6	Елена	it	84
7	Иван	it	120
8	Ксения	it	90
9	Леонид	it	104
10	Марина	it	104

Секции окна

Теперь составим рейтинг сотрудников по размеру заработной платы независимо по каждому департаменту:

было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
23	Леонид	it	104
24	Марина	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

стало

rank	name	department	salary ↓
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Ксения	it	90
4	Елена	it	84
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96

Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по департаментам, а внутри департамента — по убыванию зарплаты:

```
select
  null as "rank",
  name, department, salary
from employees
order by department, salary desc, id;
```

Теперь пройдем от первой строчки до последней и поставим ранг каждой записи. Начнем с 1 и будем увеличивать ранг каждый раз, когда значение salary меньше, чем у предыдущей записи. При переходе от департамента к департаменту будем сбрасывать ранг обратно на 1:

①

rank	name	department	salary ↓
1	Борис	hr	78
	Дарья	hr	70
	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Ксения	it	90
	Елена	it	84
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96

②

rank	name	department	salary ↓
1	Борис	hr	78
2	Дарья	hr	70
	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Ксения	it	90
	Елена	it	84
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96

③

rank	name	department	salary ↓
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Ксения	it	90
	Елена	it	84
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96

④

rank	name	department	salary ↓
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
	Марина	it	104
	Ксения	it	90
	Елена	it	84
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96

⑤

rank	name	department	salary ↓
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
	Ксения	it	90
	Елена	it	84
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96

и так далее...

Чтобы проставить ранг, достаточно на каждом шаге смотреть только на значения из столбца `salary`, выделенные синей рамкой. Это и есть окно в данном случае.

Видно, что окно меняется в зависимости от того, к какому департаменту относится текущая запись. Опишем словами:

1. Окно разбито на несколько независимых секций — по одной на департамент.
2. Внутри секции записи упорядочены по убыванию зарплаты.

Сформулируем то же самое на SQL:

```

window w as (
  partition by department
  order by salary desc
)

```

1. `partition by department` указывает, как следует разбить окно на секции;
2. `order by salary desc` задает сортировку внутри секции.

Функция расчета ранга остается прежней — `dense_rank()`.

Добавим окно и оконную функцию в исходный запрос:

```

select
  dense_rank() over w as "rank",
  name, department, salary
from employees

```

```

window w as (
  partition by department
  order by salary desc
)
order by department, "rank", id;

```

rank	name	department	salary
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Ксения	it	90
4	Елена	it	84
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96

Разделители я добавил вручную для наглядности.

Вот как движок рассчитывает ранг для каждой записи:

rank	name	department	salary
Ранжирование: ранг по секциям			

Смотреть: <https://antonz.ru/to#w22prt>

🔪 Рейтинг зарплат по городам

В компании работают сотрудники из Москвы и Самары. Предположим, мы решили ранжировать их по зарплате внутри каждого города, от меньшей зарплаты к большей.

Посчитаем рейтинг:

```
select
  dense_rank() over w as "rank",
  city, name, salary
from employees
window w as ...
order by city, "rank", id;
```

rank	city	name	salary
1	Москва	Ксения	90
2	Москва	Вероника	96
3	Москва	Анна	100
4	Москва	Марина	104
5	Москва	Иван	120
1	Самара	Дарья	70
2	Самара	Борис	78
3	Самара	Елена	84
4	Самара	Григорий	96
5	Самара	Леонид	104

Что должно быть вместо ...?

Выберите один вариант:

1. (partition by city order by salary)
2. (group by city order by salary)
3. (partition by salary order by city)
4. (order by city, salary)

Песочница: <https://antonz.ru/to#empdb>

☑ Рейтинг зарплат по городам

Секции по `city` обеспечат независимый рейтинг внутри каждого города, а сортировка по `salary` — ранжирование по зарплате.

```
(partition by city order by salary)
```

Запрос:

```
select
    dense_rank() over w as "rank",
    city, name, salary
from employees
window w as (
    partition by city
    order by salary
)
order by city, "rank", id;
```

Результат:

rank	city	name	salary
1	Москва	Ксения	90
2	Москва	Вероника	96
3	Москва	Анна	100
4	Москва	Марина	104
5	Москва	Иван	120
1	Самара	Дарья	70
2	Самара	Борис	78
3	Самара	Елена	84
4	Самара	Григорий	96
5	Самара	Леонид	104

Группы

Разобьем сотрудников на три группы в зависимости от размера зарплаты:

- высокооплачиваемые,
- средние,
- низкооплачиваемые.

было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
23	Леонид	it	104
24	Марина	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

стало

tile	name	department	salary ↓
1	Иван	it	120
1	Леонид	it	104
1	Марина	it	104
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96
2	Ксения	it	90
3	Елена	it	84
3	Борис	hr	78
3	Дарья	hr	70

Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по убыванию зарплаты:

```
select
  null as tile,
  name, department, salary
from employees
order by salary desc, id;
```

tile	name	department	salary
	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

Всего 10 записей и 3 группы — значит, две группы по 3 записи и одна 4 записи. Например, так:

- Иван, Леонид, Марина, Анна
- Вероника, Григорий, Ксения
- Елена, Борис, Дарья

Чтобы провести границы между группами, придется анализировать все зарплаты, отсортированные по убыванию. Поэтому окно будет таким:

```
window w as (order by salary desc)
```

А вот функция потребуется другая — `ntile(n)`, где `n` — количество групп. В нашем случае `n = 3`:

```
select  
  ntile(3) over w as tile,  
  name, department, salary  
from employees  
window w as (order by salary desc)  
order by salary desc, id;
```

tile	name	department	salary
1	Иван	it	120
1	Леонид	it	104
1	Марина	it	104
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96
2	Ксения	it	90
3	Елена	it	84
3	Борис	hr	78
3	Дарья	hr	70

Разделители между группами я добавил вручную для наглядности.

`ntile(n)` разбивает все записи на n групп и возвращает номер группы для каждой записи. Если общее количество записей (10 в нашем случае) не делится на размер группы (3), то первые группы будут крупнее последних.

`ntile()` всегда старается разбить данные так, чтобы группы были одинакового размера. Поэтому записи с одинаковым значением z/p вполне могут попасть в разные (соседние) группы:

```
select
  ntile(2) over w as tile,
  name, department, salary
from employees
window w as (order by salary desc, id)
order by salary desc, tile;
```

tile	name	department	salary
1	Иван	it	120
1	Леонид	it	104
1	Марина	it	104
1	Анна	sales	100
1	Вероника	sales	96
2	Григорий	sales	96
2	Ксения	it	90
2	Елена	it	84
2	Борис	hr	78
2	Дарья	hr	70

← (!)

← (!)

Группы по зарплате в городах

В компании работают сотрудники из Москвы и Самары. Мы хотим разбить их на две группы по зарплате в каждом из городов, вот так:

tile	name	city	salary
1	Ксения	Москва	90
1	Вероника	Москва	96
1	Анна	Москва	100
2	Марина	Москва	104
2	Иван	Москва	120
1	Дарья	Самара	70
1	Борис	Самара	78
1	Елена	Самара	84
2	Григорий	Самара	96
2	Леонид	Самара	104

Разделители между группами я добавил вручную для наглядности.

Напишите запрос, который это сделает.

Сортировка результата: `city, salary`

Песочница: <https://antonz.ru/to#empdb>

☒ Группы по зарплате в городах

Секция по городу, сортировка по зарплате.

```
select
  ntile(2) over w as tile,
  name, city, salary
from employees
window w as (
```

```

partition by city
order by salary
)
order by city, salary, id;

```

tile	name	city	salary
1	Ксения	Москва	90
1	Вероника	Москва	96
1	Анна	Москва	100
2	Марина	Москва	104
2	Иван	Москва	120
1	Дарья	Самара	70
1	Борис	Самара	78
1	Елена	Самара	84
2	Григорий	Самара	96
2	Леонид	Самара	104

Самые «дорогие» коллеги

Предположим, мы хотим узнать самых высокооплачиваемых людей по каждому департаменту:

id	name	department	salary
12	Борис	hr	78
25	Иван	it	120
33	Анна	sales	100

Напишите запрос, который это сделает.

Подсказка. Сначала проставьте правильный ранг каждой записи, а затем используйте CTE (<https://antonz.ru/cte>) или подзапрос, чтобы выбрать искомые три.

Песочница: <https://antonz.ru/to#empdb>

☑ Самые «дорогие» коллеги

1. Считаю ранг: секция по департаменту, сортировка по убыванию зарплаты.
2. Берем сотрудников с рангом = 1.

```
with data as (  
    select  
        id, name, department, salary,  
        dense_rank() over w as emp_rank  
    from employees  
    window w as (  
        partition by department  
        order by salary desc  
    )  
)  
select  
    id, name, department, salary  
from data  
where emp_rank = 1;
```

id	name	department	salary
12	Борис	hr	78
25	Иван	it	120
33	Анна	sales	100

Функции ранжирования

Оконные функции ранжирования:

Функция	Описание
<code>row_number()</code>	порядковый номер строки <code>number</code>
<code>dense_rank()</code>	ранг строки <code>rank</code>
<code>rank()</code>	ранг строки, но с пропусками (см. ниже)
<code>ntile(n)</code>	разбивает все строки на <code>n</code> групп и возвращает номер группы, в которую попала строка

`dense_rank()` и `ntile()` мы уже разобрали.

`row_number()` нумерует строки в порядке, указанном в `order by`. Никаких неожиданностей.

`rank()` похож на `dense_rank()`, а разницу проще всего показать на примере.

```
select
    ... over w as "rank",
    name, salary
from employees
window w as (order by salary desc)
order by "rank", id;
```

В одном случае вместо `...` укажем `dense_rank()`, а в другом — `rank()`:

`dense_rank()` назначает Анне третье место, а `rank()` — четвертое, потому что второе–третье уже заняты Леонидом и Мариной. Аналогично с Ксенией после Вероники и Григория. Вот и вся разница.

dense_rank()

rank	name	salary
1	Иван	120
2	Леонид	104
2	Марина	104
3	Анна	100
4	Вероника	96
4	Григорий	96
5	Ксения	90
6	Елена	84
7	Борис	78
8	Дарья	70

(!)

(!)

rank()

rank	name	salary
1	Иван	120
2	Леонид	104
2	Марина	104
4	Анна	100
5	Вероника	96
5	Григорий	96
7	Ксения	90
8	Елена	84
9	Борис	78
10	Дарья	70

(!)

(!)

Так держать

Вы узнали, что такое «окно», «оконная функция», и как использовать их для ранжирования данных. В следующей главе займемся оконными сравнениями!

Сравнение со смещением

Сравнение со смещением — это когда мы смотрим, в чем разница между соседними значениями. Например, сравниваем страны, которые занимают 5 и 6 место в мировом рейтинге ВВП — сильно ли отличаются? А если сравнить 1 и 6 место?

Сюда же попадают задачи, в которых мы сравниваем значение из набора с границами набора. Например, есть 100 лучших теннисисток мира. Мария Саккари занимает в рейтинге 10 место. Как ее показатели соотносятся со спортсменкой, которая занимает 1 место? А с той, кто занимает 100 место?

Мы будем сравнить сотрудников из таблицы `employees`:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

Сравнение с соседями

Упорядочим сотрудников по возрастанию зарплаты и проверим, велик ли разрыв между соседями:

было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
23	Леонид	it	104
24	Марина	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

стало

name	department	salary ↑	diff
Дарья	hr	70	
Борис	hr	78	11%
Елена	it	84	8%
Ксения	it	90	7%
Вероника	sales	96	7%
Григорий	sales	96	0%
Анна	sales	100	4%
Леонид	it	104	4%
Марина	it	104	0%
Иван	it	120	15%

Столбец `diff` показывает, на сколько процентов зарплата сотрудника отличается от предыдущего коллеги. Видно, что больших разрывов нет. Самые крупные — между Дарьей и Борисом (11%) и Мариной и Иваном (15%).

Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по возрастанию зарплаты:

```
select
    name, department, salary,
    null as prev
from employees
order by salary, id;
```

Теперь пройдем от первой строчки до последней, на каждом шаге «подтягивая» зарплату предыдущего сотрудника:

1

name	department	salary ↑	prev
Дарья	hr	70	
Борис	hr	78	
Елена	it	84	
Ксения	it	90	
Вероника	sales	96	
Григорий	sales	96	
Анна	sales	100	
Леонид	it	104	
Марина	it	104	
Иван	it	120	

2

name	department	salary ↑	prev
Дарья	hr	70	
Борис	hr	78	70
Елена	it	84	
Ксения	it	90	
Вероника	sales	96	
Григорий	sales	96	
Анна	sales	100	
Леонид	it	104	
Марина	it	104	
Иван	it	120	

3

name	department	salary ↑	prev
Дарья	hr	70	
Борис	hr	78	70
Елена	it	84	78
Ксения	it	90	
Вероника	sales	96	
Григорий	sales	96	
Анна	sales	100	
Леонид	it	104	
Марина	it	104	
Иван	it	120	

4

name	department	salary ↑	prev
Дарья	hr	70	
Борис	hr	78	70
Елена	it	84	78
Ксения	it	90	84
Вероника	sales	96	
Григорий	sales	96	
Анна	sales	100	
Леонид	it	104	
Марина	it	104	
Иван	it	120	

5

name	department	salary ↑	prev
Дарья	hr	70	
Борис	hr	78	70
Елена	it	84	78
Ксения	it	90	84
Вероника	sales	96	90
Григорий	sales	96	
Анна	sales	100	
Леонид	it	104	
Марина	it	104	
Иван	it	120	

и так далее...

и так далее...

Одной gifкой:

name	department	salary	prev
Смещение: сравнение с предыдущим			

Смотреть: <https://antonz.ru/to#w23lag>

Видно, что окно в данном случае охватывает текущую и предыдущую запись. Оно сдвигается вниз на каждом шаге, скользит. Это логичная трактовка происходящего, и задать скользящее окно в SQL можно. Но у таких окон более сложный синтаксис, поэтому отложим их до главы о скользящих агрегатах.

Вместо этого возьмем более простое и знакомое нам окно — все записи, упорядоченные по возрастанию salary:

name	department	salary ↑	prev
Дарья	hr	70	
Борис	hr	78	
Елена	it	84	
Ксения	it	90	
Вероника	sales	96	
Григорий	sales	96	
Анна	sales	100	
Леонид	it	104	
Марина	it	104	
Иван	it	120	

А чтобы на каждом шаге подтягивать зарплату предыдущего сотрудника, будем использовать оконную функцию `lag()`:

```
lag(salary, 1) over w
```

Функция `lag()` возвращает значение из указанного столбца, отстоящее от текущего на указанное количество записей назад. В нашем случае — `salary` от предыдущей записи.

Добавим окно и оконную функцию в исходный запрос:

```
select
    id, name, department, salary,
    lag(salary, 1) over w as prev
from employees
window w as (order by salary, id)
order by salary, id;
```

id	name	department	salary	prev
11	Дарья	hr	70	
12	Борис	hr	78	70
21	Елена	it	84	78
22	Ксения	it	90	84
31	Вероника	sales	96	90
32	Григорий	sales	96	96
33	Анна	sales	100	96
23	Леонид	it	104	100
24	Марина	it	104	104
25	Иван	it	120	104

Столбец `prev` показывает зарплату предыдущего сотрудника. Осталось посчитать разницу между `prev` и `salary` в процентах:

```
with emp as (
    select
        id, name, department, salary,
        lag(salary, 1) over w as prev
```

```

    from employees
    window w as (order by salary, id)
)
select
    name, department, salary,
    round((salary - prev)*100.0 / prev) as diff
from emp
order by salary, id;

```

name	department	salary	diff
Дарья	hr	70	
Борис	hr	78	11
Елена	it	84	8
Ксения	it	90	7
Вероника	sales	96	7
Григорий	sales	96	0
Анна	sales	100	4
Леонид	it	104	4
Марина	it	104	0
Иван	it	120	15

Можно избавиться от промежуточной таблицы emp, подставив вместо prev вызов оконной функции:

```

select
    name, department, salary,
    round(
        (salary - lag(salary, 1) over w)*100.0 /
        lag(salary, 1) over w
    ) as diff
from employees
window w as (order by salary, id)
order by salary, id;

```

Здесь мы заменили `prev` → `lag(salary, 1) over w`. Конструкцию вида `function_name(...)` `over window_name` движок заменяет на конкретное значение, которое вернула функция. Так что оконную функцию можно вызывать прямо внутри вычислений, и вы не раз встретите такие запросы в документации и примерах.

🔪 Зарплата соседнего сотрудника

Есть таблица сотрудников `employees`. Мы хотим для каждого сотрудника увидеть зарплаты предыдущего и следующего коллеги:

name	department	prev	salary	next
Дарья	hr		70	78
Борис	hr	70	78	84
Елена	it	78	84	90
Ксения	it	84	90	96
Вероника	sales	90	96	96
Григорий	sales	96	96	100
Анна	sales	96	100	104
Леонид	it	100	104	104
Марина	it	104	104	120
Иван	it	104	120	

Напишите запрос, который это сделает.

Чтобы получить зарплату следующего сотрудника, используйте функцию `lead()`. Она работает точно так же как `lag()`, только «заглядывает» не назад, а вперед.

Сортировка результата: `salary, id`

Пустые ячейки у Дарьи и Ивана — это `null`.

Песочница: <https://antonz.ru/to#empdb>

☑ Зарплата соседнего сотрудника

Окно отсортировано по зарплате. `lag()` выбирает зарплату предыдущего сотрудника, `lead()` — следующего.

```
select
    name, department,
    lag(salary, 1) over w as prev,
    salary,
    lead(salary, 1) over w as next
from employees
window w as (order by salary)
order by salary, id;
```

name	department	prev	salary	next
Дарья	hr		70	78
Борис	hr	70	78	84
Елена	it	78	84	90
Ксения	it	84	90	96
Вероника	sales	90	96	96
Григорий	sales	96	96	100
Анна	sales	96	100	104
Леонид	it	100	104	104
Марина	it	104	104	120
Иван	it	104	120	

Сравнение с границами

Посмотрим, как зарплата сотрудника соотносится с минимальной и максимальной зарплатой в его департаменте:

Для каждого сотрудника столбец `low` показывает минимальную зарплату родного департамента, а столбец `high` — максимальную.

было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
23	Леонид	it	104
24	Марина	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

стало

name	depart	salary ↑	low	high
Дарья	hr	70	70	78
Борис	hr	78	70	78
Елена	it	84	84	120
Ксения	it	90	84	120
Леонид	it	104	84	120
Марина	it	104	84	120
Иван	it	120	84	120
Вероника	sales	96	96	100
Григорий	sales	96	96	100
Анна	sales	100	96	100

Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по департаментам, а внутри департамента — по возрастанию зарплаты:

```
select
    name, department, salary,
    null as low,
    null as high
from employees
order by department, salary, id;
```

Теперь пройдем от первой строчки до последней, на каждом шаге «подтягивая» наименьшую и наибольшую зарплаты в отделе:

①

name	depart	salary ↑	low	high
Дарья	hr	70	70	78
Борис	hr	78		
Елена	it	84		
Ксения	it	90		
Леонид	it	104		
Марина	it	104		
Иван	it	120		
Вероника	sales	96		
Григорий	sales	96		
Анна	sales	100		

②

name	depart	salary ↑	low	high
Дарья	hr	70	70	78
Борис	hr	78	70	78
Елена	it	84		
Ксения	it	90		
Леонид	it	104		
Марина	it	104		
Иван	it	120		
Вероника	sales	96		
Григорий	sales	96		
Анна	sales	100		

③

name	depart	salary	↑	low	high
Дарья	hr	70		70	78
Борис	hr	78		70	78
Елена	it	84		84	120
Ксения	it	90			
Леонид	it	104			
Марина	it	104			
Иван	it	120			
Вероника	sales	96			
Григорий	sales	96			
Анна	sales	100			

④

name	depart	salary	↑	low	high
Дарья	hr	70		70	78
Борис	hr	78		70	78
Елена	it	84		84	120
Ксения	it	90		84	120
Леонид	it	104			
Марина	it	104			
Иван	it	120			
Вероника	sales	96			
Григорий	sales	96			
Анна	sales	100			

⑤

name	depart	salary	↑	low	high
Дарья	hr	70		70	78
Борис	hr	78		70	78
Елена	it	84		84	120
Ксения	it	90		84	120
Леонид	it	104		84	120
Марина	it	104			
Иван	it	120			
Вероника	sales	96			
Григорий	sales	96			
Анна	sales	100			

и так далее...

Одной gifкой:

name	depart	salary	low	high
Смещение: границы секции				

Смотреть: <https://antonz.ru/to#w23nth>

Окно состоит из трех секций. Секция на каждом шаге охватывает весь департамент сотрудника. Записи при этом упорядочены по возрастанию зарплаты внутри департамента, чтобы минимальная и максимальная зарплаты всегда находились на границах секции:

```
window w as (  
  partition by department  
  order by salary  
)
```

Хотелось бы воспользоваться функциями `lag()` и `lead()`, чтобы получить диапазон зарплат в отделе. Но они заглядывают на фиксированное количество записей назад или вперед. Нам же нужно нечто другое:

- `low` — зарплата первого сотрудника, входящего в секцию окна;
- `high` — зарплата последнего сотрудника, входящего в секцию.

К счастью, есть оконные функции ровно для этого:

```
first_value(salary) over w as low,  
last_value(salary) over w as high
```

Добавим окно и оконную функцию в исходный запрос:

```
select  
  name, department, salary,  
  first_value(salary) over w as low,  
  last_value(salary) over w as high  
from employees  
window w as (  
  partition by department  
  order by salary  
)  
order by department, salary, id;
```


name	department	salary	low	high
Дарья	hr	70	70	70
Борис	hr	78	70	78
Елена	it	84	84	84
Ксения	it	90	84	90
Леонид	it	104	84	104
Марина	it	104	84	104
Иван	it	120	84	120
Вероника	sales	96	96	96
Григорий	sales	96	96	96
Анна	sales	100	96	100

`low` рассчитался корректно, а вот с `high` какая-то ерунда. Вместо того чтобы равняться максимальной зарплате департамента, он меняется от сотрудника к сотруднику. Что ж, давайте разбираться.

Окно, секция, фрейм

До сих пор все было логично:

- есть окно, которое состоит из одной или нескольких секций;
- внутри секции записи упорядочены по конкретному столбцу.

На предыдущем шаге мы разделили окно на три секции — по департаментам, и упорядочили записи внутри секций по зарплате:

```

window w as (
  partition by department
  order by salary
)

```

Допустим, движок выполняет запрос, и текущая запись — Леонид из `it`-отдела. Мы ожидаем, что `first_value()` вернет первую запись `it`-секции (`salary = 84`), а `last_value()` — последнюю (`salary = 120`):

ожидание

hr	
Дарья	70
Борис	78
it	
Елена	84
Ксения	90
Леонид	104
Марина	104
Иван	120
sales	
Вероника	96
Григорий	96
Анна	100

← first_value()

← last_value()

реальность

hr	
Дарья	70
Борис	78
it	
Елена	84
Ксения	90
Леонид	104
Марина	104
Иван	120
sales	
Вероника	96
Григорий	96
Анна	100

← first_value()

← last_value()

Дело в том, что функции `first_value()` и `last_value()` работают не просто с секцией окна. Они работают с фреймом внутри секции:

employees

hr	
Дарья	70
Борис	78
it	
Елена	84
Ксения	90
Леонид	104
Марина	104
Иван	120
sales	
Вероника	96
Григорий	96
Анна	100

← начало фрейма

← текущая запись

← конец фрейма

Фрейм находится в той же секции, где текущая запись (Леонид):

- начало фрейма = начало секции (Елена);
- конец фрейма = последняя запись со значением `salary`, равным текущей записи (Марина).

Где заканчивается фрейм

Если с началом фрейма все понятно, то с концом у людей часто возникают вопросы. Рассмотрим несколько примеров, чтобы стало понятнее. Текущая запись во всех примерах — Леонид.

Елена	84	←	начало фрейма
Ксения	90		
Леонид	104	←	текущая запись
Марина	104	←	конец фрейма
Иван	120		

Конец фрейма — последняя запись со значением зарплаты, равным текущей записи. Текущая запись — Леонид с зарплатой 104. Последняя запись с зарплатой 104 — Марина. Значит, конец фрейма — Марина.

Елена	84	←	начало фрейма
Ксения	90		
Леонид	104	←	текущая запись и конец фрейма
Марина	110		
Иван	120		

Допустим, Марине повысили зарплату до 110. Текущая запись — Леонид с зарплатой 104. Последняя запись с зарплатой 104 — тоже Леонид. Значит, конец фрейма — Леонид.

Елена	84	←	начало фрейма
Ксения	90		
Леонид	104	←	текущая запись
Марина	104		
Иван	104	←	конец фрейма

Допустим, Ивану понизили зарплату до 104. Текущая запись — Леонид с зарплатой 104. Последняя запись с зарплатой 104 — Иван. Значит, конец фрейма — Иван.

Секция фиксирована, фрейм же зависит от текущей записи и постоянно меняется:

Секция	Фрейм
name department salary	name department salary
Секция	Фрейм

<https://antoz.ru/to#w23prt>

<https://antonz.ru/to#w23frm>

`first_value()` возвращает первую строчку фрейма, а не секции. Но поскольку начало фрейма совпадает с началом секции, функция отрабатывает как мы ожидали.

`last_value()` возвращает последнюю строчку фрейма, а не секции. Именно поэтому в нашем запросе она вернула не максимальную зарплату для каждого отдела, а какую-то ерунду.

Чтобы `last_value()` работала как мы ожидаем, придется «прибить» границы фрейма к границам секции. Тогда для каждой секции фрейм будет в точности совпадать с ней:

employees

hr

Дарья	70
Борис	78

it

Елена	84
Ксения	90
Леонид	104
Марина	104
Иван	120

sales

Вероника	96
Григорий	96
Анна	100

employees

hr

Дарья	70
Борис	78

it

Елена	84
Ксения	90
Леонид	104
Марина	104
Иван	120

sales

Вероника	96
Григорий	96
Анна	100

employees

hr

Дарья	70
Борис	78

it

Елена	84
Ксения	90
Леонид	104
Марина	104
Иван	120

sales

Вероника	96
Григорий	96
Анна	100

Подытожим, как работают `first_value()` и `last_value()`:

1. Есть окно, которое состоит из одной или нескольких секций (`partition by department`).
2. Внутри секции записи упорядочены по конкретному столбцу (`order by salary`).
3. У каждой записи в секции свой фрейм. По умолчанию начало фрейма совпадает с началом секции, а конец для каждой записи свой.
4. Конец фрейма можно приклеить к концу секции, чтобы фрейм в точности совпадал с секцией.
5. Функция `first_value()` возвращает значение из первой строки фрейма.
6. Функция `last_value()` возвращает значение из последней строки фрейма.

Теперь разберемся, как прибить фрейм к окну — и закончим с запросом по диапазону зарплат в департаментах.

Если не совсем поняли, что такое фрейм и как он формируется — ничего страшного. Фреймы — одна из самых сложных тем в «окошках», и за один прием их не объяснить. Мы будем изучать фреймы на протяжении всей книги, и постепенно все разберем.

Сравнение с границами, окончание

Возьмем наше окно:

```
window w as (  
    partition by department  
    order by salary  
)
```

И настроим его, чтобы фрейм в точности совпадал с секцией (департаментом):

```
window w as (  
    partition by department
```

```

order by salary
rows between unbounded preceding and unbounded
following
)

```

Не будем сейчас разбирать конструкцию `rows between` — ее время придет в главе про скользящие агрегаты. Важно, что благодаря ей фрейм совпадает с секцией, а значит `last_value()` вернет максимальную зарплату по департаменту:

```

select
  name, department, salary,
  first_value(salary) over w as low,
  last_value(salary) over w as high
from employees
window w as (
  partition by department
  order by salary
  rows between unbounded preceding and unbounded
following
)
order by department, salary, id;

```

name	department	salary	low	high
Дарья	hr	70	70	78
Борис	hr	78	70	78
Елена	it	84	84	120
Ксения	it	90	84	120
Леонид	it	104	84	120
Марина	it	104	84	120
Иван	it	120	84	120
Вероника	sales	96	96	100
Григорий	sales	96	96	100
Анна	sales	100	96	100

Теперь движок заполняет low и high так же, как мы делали это вручную:

name	depart	salary	low	high
Смещение: границы секции				

Смотреть: <https://antonz.ru/to#w23nth>

✍ Процент от максимальной зарплаты в городе

Есть таблица сотрудников `employees`. Мы хотим для каждого сотрудника увидеть, сколько процентов составляет его зарплата от максимальной в городе:

name	city	salary	percent
Ксения	Москва	90	75
Вероника	Москва	96	80
Анна	Москва	100	83
Марина	Москва	104	87
Иван	Москва	120	100
Дарья	Самара	70	67
Борис	Самара	78	75
Елена	Самара	84	81
Григорий	Самара	96	92
Леонид	Самара	104	100

Напишите запрос, который это сделает.

Для округления в столбце `percent` используйте функцию `round()`.

Сортировка результата: `city, salary`

Песочница: <https://antonz.ru/to#empdb>

☒ Процент от максимальной зарплаты в городе

Окно:

- секция по городу,
- сортировка по зарплате,
- фрейм совпадает с секцией.

Максимальная зарплата — это `last_value(salary)` по фрейму.

Искомый процент — `salary / last_value(salary)`.

```
select
    name, city, salary,
    round(
        salary*100.0 / last_value(salary) over w
    ) as percent
from employees
window w as (
    partition by city
    order by salary
    rows between unbounded preceding and unbounded
following
)
order by city, salary, id;
```


Функции смещения

Оконные функции смещения:

Функция Описание

<code>lag(value, offset)</code>	значение <code>value</code> из строки, отстоящей на <code>offset</code> строк назад от текущей
<code>lead(value, offset)</code>	значение <code>value</code> из строки, отстоящей на <code>offset</code> строк вперед от текущей
<code>first_value(value)</code>	значение <code>value</code> из первой строки фрейма
<code>last_value(value)</code>	значение <code>value</code> из последней строки фрейма
<code>nth_value(value, n)</code>	значение <code>value</code> из <code>n</code> -й строки фрейма

`lag()` и `lead()` действуют относительно текущей строки, заглядывая вперед или назад на указанное количество строк.

it		
Елена	84	
Ксения	90	
Леонид	104	<code>lag(1)</code>
Марина	104	← текущая запись
Иван	120	<code>lead(1)</code>

`first_value()`, `last_value()` и `nth_value()` действуют относительно границ фрейма, выбирая указанную строку в пределах фрейма.

it			it		
Елена	84	<code>first_value()</code>	Елена	84	<code>nth_value(1)</code>
Ксения	90		Ксения	90	<code>nth_value(2)</code>
Леонид	104	<code>last_value()</code>	Леонид	104	<code>nth_value(3)</code>
Марина	104		Марина	104	<code>nth_value(4)</code>
Иван	120		Иван	120	

Чтобы границы фрейма совпадали с границами секции (или всего окна, если секция одна) — используют такую конструкцию в определении окна:

rows between unbounded preceding and unbounded following

Так держать

Вы узнали, как сравнивать строки с соседями и границами окна. В следующей главе будем агрегировать данные!

Агрегация

Агрегация — это когда мы считаем суммарные или средние показатели (*агрегаты*). Например, среднюю зарплату по каждому региону или количество золотых медалей у каждой страны в зачете Олимпийских игр.

Мы будем агрегировать данные по сотрудникам из таблицы `employees`:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

Агрегат по секции

У каждого департамента есть фонд оплаты труда — денежная сумма, которая ежемесячно уходит на выплату зарплат сотрудникам. Посмотрим, какой процент от этого фонда составляет зарплата каждого сотрудника:

было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
23	Леонид	it	104
24	Марина	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

стало

name	depart	salary ↑	fund	perc
Дарья	hr	70	148	47%
Борис	hr	78	148	53%
Елена	it	84	502	17%
Ксения	it	90	502	18%
Леонид	it	104	502	21%
Марина	it	104	502	21%
Иван	it	120	502	24%
Вероника	sales	96	292	33%
Григорий	sales	96	292	33%
Анна	sales	100	292	34%

Столбец `fund` показывает фонд оплаты труда отдела, а `perc` — долю зарплаты сотрудника от этого фонда. Видно, что в HR и продажах все более-менее ровно, а у айтишников есть заметный разброс зарплат.

Как перейти от «было» к «стало»?

Отсортируем таблицу по департаментам:

```
select
    name, department, salary,
    null as fund, null as perc
from employees
order by department, salary, id;
```

Теперь пройдем от первой строчки до последней. На каждом шаге будем считать:

- `fund` — сумму зарплат по департаменту в целом;
- `perc` — долю зарплаты сотрудника от этой суммы.

①

name	depart	salary ↑	fund	perc
Дарья	hr	70	148	47 %
Борис	hr	78		
Елена	it	84		
Ксения	it	90		
Леонид	it	104		
Марина	it	104		
Иван	it	120		
Вероника	sales	96		
Григорий	sales	96		
Анна	sales	100		

②

name	depart	salary ↑	fund	perc
Дарья	hr	70	148	47 %
Борис	hr	78	148	53 %
Елена	it	84		
Ксения	it	90		
Леонид	it	104		
Марина	it	104		
Иван	it	120		
Вероника	sales	96		
Григорий	sales	96		
Анна	sales	100		

③

name	depart	salary ↑	fund	perc
Дарья	hr	70	148	47 %
Борис	hr	78	148	53 %
Елена	it	84	502	17 %
Ксения	it	90		
Леонид	it	104		
Марина	it	104		
Иван	it	120		
Вероника	sales	96		
Григорий	sales	96		
Анна	sales	100		

④

name	depart	salary ↑	fund	perc
Дарья	hr	70	148	47 %
Борис	hr	78	148	53 %
Елена	it	84	502	17 %
Ксения	it	90	502	18 %
Леонид	it	104		
Марина	it	104		
Иван	it	120		
Вероника	sales	96		
Григорий	sales	96		
Анна	sales	100		

⑤

name	depart	salary ↑	fund	perc
Дарья	hr	70	148	47 %
Борис	hr	78	148	53 %
Елена	it	84	502	17 %
Ксения	it	90	502	18 %
Леонид	it	104	502	21 %
Марина	it	104		
Иван	it	120		
Вероника	sales	96		
Григорий	sales	96		
Анна	sales	100		

и так далее...

Одной gifкой:

name	depart	salary	fund	perc
Агрегация: сумма по секции				

Смотреть: <https://antonz.ru/to#w24sum>

Окно состоит из секций по департаментам. При этом порядок записей в секции неважен: мы считаем сумму значений salary, а она не зависит от порядка.

```
window w as (
    partition by department
)
```

Для расчета fund подойдет обычная функция `sum()` — ее можно использовать поверх окна. А `perc` посчитаем как `salary / fund`:

```
select
    name, department, salary,
    sum(salary) over w as fund,
    round(salary * 100.0 / sum(salary) over w as
perc
from employees
window w as (partition by department
order by department, salary, id;
```

name	department	salary	fund	perc
Дарья	hr	70	148	47
Борис	hr	78	148	53
Елена	it	84	502	17
Ксения	it	90	502	18
Леонид	it	104	502	21
Марина	it	104	502	21
Иван	it	120	502	24
Вероника	sales	96	292	33
Григорий	sales	96	292	33
Анна	sales	100	292	34

Функция `sum()` работает без неожиданностей — считает сумму значений по всей секции, которой принадлежит текущая строка.

Фонд оплаты труда по городу

Есть таблица сотрудников `employees`. Мы хотим для каждого сотрудника увидеть, сколько процентов составляет его зарплата от общего фонда труда по городу:

name	city	salary	fund	perc
Ксения	Москва	90	510	18
Вероника	Москва	96	510	19
Анна	Москва	100	510	20
Марина	Москва	104	510	20
Иван	Москва	120	510	24
Дарья	Самара	70	432	16
Борис	Самара	78	432	18
Елена	Самара	84	432	19
Григорий	Самара	96	432	22
Леонид	Самара	104	432	24

Напишите запрос, который это сделает. Для округления в столбце perc используйте функцию `round()`.

Сортировка результата: `city, salary`

Песочница: <https://antonz.ru/to#empdb>

☑ Фонд оплаты труда по городу

Окно: секция по городу.

Фонд оплаты труда по городу = сумма по зарплатам внутри секции.

Процент от ФОТ = зарплата сотрудника деленная на ФОТ по городу.

```
select
    name, city, salary,
    sum(salary) over w as fund,
    round(salary * 100.0 / sum(salary) over w as perc
from employees
window w as (partition by city)
order by city, salary, id;
```

name	city	salary	fund	perc
Ксения	Москва	90	510	18
Вероника	Москва	96	510	19
Анна	Москва	100	510	20
Марина	Москва	104	510	20
Иван	Москва	120	510	24
Дарья	Самара	70	432	16
Борис	Самара	78	432	18
Елена	Самара	84	432	19
Григорий	Самара	96	432	22
Леонид	Самара	104	432	24

✂ Средняя зарплата по департаменту

Есть таблица сотрудников `employees`. Мы хотим для каждого сотрудника увидеть:

- сколько человек трудится в его отделе (`emp_cnt`);
- какая средняя зарплата по отделу (`sal_avg`);
- на сколько процентов отклоняется его зарплата от средней по отделу (`diff`).

name	department	salary	emp_cnt	sal_avg	diff
Дарья	hr	70	2	74	-5
Борис	hr	78	2	74	5
Елена	it	84	5	100	-16
Ксения	it	90	5	100	-10
Леонид	it	104	5	100	4
Марина	it	104	5	100	4
Иван	it	120	5	100	20
Вероника	sales	96	3	97	-1
Григорий	sales	96	3	97	-1
Анна	sales	100	3	97	3

Напишите запрос, который это сделает. При этом:

- для подсчета количества используйте функцию `count()`;
- для расчета среднего используйте функцию `avg()`;
- для округления используйте функцию `round()`.

Сортировка результата: `department, salary, id`

Подсказка. Формула расчета `diff`:

`diff = round((salary - sal_avg) * 100.0 / sal_avg)`

Песочница: <https://antonz.ru/to#empdb>

☑ Средняя зарплата по департаменту

Окно: секция по департаменту.

Количество сотрудников в отделе = количество строк в секции.

Средняя зарплата по отделу = средняя зарплата по секции.
 Отклонение = (зарплата*сотрудника — средняя*зарплата) /
 средняя_зарплата

```

select
  name, department, salary,
  count(*) over w as emp_cnt,
  round(avg(salary) over w, 0) as sal_avg,
  round(
    (salary - avg(salary) over w) * 100.0 /
    avg(salary) over w
  ) as diff
from employees
window w as (partition by department)
order by department, salary, id;

```

name	department	salary	emp_cnt	sal_avg	diff
Дарья	hr	70	2	74	-5
Борис	hr	78	2	74	5
Елена	it	84	5	100	-16
Ксения	it	90	5	100	-10
Леонид	it	104	5	100	4
Марина	it	104	5	100	4
Иван	it	120	5	100	20
Вероника	sales	96	3	97	-1
Григорий	sales	96	3	97	-1
Анна	sales	100	3	97	3

Фильтрация и порядок выполнения

Вернемся к запросу, который считал фонд оплаты труда по департаменту:

```

select
  name, department, salary,
  sum(salary) over w as fund

```

```

from employees
window w as (partition by department)
order by department, salary, id;

```

name	department	salary	fund
Дарья	hr	70	148
Борис	hr	78	148
Елена	it	84	502
Ксения	it	90	502
Леонид	it	104	502
Марина	it	104	502
Иван	it	120	502
Вероника	sales	96	292
Григорий	sales	96	292
Анна	sales	100	292

Допустим, мы хотим оставить в отчете только самарских сотрудников. Добавим фильтр:

```

select
    name, salary,
    sum(salary) over w as fund
from employees
where city = 'Самара'
window w as (partition by department)
order by department, salary, id;

```

name	salary	fund
Дарья	70	148
Борис	78	148
Елена	84	188
Леонид	104	188
Григорий	96	96

Фильтр сработал. Вот только значения `fund` отличаются от ожидаемых:

ожидание

name	salary	fund
Дарья	70	148
Борис	78	148
Елена	84	502
Леонид	104	502
Григорий	96	292

реальность

name	salary	fund
Дарья	70	148
Борис	78	148
Елена	84	188
Леонид	104	188
Григорий	96	96

Все дело в порядке выполнения операций. Вот в какой последовательности действует движок, когда выполняет запрос:

1. Взять нужные таблицы (`from`) и соединить их при необходимости (`join`).
2. Отфильтровать строки (`where`).
3. Сгруппировать строки (`group by`).
4. Отфильтровать результат группировки (`having`).
5. Взять конкретные столбцы из результата (`select`).
- 6. Рассчитать значения оконных функций** (`function() over window`).
7. Отсортировать то, что получилось (`order by`).

Таким образом, окна обрабатывают предпоследним шагом, уже после фильтрации и группировки результатов. Поэтому в нашем запросе `fund` отражает не сумму всех зарплат по департаменту, а сумму только по самарским сотрудникам.

Решение — использовать подзапрос с окном и фильтровать его в основном запросе:

```
with emp as (
  select
    name, city, salary,
    sum(salary) over w as fund
  from employees
  window w as (partition by department)
```

```

    order by department, salary, id
)
select name, salary, fund
from emp where city = 'Самапа';

```

name	salary	fund
Дарья	70	148
Борис	78	148
Елена	84	502
Леонид	104	502
Григорий	96	292

Описание окна

До сих пор мы описывали окно в блоке window и ссылались на него в выражении over:

```

select
    name, department, salary,
    count(*) over w as emp_count,
    sum(salary) over w as fund
from employees
window w as (partition by department)
order by department, salary, id;

```

name	department	salary	emp_count	fund
Дарья	hr	70	2	148
Борис	hr	78	2	148
Елена	it	84	5	502
Ксения	it	90	5	502
Леонид	it	104	5	502
Марина	it	104	5	502
Иван	it	120	5	502
Вероника	sales	96	3	292
Григорий	sales	96	3	292
Анна	sales	100	3	292

Это не единственный способ. SQL разрешает вообще не использовать `window` и описывать окно прямо внутри `over`:

```
select
    name, department, salary,
    count(*) over (partition by department) as emp_count,
    sum(salary) over (partition by department) as fund
from employees
order by department, salary, id;
```

Мне больше нравится вариант с `window` — его легче читать, и можно явно переиспользовать окно. Но в документации часто встречается определение окна внутри `over`, поэтому не удивляйтесь, когда увидите его.

Кстати, определение окна может быть пустым:

```
select
    name, department, salary,
    count(*) over () as emp_count,
    sum(salary) over () as fund
from employees
order by department, salary, id;
```

Такое окно включает все строки, так что `emp_count` покажет общее количество сотрудников, а `fund` — общий фонд оплаты труда по всем записям `employees`.

Порядок выполнения 1

Есть такой запрос:

```
select
    city,
    department,
```

```
sum(salary) as dep_salary,  
sum(sum(salary)) over (partition by city) as x,  
sum(sum(salary)) over () as y  
from employees  
group by city, department  
order by city, department;
```

Что вернет столбец x?

Выберите один вариант:

- Суммарную зарплату по городу.
- Суммарную зарплату по департаменту в городе.
- Суммарную зарплату по департаменту.
- Запрос вернет ошибку.

Песочница: <https://antonz.ru/to#empdb>

☒ Порядок выполнения 1

Ответ — «суммарную зарплату по городу».

Оконные функции выполняются уже после группировки. Поэтому можно представить, что запрос выполняется в два этапа.

Первый этап:

city	department	dep_salary
Москва	it	314
Москва	sales	196
Самара	hr	148
Самара	it	188
Самара	sales	96

И второй этап, который выполняет такое выражение:

```
sum(dep_salary) over (partition by city) as x
```

Это сумма зарплат по департаментам в пределах города — то есть суммарная зарплата по городу.

✍ Порядок выполнения 2

Есть такой запрос:

```
select
  city,
  department,
  sum(salary) as dep_salary,
  sum(sum(salary)) over (partition by city) as x,
  sum(sum(salary)) over () as y
from employees
group by city, department
order by city, department;
```

Что вернет столбец `y`?

Выберите один вариант:

- Суммарную зарплату по городу.
- Суммарную зарплату по департаменту.
- Суммарную зарплату по всем сотрудникам.
- Запрос вернет ошибку.

Песочница: <https://antonz.ru/to#empdb>

☑ Порядок выполнения 2

Ответ — «суммарную зарплату по всем сотрудникам».

Оконные функции выполняются уже после группировки. Поэтому можно представить, что запрос выполняется в два этапа.

Первый этап:

```
select
  city,
  department,
  sum(salary) as dep_salary
from employees
group by city, department;
```

city	department	dep_salary
Москва	it	314
Москва	sales	196
Самапа	hr	148
Самапа	it	188
Самапа	sales	96

И второй этап, который выполняет такое выражение:

```
sum(dep_salary) over () as y
```

Это сумма зарплат по всем департаментам — то есть суммарная зарплата по всем сотрудникам.

Функции агрегации

Оконные функции агрегации:

Функция	Описание
<code>min(value)</code>	минимальное value среди строк секции окна
<code>max(value)</code>	максимальное value

<code>count(value)</code>	количество <code>value</code> , не равных <code>null</code>
<code>avg(value)</code>	среднее значение по всем <code>value</code>
<code>sum(value)</code>	сумма значений <code>value</code>
<code>group_concat(value, separator)</code>	соединение значений <code>value</code> через разделитель <code>separator</code> (SQLite и MySQL)
<code>string_agg(value, separator)</code>	аналог <code>group_concat()</code> в PostgreSQL и MS SQL

Так держать

Вы узнали, как считать фиксированные агрегаты в окнах. В следующей главе разберемся со скользящими агрегатами!

Скользящие агрегаты

Скользящие агрегаты — это те же сумма, среднее и прочие сводные показатели. Только рассчитывают их не по всем элементам набора, а более хитрым способом.

Разберемся на примерах. Здесь возьмем таблицу `expenses`, которая содержит месячные доходы и расходы компании, где трудятся наши сотрудники.

Для краткости будем рассматривать только первые девять месяцев 2020 года:

```
select
  year, month, income, expense
from expenses
where year = 2020 and month ≤ 9
order by month;
```

year	month	income	expense
2020	1	94	82
2020	2	94	75
2020	3	94	104
2020	4	100	94
2020	5	100	99
2020	6	100	105
2020	7	100	95
2020	8	100	110
2020	9	104	104

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

Скользящее среднее

Судя по данным, доходы компании растут: 94К Р в январе → 104К Р в сентябре. А вот растут ли расходы? Сходу сложно сказать, месяц на месяц не приходится.

Чтобы сгладить эти скачки, используют «скользящее среднее» — для каждого месяца рассчитывают средний расход с учетом предыдущего и следующего месяца:

- скользящее среднее за февраль = (январь + февраль + март) / 3;
- за март = (февраль + март + апрель) / 3;
- за апрель = (март + апрель + май) / 3;
- и так далее.

Рассчитаем скользящее среднее по всем месяцам:

было				стало			
year	month	income	expense	year ↑	month ↑	expense	roll_avg
2020	1	94	82	2020	1	82	79
2020	2	94	75	2020	2	75	87
2020	3	94	104	2020	3	104	91
2020	4	100	94	2020	4	94	99
2020	5	100	99	2020	5	99	99
2020	6	100	105	2020	6	105	100
2020	7	100	95	2020	7	95	103
2020	8	100	110	2020	8	110	103
2020	9	104	104	2020	9	104	107

Столбец `roll_avg` показывает скользящее среднее по расходам за три месяца (текущий, предыдущий и следующий). Теперь хорошо видно, что расходы стабильно растут.

Как перейти от «было» к «стало»?

Отсортируем таблицу по месяцам:

```
select
    year, month, expense,
    null as roll_avg
from expenses
where year = 2020 and month ≤ 9
order by year, month;
```

year	month	expense	roll_avg
2020	1	82	
2020	2	75	
2020	3	104	
2020	4	94	
2020	5	99	
2020	6	105	
2020	7	95	
2020	8	110	
2020	9	104	

Теперь пройдем от первой строчки до последней. На каждом шаге будем считать среднее по предыдущему, текущему и следующему значению из столбца `expense`:

①

year ↑	month ↑	expense	roll_avg
2020	1	82	79
2020	2	75	
2020	3	104	
2020	4	94	
2020	5	99	
2020	6	105	
2020	7	95	
2020	8	110	
2020	9	104	

②

year ↑	month ↑	expense	roll_avg
2020	1	82	79
2020	2	75	87
2020	3	104	
2020	4	94	
2020	5	99	
2020	6	105	
2020	7	95	
2020	8	110	
2020	9	104	

③

year ↑	month ↑	expense	roll_avg
2020	1	82	79
2020	2	75	87
2020	3	104	91
2020	4	94	
2020	5	99	
2020	6	105	
2020	7	95	
2020	8	110	
2020	9	104	

④

year ↑	month ↑	expense	roll_avg
2020	1	82	79
2020	2	75	87
2020	3	104	91
2020	4	94	99
2020	5	99	
2020	6	105	
2020	7	95	
2020	8	110	
2020	9	104	

⑤

year ↑	month ↑	expense	roll_avg
2020	1	82	79
2020	2	75	87
2020	3	104	91
2020	4	94	99
2020	5	99	99
2020	6	105	
2020	7	95	
2020	8	110	
2020	9	104	

и так далее...

Одной гифкой:

year	month	expense	roll_avg
Агрегация: скользящее среднее			

Смотреть: <https://antonz.ru/to#w25avg>

По-хорошему, стоило бы учесть декабрь 2019 при расчете среднего за январь 2020, а октябрь 2020 — при расчете среднего за сентябрь 2020. Но для простоты не будем этого делать.

Рамка на каждом шаге сдвигается вниз, скользит — так и получается скользящее среднее. Чтобы описать на SQL, придется вспомнить концепцию фреймов, с которой мы познакомились в главе «Смещение»:

1. Окно состоит из одной или нескольких секций (в нашем случае секция одна — все записи `expenses`).
2. Внутри секции записи упорядочены по конкретным столбцам (`order by year, month`).
3. У каждой записи свой фрейм.

Фрейм на каждом шаге охватывает три записи — текущую, предыдущую и следующую:

expenses

year/mon	exp
2020 1	82
2020 2	75
2020 3	104
2020 4	94
2020 5	99
2020 6	105
2020 7	95
2020 8	110
2020 9	104

← текущая запись

expenses

year/mon	exp
2020 1	82
2020 2	75
2020 3	104
2020 4	94
2020 5	99
2020 6	105
2020 7	95
2020 8	110
2020 9	104

← текущая запись

Вот как записать это на SQL:

```

window w as (
    order by year, month
    rows between 1 preceding and 1 following
)

```

Строчка `rows` — это определение фрейма. Она говорит:

Выбрать строки от 1 предыдущей до 1 следующей»

На следующем шаге разберемся с фреймами подробно, а пока закончим с нашим запросом.

Считаем среднее по расходам — это функция `avg()`:

```
avg(expense) over w
```

Добавим округление и сведем все вместе:

```

select
    year, month, expense,
    round(avg(expense) over w) as roll_avg
from expenses
where year = 2020 and month ≤ 9
window w as (
    order by year, month
    rows between 1 preceding and 1 following
)
order by year, month;

```

year	month	expense	roll_avg
2020	1	82	79
2020	2	75	87
2020	3	104	91
2020	4	94	99
2020	5	99	99
2020	6	105	100
2020	7	95	103
2020	8	110	103
2020	9	104	107

Скользящее среднее по расходам готово!

Фрейм

В общем случае определение фрейма выглядит так:

rows between X preceding and Y following

Где X — количество строк перед текущей, а Y — количество строк после текущей:

year/mon	exp
2020 1	82
2020 2	75
2020 3	104
2020 4	94
2020 5	99
2020 6	105
2020 7	95
2020 8	110
2020 9	104

rows between

2 preceding

and

1 following

year/mon	exp
2020 1	82
2020 2	75
2020 3	104
2020 4	94
2020 5	99
2020 6	105
2020 7	95
2020 8	110
2020 9	104

rows between

1 preceding

and

2 following

Если указать вместо X или Y значение `unbounded` — это значит «от/до границы секции»:

year/mon	exp	rows between
2020 1	82	unbounded preceding
2020 2	75	
2020 3	104	
2020 4	94	
2020 5	99	and
2020 6	105	1 following
2020 7	95	
2020 8	110	
2020 9	104	

year/mon	exp	rows between
2020 1	82	
2020 2	75	
2020 3	104	1 preceding
2020 4	94	and
2020 5	99	
2020 6	105	
2020 7	95	
2020 8	110	
2020 9	104	unbounded following

Если указать вместо X `preceding` или Y `following` значение `current row` — это значит «текущая запись»:

year/mon	exp	rows between
2020 1	82	
2020 2	75	2 preceding
2020 3	104	
2020 4	94	and current row
2020 5	99	
2020 6	105	
2020 7	95	
2020 8	110	
2020 9	104	

year/mon	exp	rows between
2020 1	82	
2020 2	75	
2020 3	104	
2020 4	94	current row and
2020 5	99	
2020 6	105	2 following
2020 7	95	
2020 8	110	
2020 9	104	

Фрейм никогда не выходит за границы секции, если столкнулся с ней — обрезается:

year/mon	exp	rows between
2020 1	82	2 preceding
2020 2	75	and
2020 3	104	
2020 4	94	2 following
2020 5	99	
2020 6	105	
2020 7	95	
2020 8	110	
2020 9	104	

year/mon	exp	rows between
2020 1	82	
2020 2	75	
2020 3	104	
2020 4	94	
2020 5	99	
2020 6	105	2 preceding
2020 7	95	
2020 8	110	and
2020 9	104	2 following

У фреймов намного больше возможностей, но мы пока ограничимся этими. Обсудим остальные в следующей части книги — она полностью посвящена фреймам.

✍ Определение фрейма 1

Вот границы фрейма для пятой записи:

year	month	income	expense	
2020	1	94	82	
2020	2	94	75	← начало фрейма
2020	3	94	104	
2020	4	100	94	
2020	5	100	99	← текущая запись
2020	6	100	105	
2020	7	100	95	← конец фрейма
2020	8	100	110	
2020	9	104	104	

Как выглядит определение фрейма?

Выберите один вариант:

- `rows between 2 preceding and 2 following`
- `rows between 2 preceding and 3 following`
- `rows between 3 preceding and 2 following`
- `rows between 3 preceding and 3 following`

☑ Определение фрейма 1

year	month	income	expense	
2020	1	94	82	
2020	2	94	75	← начало фрейма
2020	3	94	104	
2020	4	100	94	
2020	5	100	99	← текущая запись
2020	6	100	105	
2020	7	100	95	← конец фрейма
2020	8	100	110	
2020	9	104	104	

Определение фрейма выглядит так:

`rows between 3 preceding and 2 following`

☑ Определение фрейма 2

Вот границы фрейма для седьмой записи:

year	month	income	expense	
2020	1	94	82	
2020	2	94	75	
2020	3	94	104	
2020	4	100	94	
2020	5	100	99	← начало фрейма
2020	6	100	105	
2020	7	100	95	← текущая запись
2020	8	100	110	
2020	9	104	104	← конец фрейма

Как может выглядеть определение фрейма?

Выберите все подходящие варианты:

- rows between 2 preceding and 2 following
- rows between 2 preceding and 3 following
- rows between 3 preceding and 3 following
- rows between 2 preceding and unbounded following

☑ Определение фрейма 2

year	month	income	expense	
2020	1	94	82	
2020	2	94	75	
2020	3	94	104	
2020	4	100	94	
2020	5	100	99	← начало фрейма
2020	6	100	105	
2020	7	100	95	← текущая запись
2020	8	100	110	
2020	9	104	104	← конец фрейма

Фрейм никогда не выходит за границы секции, если столкнулся с ней — обрезается. Поэтому подходят все варианты, кроме 3 preceding and 3 following.

```
rows between 2 preceding and 2 following
rows between 2 preceding and 3 following
rows between 2 preceding and unbounded following
```

Скользящее среднее по доходам

Есть таблица доходов–расходов expenses. Мы хотим рассчитать скользящее среднее по доходам за предыдущий и текущий месяц:

year	month	income	roll_avg
2020	1	94	94
2020	2	94	94
2020	3	94	94
2020	4	100	97
2020	5	100	100
2020	6	100	100
2020	7	100	100
2020	8	100	100
2020	9	104	102

Напишите запрос, который это делает.

Фильтрация: year = 2020 и month ≤ 9

Сортировка результата: year, month

Песочница: <https://antonz.ru/to#empdb>

☑ Скользящее среднее по доходам

Окно:

- сортировка по году и месяцу;
- фрейм от предыдущей строки до текущей;

Функция `avg()` считает средний доход.

```
select
    year, month, income,
    round(avg(income) over w) as roll_avg
from expenses
where year = 2020 and month ≤ 9
window w as (
    order by year, month
    rows between 1 preceding and current row
)
order by year, month;
```

year	month	income	roll_avg
2020	1	94	94
2020	2	94	94
2020	3	94	94
2020	4	100	97
2020	5	100	100
2020	6	100	100
2020	7	100	100
2020	8	100	100
2020	9	104	102

Сумма нарастающим итогом

Благодаря скользящему среднему, мы выяснили, что доходы и расходы компании растут. А как они соотносятся друг с другом? Хочется понять, находится ли компания «в плюсе» или «в минусе» с учетом всех заработанных и потраченных денег.

Причем важно понимать не на конец периода, а на каждый месяц. Потому что если в сентябре у компании все ОК, а в июне она ушла в минус — это потенциальная проблема (такую ситуацию называют «кассовым разрывом»).

Поэтому посчитаем доходы и расходы по месяцам нарастающим итогом (кумулятивно):

- кумулятивный доход за январь = январь;
- за февраль = январь + февраль;
- за март = январь + февраль + март;
- за апрель = январь + февраль + март + апрель;
- и так далее.

month ↑	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75	188	157	31
3	94	104	282	261	21
4	100	94	382	355	27
5	100	99	482	454	28
6	100	105	582	559	23
7	100	95	682	654	28
8	100	110	782	764	18
9	104	104	886	868	18

Столбцы `t_` показывают значения нарастающим итогом:

- доходы (`t_income`),
- расходы (`t_expense`),
- прибыль (`t_profit`).

`t_profit = t_income - t_expense`

Как рассчитать кумулятивные показатели?

Отсортируем таблицу по месяцам:

```
select
    year, month, income, expense,
    null as t_income,
    null as t_expense,
    null as t_profit
from expenses
where year = 2020 and month ≤ 9
order by year, month;
```

year	month	income	expense	t_income	t_expen	t_prof
2020	1	94	82			
2020	2	94	75			
2020	3	94	104			
2020	4	100	94			
2020	5	100	99			
2020	6	100	105			
2020	7	100	95			
2020	8	100	110			
2020	9	104	104			

Теперь пройдем от первой строчки до последней. На каждом шаге будем считать суммарные показатели от начала таблицы до текущей строки:

①

month ↑	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75			
3	94	104			
4	100	94			
5	100	99			
6	100	105			
7	100	95			
8	100	110			
9	104	104			

②

month ↑	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75	188	157	31
3	94	104			
4	100	94			
5	100	99			
6	100	105			
7	100	95			
8	100	110			
9	104	104			

③

month ↑	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75	188	157	31
3	94	104	282	261	21
4	100	94			
5	100	99			
6	100	105			
7	100	95			
8	100	110			
9	104	104			

④

month ↑	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75	188	157	31
3	94	104	282	261	21
4	100	94	382	355	27
5	100	99			
6	100	105			
7	100	95			
8	100	110			
9	104	104			

⑤

month ↑	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75	188	157	31
3	94	104	282	261	21
4	100	94	382	355	27
5	100	99	482	454	28
6	100	105			
7	100	95			
8	100	110			
9	104	104			

и так далее...

Одной gifкой:

month	income	expense	t_income	t_expense	t_profit
Агрегация: сумма нарастающим итогом					

Смотреть: <https://antonz.ru/to#w25sum>

Рамка на каждом шаге охватывает строки от начала таблицы до текущей записи. Мы уже знаем, как сформулировать подходящий фрейм:

```
window w as (  
    order by year, month  
    rows between unbounded preceding and current  
row  
)
```

Считаем сумму по доходам и расходам — это функция `sum()`:

```
sum(income) over w as t_income,  
sum(expense) over w as t_expense,
```

Прибыль считаем как разницу между доходами и расходами:

```
sum(income) over w - sum(expense) over w as t_profit
```

Все вместе:

```
select  
    year, month, income, expense,  
    sum(income) over w as t_income,  
    sum(expense) over w as t_expense,  
    sum(income) over w - sum(expense) over w as  
t_profit  
from expenses  
where year = 2020 and month ≤ 9  
window w as (  
    order by year, month  
    rows between unbounded preceding and current  
row  
)  
order by year, month;
```


year	month	income	expense	t_income	t_expen	t_prof
2020	1	94	82	94	82	12
2020	2	94	75	188	157	31
2020	3	94	104	282	261	21
2020	4	100	94	382	355	27
2020	5	100	99	482	454	28
2020	6	100	105	582	559	23
2020	7	100	95	682	654	28
2020	8	100	110	782	764	18
2020	9	104	104	886	868	18

По `t_profit` видно, что дела у компании идут неплохо. В некоторых месяцах расходы превышают доходы, но благодаря накопленной «денежной подушке» кассового разрыва не происходит.

✂ Фонд оплаты труда нарастающим итогом

Вернемся к таблице `employees`:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Мы хотим посчитать фонд оплаты труда нарастающим итогом независимо для каждого департамента:

id	name	department	salary	total
11	Дарья	hr	70	70
12	Борис	hr	78	148
21	Елена	it	84	84
22	Ксения	it	90	174
23	Леонид	it	104	278
24	Марина	it	104	382
25	Иван	it	120	502
...	...	...	...	...

Напишите запрос, который это сделает.

Сортировка результата: `department, salary, id`

Песочница: <https://antonz.ru/to#empdb>

☒ Фонд оплаты труда нарастающим итогом

Окно:

- секция по департаменту;
- сортировка по зарплате;
- фрейм от начала секции до текущей записи.

Функция `sum()` считает суммарную зарплату.

```
select
    id, name, department, salary,
    sum(salary) over w as total
from employees
window w as (
    partition by department
    order by salary, id
    rows between unbounded preceding and current row
)
order by department, salary, id;
```

id	name	department	salary	total
11	Дарья	hr	70	70
12	Борис	hr	78	148
21	Елена	it	84	84
22	Ксения	it	90	174
23	Леонид	it	104	278
24	Марина	it	104	382
25	Иван	it	120	502
31	Вероника	sales	96	96
32	Григорий	sales	96	192
33	Анна	sales	100	292

Фрейм по умолчанию

Возьмем запрос, который считает расходы нарастающим итогом:

```
select
    year, month, expense,
    sum(expense) over w as total
from expenses
where year = 2020 and month ≤ 9
window w as (
    order by year, month
    rows between unbounded preceding and current
row
)
order by year, month;
```

year	month	expense	total
2020	1	82	82
2020	2	75	157
2020	3	104	261
2020	4	94	355
2020	5	99	454
2020	6	105	559
2020	7	95	654
2020	8	110	764
2020	9	104	868

И уберем из него определение фрейма:

```
select
  year, month, expense,
  sum(expense) over w as total
from expenses
where year = 2020 and month ≤ 9
window w as (
  order by year, month
)
order by year, month;
```

Ожидаем, что в `total` теперь всегда будет одно и то же число — суммарные расходы за 9 месяцев. А вместо этого видим вот что:

ожидание

month	expense	total
1	82	868
2	75	868
3	104	868
4	94	868
5	99	868
6	105	868
7	95	868
8	110	868
9	104	868

реальность

month	expense	total
1	82	82
2	75	157
3	104	261
4	94	355
5	99	454
6	105	559
7	95	654
8	110	764
9	104	868

Запрос без фрейма все равно посчитал кумулятивные расходы — в точности как запрос с фреймом. Как так?

Все дело в наличии сортировки в окне (`order by year, month`). Правило такое:

- если в окне есть `order by`,
- и используется функция агрегации,
- и не указано определение фрейма,
- то используется фрейм по умолчанию.

Фрейм по умолчанию в нашем запросе распространяется от первой до текущей записи. Поэтому результаты совпадают с запросом, где фрейм был указан явно:

rows between unbounded preceding and current row

Но так бывает не всегда. Поэтому я рекомендую указывать фрейм явно — по крайней мере пока не разберетесь досконально в разновидностях фреймов. Добавили в окно `order by` — добавьте сразу и фрейм.

Если убрать из окна `order by`, агрегат из скользящего превратится в обычный:

year	month	expense	total
2020	1	82	868
2020	2	75	868
2020	3	104	868
2020	4	94	868
2020	5	99	868
2020	6	105	868
2020	7	95	868
2020	8	110	868
2020	9	104	868

```
select
  year, month, expense,
  sum(expense) over () as total
from expenses
where year = 2020 and month ≤ 9
order by year, month;
```

Здесь без неожиданностей.

Функции для скользящих агрегатов

Скользящие агрегаты используют те же самые функции, что и агрегаты обычные:

- `min()` и `max()`,
- `count()`, `avg()` и `sum()`,
- `group_concat()`.

Разница только в наличии фрейма у скользящих агрегатов.

Так держать

Вы узнали, как считать скользящие агрегаты в окнах. В следующей главе поработаем со статистикой!

Статистика

Когда речь идет о числовых показателях (зарплата сотрудника, температура воздуха, время ответа сервиса), нам часто интересно понять место отдельного значения в ряду других — чтобы понять, насколько оно «хорошее» или «плохое».

Например, любимое развлечение «белых воротничков» всех времен — меряться зарплатами.

Мы продолжим использовать таблицу `employees`:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

Кумулятивное распределение

Возьмем зарплату каждого сотрудника и определим, какой процент людей получает столько же или меньше:

было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
23	Леонид	it	104
24	Марина	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

стало

id	name	salary ↑	perc
11	Дарья	70	10 %
12	Борис	78	20 %
21	Елена	84	30 %
22	Ксения	90	40 %
31	Вероника	96	60 %
32	Григорий	96	60 %
33	Анна	100	70 %
23	Леонид	104	90 %
24	Марина	104	90 %
25	Иван	120	100 %

Рассчитать такое в SQL поможет функция `cume_dist()`:

```
select
    name, salary,
    cume_dist() over w as perc
from employees
window w as (order by salary)
order by salary, id;
```

name	salary	perc
Дарья	70	0.1
Борис	78	0.2
Елена	84	0.3
Ксения	90	0.4
Вероника	96	0.6
Григорий	96	0.6
...	...	...
Иван	120	1.0

Проценты здесь представлены десятичными дробями: 0.1 = 10%, 0.2 = 20%, и так далее. Это стандартное представление для SQL.

Вот что делает `cume_dist()`:

1. Располагает записи в порядке, указанном в `order by` окна (в нашем случае — по возрастанию зарплаты).
2. Находит текущую запись в общем ряду (зарплату текущего сотрудника среди всех зарплат).
3. Считает, сколько записей $\leq$ текущей по значению столбца из `order by` (сколько людей получают такую же или меньшую зарплату).
4. Делит на общее количество записей (на количество сотрудников).

$$\text{cume_dist} = \frac{\text{количество записей} \leq \text{текущей}}{\text{общее количество записей}}$$

В результате `cume_dist` возвращает процент записей со значением $\leq$ текущего (процент людей, которые получают такую же или меньшую зарплату).

Относительный ранг

Зададимся похожим вопросом: какой процент людей получает строго меньше, чем конкретный сотрудник?

Ответить поможет функция `percent_rank()`:

```
select
    name, salary,
    round(percent_rank() over w, 2) as perc
from employees
window w as (order by salary)
order by salary, id;
```

name	salary	perc
Дарья	70	0.0
Борис	78	0.11
Елена	84	0.22
Ксения	90	0.33
Вероника	96	0.44
Григорий	96	0.44
Анна	100	0.67
Леонид	104	0.78
Марина	104	0.78
Иван	120	1.0

Несмотря на радикально отличающееся название, `percent_rank()` делает почти то же самое, что и `cume_dist()`:

- `cume_dist()` считает процент записей, у которых значение столбца из `order by` $\leq$ текущему;

- `percent_rank()` считает процент записей, у которых значение столбца из `order by` < текущего (исключая текущую запись).

«Исключая текущую запись» — важное дополнение. Например, если бы меня попросили посчитать процент людей с меньшей зарплатой, чем у Ивана, я бы мог посчитать так:

$$\text{percent_rank} = 9 / 10 = 0.9 = 90\%$$

То есть получилось бы, что 90% сотрудников получают меньше Ивана. `percent_rank` же считает иначе. Он как бы говорит: все остальные сотрудники получают меньше Ивана. То есть 100% остальных сотрудников получают меньше Ивана. Поэтому «исключая текущую запись» и поэтому `percent_rank` для Ивана равен 1.0 (100%).

Вот `cume_dist()` и `percent_rank()` рядышком:

select

 name, salary,

 cume_dist() **over w as** cd,

 round(percent_rank() **over w**, 2) **as** pr

from employees

window w as (order by salary)

order by salary, id;

name	salary	cd	pr
Дарья	70	0.1	0.0
Борис	78	0.2	0.11
Елена	84	0.3	0.22
Ксения	90	0.4	0.33
Вероника	96	0.6	0.44
Григорий	96	0.6	0.44
Анна	100	0.7	0.67
Леонид	104	0.9	0.78
Марина	104	0.9	0.78
Иван	120	1.0	1.0

`cume_dist()` и `percent_rank()` во многом похожи на функцию ранжирования `rank()`:

- `rank()` считает абсолютный ранг строки (ее место относительно других строк согласно `order by` окна)
- `cume_dist()` и `percent_rank()` считают относительный ранг (в процентах относительно других строк).

Как и `rank()`, `cume_dist()` и `percent_rank()` имеют смысл только при заданном `order by` окна. Как и `rank()`, они поддерживают секции (`partition by`) и не поддерживают фреймы.

✂ Жаркий март

Есть таблица `weather` — это среднедневная температура в Лондоне за 6 месяцев 2020 года:

wdate	wtemp
2020-01-01	4.1
2020-01-02	8.6
2020-01-03	9.9
...	...
2020-06-28	17.2
2020-06-29	16.9
2020-06-30	16.2

Напишите запрос, который рассчитает `cume_dist` (`cd`) и `percent_rank` (`pr`) по температуре для всех дней марта, и вернет пять дней с самой высокой температурой:

wdate	wtemp	cd	pr
2020-03-18	12.5	1	1
2020-03-08	11.9	0.97	0.97
2020-03-11	11.7	0.94	0.93
2020-03-15	10.9	0.9	0.9
2020-03-14	10.1	0.87	0.87

Значения `cd` и `pr` округлите до двух знаков после запятой.
Сортировка результата: `wtemp desc`

Песочница: <https://antonz.ru/to#w26my>

Скачать: <https://antonz.ru/to#w26sql>

☑ Жаркий март

Запрос:

- выбираем только мартовские даты;
- окно по возрастанию температуры;
- сортировка результата по убыванию температуры;
- лимит результата на 5 записей, чтобы получить 5 самых теплых дней.

select

```
wdate, wtemp,  
round(cume_dist() over w, 2) as cd,  
round(percent_rank() over w, 2) as pr
```

from weather

where wdate **between** '2020-03-01' **and** '2020-03-31'

window w **as** (**order by** wtemp)

order by wtemp **desc**

limit 5;

✍ Начало весны

Продолжаем работать с таблицей температур `weather(wdate, wtemp)`. Хотим для каждого из дней с 1 по 5 марта определить, какой процент дней в марте имеют такую же или меньшую температуру:

wdate	wtemp	perc
2020-03-01	7.6	0.52
2020-03-02	7.0	0.32
2020-03-03	6.0	0.16
2020-03-04	4.2	0.06
2020-03-05	5.7	0.1

Напишите запрос, который это сделает. Значение `perc` округлите до двух знаков после запятой.

Песочница: <https://antonz.ru/to#w26my>

Скачать: <https://antonz.ru/to#w26sql>

☑ Начало весны

Запрос:

- выбираем только мартовские даты;
- окно по возрастанию температуры;
- сортировка результата по дате;
- лимит результата на 5 записей, чтобы получить 5 первых дней марта.

```
select
  wdate, wtemp,
  round(cume_dist() over w, 2) as perc
from weather
where wdate between '2020-03-01' and '2020-
03-31'
window w as (order by wtemp)
order by wdate
limit 5;
```

wdate	wtemp	perc
2020-03-01	7.6	0.52
2020-03-02	7.0	0.32
2020-03-03	6.0	0.16
2020-03-04	4.2	0.06
2020-03-05	5.7	0.1

Счастливые дни

Продолжаем работать с таблицей температур `weather(wdate, wtemp)`. Хотим для седьмого числа каждого месяца (7 января, 7 февраля, 7 марта, ...) определить, какой процент дней в соответствующем месяце имеют такую же или меньшую температуру.

Напишите запрос, который это сделает. Значение `perc` округлите до двух знаков после запятой.

Вам помогут эти функции (если используете песочницу MySQL по ссылке ниже):

wdate	wtemp	perc
2020-01-07	8.1	0.61
2020-02-07	5.0	0.1
2020-03-07	7.5	0.48
2020-04-07	12.3	0.53
2020-05-07	14.5	0.45
2020-06-07	13.4	0.1

- `month(wdate)` — возвращает порядковый номер месяца для указанной даты (1–12);
- `day(wdate)` — возвращает порядковый номер дня месяца для указанной даты (1–31).

Песочница: <https://antonz.ru/to#w26my>

Скачать: <https://antonz.ru/to#w26sql>

☑ Счастливые дни

Запрос:

- промежуточное табличное выражение для расчета `cume_dist`;
- окно с секциями по месяцам и сортировкой по температуре;
- отбор результата по порядковому номеру дня месяца.

```
with temps as (  
    select  
        wdate, wtemp,  
        round(cume_dist() over w, 2) as perc  
    from weather  
    window w as (  
        partition by month(wdate)  
        order by wtemp  
    )  
)  
select wdate, wtemp, perc  
from temps  
where day(wdate) = 7  
order by wdate;
```

wdate	wtemp	perc
2020-01-07	8.1	0.61
2020-02-07	5.0	0.1
2020-03-07	7.5	0.48
2020-04-07	12.3	0.53
2020-05-07	14.5	0.45
2020-06-07	13.4	0.1

Сводные значения

Чтобы упростить себе жизнь, аналитики любят описывать весь набор данных одним или несколькими числами. Такие числа называются *сводными значениями* или *статистиками*.

Пожалуй, самая популярная статистика — это *среднее арифметическое значение* (average). Все знают функцию `avg()`, которая его считает.

Допустим, сотрудники компании арендовали кафе для корпоратива. Если бы мы вздумали проанализировать доходы посетителей кафе в этот момент, увидели бы следующую картину:

Дарья	Борис	Елена	Ксения	Вероника	Григорий	Анна	Леонид	Марина
70	78	84	90	96	96	100	104	104

Здесь 9 человек, потому что Иван уволился накануне мероприятия.

Средний доход посетителя кафе составляет $\text{sum}(\text{salary}) / 9 \approx 91$. Достаточно адекватная оценка, которая действительно описывает «среднего» посетителя.

Но тут в кафе входит Иван. Оказывается, с сегодняшнего дня он работает вице-президентом крупной корпорации. Доход Ивана теперь составляет 1700К. И вот что случилось со средним доходом посетителя:

$$\text{sum}(\text{salary}) / 10 \approx 252$$

Средний доход теперь равен 252К. Но хорошо ли он описывает типичного посетителя? Не думаю: ведь все, кроме Ивана, получают намного меньше (а сам Иван — намного больше).

Среднее арифметическое — не всегда адекватный показатель: оно подвержено влиянию экстремально малых и больших значений (таких как Иван в нашем примере). Достаточно одного или нескольких «выбросов», чтобы испортить всю картину и дать ложное представление о среднем значении.

Медиана

Чтобы посчитать среднее значение, устойчивое к выбросам, вместо среднего арифметического используют *медиану* (*median*).

Чтобы получить медиану, нужно упорядочить элементы по возрастанию и взять тот, что посередине. В нашем примере с доходами посетителей бара:

70 78 84 90 96 96 100 104 104 1700

↑ ↑

Здесь два «срединных» значения (пятое и шестое). В таких случаях медиану считают как среднее арифметическое между ними, но у нас значения одинаковые, так что и считать нечего: медиана равна 96.

Доход 96К намного лучше описывает типичного посетителя, чем 252К, не правда ли?

Среднее врет
когда есть выбросы

2 2 4 4	среднее = 5	✓
4 6 8 10	хорошо описывает «типичного представителя»	
2 2 4 4	среднее = 129	✗
4 6 8 1002	плохо описывает «типичного представителя»	

↑
выброс!

Медиана не врет
устойчива к выбросам

2 2 4 4	медиана = 4	✓
4 6 8 10	хорошо описывает «типичного представителя»	
2 2 4 4	медиана = 4	✓
4 6 8 1002	хорошо описывает «типичного представителя»	

↑
выброс!

чтобы получить медиану,
расставьте элементы
по возрастанию и возьмите
тот, что посередине

Процентили

Мы выяснили, что медиана дохода составляет 96К. Это полезное знание: оно описывает всех людей в баре одним числом — *усредненным доходом*. Другой интересный вопрос, который часто возникает: как охарактеризовать *большинство посетителей*?

Можно ли сказать, что у большинства посетителей доход $\leq 96\text{К}$? Нет: по определению медианы, у половины доход меньше 96К, а у половины больше. Говоря о большинстве, используют *процентили* (percentile).

Процентиль характеризует конкретный процент выборки. Вот некоторые распространенные процентили:

- если 25-й процентиль = X , то у 25% элементов значение $\leq X$;
- если 50-й процентиль = X , то у 50% элементов значение $\leq X$;
- если 75-й процентиль = X , то у 75% элементов значение $\leq X$;
- если 90-й процентиль = X , то у 90% элементов значение $\leq X$.

В нашем примере с доходами посетителей бара:

- 25-й процентиль = 84 (у 25% людей доход $\leq 84K$);
- 50-й процентиль = 96 (у 50% людей доход $\leq 96K$);
- 75-й процентиль = 104 (у 75% людей доход $\leq 104K$);
- 90-й процентиль = 104 (у 90% людей доход $\leq 104K$).

50-й процентиль и медиана — это одно и то же.

Большинство обычно характеризуют 90-м, 95-м или 99-м процентилем.



Медиана
описывает типичное значение

90-й процентиль
описывает большинство значений

используйте их
вместо
среднего

Прежде чем двигаться дальше, закрепим понимание задачами.

Медиана количества страниц

Пусть есть 10 книг с таким количеством страниц:

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Чему равна медиана количества страниц?

Выберите один вариант:

- 5
- 6
- 5.5
- непонятно

☒ Медиана количества страниц

Для множества с четным количеством элементов медиана рассчитывается как среднее арифметическое двух «серединовых» элементов:

1	2	3	4	5	6	7	8	9	10
				↑	↑				
(5 + 6)				/ 2 = 5.5					

Среднее арифметическое и медиана

Может ли среднее арифметическое быть меньше медианы?

Выберите один вариант:

- Нет, среднее всегда больше либо равно медиане.
- Да, из-за влияния на среднее экстремально малых значений.
- Да, из-за влияния на медиану экстремально больших значений.
- Да, среднее всегда меньше медианы.

☒ Среднее арифметическое и медиана

Допустим, есть девять книг с таким количеством страниц:

3	839	916	943	761	999	881	903	1000
---	-----	-----	-----	-----	-----	-----	-----	------

Восемь толстых книг и одна ну очень тоненькая на 3 страницы. Среднее арифметическое у такой выборки равно 805 страницам, а медиана — 903. Это влияние той самой книги из трех страниц.

Поэтому правильный ответ — «Да, из-за влияния на среднее экстремально малых значений».

99-й процентиль рейтинга

Есть набор данных о книгах. У каждой книги есть рейтинг — среднее арифметическое по оценкам читателей (от 1 до 5).

Допустим, 99-й процентиль рейтинга по всем книгам равен 4.57. Какие выводы можно из этого сделать?

Выберите все подходящие варианты:

- Меньше 1% книг получили от читателей только «пятерки».
- У большинства книг рейтинг не больше 4.57.
- У 1% книг рейтинг равен 4.57.
- 99% книг получили от читателей только «четверки».

☒ 99-й процентиль рейтинга

Если 99-й процентиль рейтинга по всем книгам равен 4.57, это значит, что у 99% книг рейтинг 4.57 *или меньше*. Следовательно, у оставшегося 1% оценка *больше* 4.57.

Поэтому эти варианты не подходят:

- 99% книг получили от читателей только «четверки».
- У 1% книг рейтинг равен 4.57.

Правильные варианты:

- Меньше 1% книг получили от читателей только «пятерки».
- У большинства книг рейтинг не больше 4.57.

95-й процентиль рейтинга

Есть набор данных о книгах. У каждой книги есть рейтинг по оценкам читателей (от 1 до 5).

У скольких книг рейтинг *не больше* 95-го процентиля? Дайте ответ в процентах от общего количества книг.

☒ 95-й процентиль рейтинга

По определению процентиля: если 95-й процентиль рейтинга равен X , то 95% процентов книг получили рейтинг меньше или равный X . Следовательно, рейтинг не больше 95-го процентиля у 95% книг.

Поэтому правильный ответ — 95%.

Процентили в SQL

Наверно, вы догадались, что я не просто так рассказывал о медиане и процентилях. Конечно, их можно рассчитать и в SQL.

Хорошенько разберитесь в теории, прежде чем переходить к этому разделу. Процентили в SQL считаются несколько необычно. Важно понимать «матчасть», чтобы не запутаться в происходящем.

Вернемся к нашей таблице сотрудников:

id	name	salary
11	Дарья	70
12	Борис	78
21	Елена	84
22	Ксения	90
31	Вероника	96
32	Григорий	96
33	Анна	100
23	Леонид	104
24	Марина	104
25	Иван	120

Как человек, знакомый с функцией `avg()`, я бы предположил, что медиана и 90-й перцентиль считаются как-то так:

```
select
  round(avg(salary)) as average,
  percentile(salary, 50) as median,
  percentile(salary, 90) as p90
from employees;
```

average	median	p90
94	96	106

Но авторы стандарта SQL решили иначе:

```
select
  round(avg(salary)) as average,
  percentile_disc(0.50) within group (order by
salary) as median,
  percentile_disc(0.90) within group (order by
salary) as p90
from employees;
```

Столбец, по которому рассчитывается перцентиль, не указывается аргументом функции. Вместо этого он идет отдельным блоком `within group`, внутри которого задан `order by`. Порог перцентиль указывается не в процентах (50, 90), а десятичной дробью (0.5, 0.9).

Обратите внимание: перцентиль здесь не оконная функция. Пока мы вычисляем его как обычную функцию агрегации (как `avg()` или `sum()` без окна), пусть и со странным синтаксисом.

Помимо `percentile_disc()` существует функция `percentile_cont()`:

- `percentile_disc` рассматривает набор данных как дискретный (то есть состоящий из отдельных значений). Всегда возвращает конкретное значение из тех, что есть в таблице.
- `percentile_cont` рассматривает набор данных как непрерывный (как будто значения в наборе — это выборка из некоторого непрерывного распределения данных). Рассчитывает процентиль аналитически по формуле.

На примере:

```
select
    '25' as perc,
    percentile_disc(0.25) within group (order by
salary) as disc,
    percentile_cont(0.25) within group (order by
salary) as cont
from employees
union all
select
    '50',
    percentile_disc(0.50) within group (order by
salary),
    percentile_cont(0.50) within group (order by
salary)
from employees
union all
select
    '75',
    percentile_disc(0.75) within group (order by
salary),
    percentile_cont(0.75) within group (order by
salary)
from employees
union all
```



```
select
    '90',
    percentile_disc(0.90) within group (order by
salary),
    percentile_cont(0.90) within group (order by
salary)
from employees;
```

perc	disc	cont
25	84	85.5
50	96	96
75	104	103
90	104	105.6

Как видите, `percentile_disc` выдал те значения зарплаты, которые встречаются в столбце `salary`, а `percentile_cont` к ним не привязывался. При этом в целом значения `disc` и `cont` достаточно близки. Как правило, чем больше в наборе данных значений, тем ближе они будут.

С поддержкой в разных СУБД все не очень хорошо:

- SQLite и MySQL вовсе не поддерживают процентиля.
- PostgreSQL поддерживает частично, без «окон» (об этом чуть позже).
- MS SQL и Oracle поддерживают полноценно, с «окнами».

Медиана температуры

Есть таблица `weather` — это среднедневная температура в Лондоне за 6 месяцев 2020 года:

wdate	wtemp
2020-01-01	4.1
2020-01-02	8.6
2020-01-03	9.9
...	...
2020-06-28	17.2
2020-06-29	16.9
2020-06-30	16.2

Напишите запрос, который рассчитает среднее арифметическое, медиану и 90-й процентиль температуры по каждому месяцу:

wmonth	t_avg	t_med	t_p90
1	7.55	7.8	10.6
2	8.24	7.9	11.7
3	...	...	...
4	...	...	...
5	15.26	15.6	20
6	17.72	17.5	24.6

Используйте выражение `extract(month from wdate)`, чтобы извлечь месяц из значений `wdate`. Используйте функцию `percentile_disc()` для расчета медианы и 90-го процентиля.

Песочница: <https://antonz.ru/to#w26pg>

Скачать: <https://antonz.ru/to#w26sql>

☑ Медиана температуры

Группируем по месяцам, считаем медиану и 90-й процентиль для каждого месяца.

```
select
  extract(month from wdate) as wmonth,
  round(avg(wtemp)::decimal, 2) as t_avg,
  percentile_disc(0.5) within group (order by
wtemp) as t_med,
  percentile_disc(0.9) within group (order by
wtemp) as t_p90
from weather
group by extract(month from wdate)
order by wmonth;
```

wmonth	t_avg	t_med	t_p90
1	7.55	7.8	10.6
2	8.24	7.9	11.7
3	7.97	7.6	10.9
4	12.35	12.1	15.9
5	15.26	15.6	20
6	17.72	17.5	24.6

Процентиль как оконная функция

Процентиль как функция агрегации задается так:

```
percentile_disc(PERCENT) within group (order by
COLUMN)
```

Где PERCENT — порог percentиля (десятичная дробь от 0 до 1), а COLUMN — столбец, по которому считается процентиль.

С таким синтаксисом мы легко подсчитаем медиану заработной платы по всем сотрудникам. Но что делать, если нас интересует медиана по каждому департаменту?

Для среднего арифметического мы бы использовали `avg()` поверх окна с секциями по департаментам:

```
select
    name, department, salary,
    round(avg(salary) over w) as dep_avg
from employees
window w as (partition by department)
order by department, id;
```

name	department	salary	dep_avg
Дарья	hr	70	74
Борис	hr	78	74
Елена	it	84	100
Ксения	it	90	100
Леонид	it	104	100
Марина	it	104	100
Иван	it	120	100
Вероника	sales	96	97
Григорий	sales	96	97
Анна	sales	100	97

Аналогично можно использовать и `percentile_disc()`:

```
select
    name, department, salary,
    percentile_disc(0.5)
        within group (order by salary)
        over (partition by department) as dep_p50
from employees
order by department, id;
```

name	department	salary	dep_p50
Дарья	hr	70	70
Борис	hr	78	70
Елена	it	84	104
Ксения	it	90	104
Леонид	it	104	104
Марина	it	104	104
Иван	it	120	104
Вероника	sales	96	96
Григорий	sales	96	96
Анна	sales	100	96

Единственная часть «окошек», которую поддерживает процентиль — секции (`partition by`). Сортировка не поддерживается (она уже задана в `within group`), фреймы тоже. Полный синтаксис выглядит так:

```
percentile_disc(PERCENT)
  within group (order by COLUMN)
  over (partition by OTHER_COLUMNS)
```

К сожалению, секции поддерживаются только в MS SQL и Oracle. В PostgreSQL использовать проценты как оконные функции не получится.

Статистические функции

Оконные функции для расчета статистик:

Function	Description
<code>cume_dist()</code>	процент строк, которые меньше либо равны текущей
<code>percent_rank()</code>	процент строк, которые строго меньше текущей
<code>percentile_disc(N)</code>	N-й процентиль дискретного распределения
<code>percentile_cont(N)</code>	N-й процентиль непрерывного распределения

`cume_dist()` и `percent_rank()` поддерживают сортировку и секции в окне, а `percentile_disc()` и `percentile_cont()` — только секции.

Все статистические функции не поддерживают фреймы.

Так держать

Вы узнали, как считать статистики по набору данных. Осталось подвести итоги первой части книги!

Резюме

Вот задачи, которые решаются с помощью оконных функций в SQL:

- Ранжирование (всевозможные рейтинги).
- Сравнение со смещением (соседние элементы и границы).
- Агрегация (количество, сумма и среднее).
- Скользящие агрегаты (сумма и среднее в динамике).
- Статистика (относительные ранги и сводные показатели).

Оконные функции вычисляют результат по строкам, которые попали в окно. Определение окна указывает, как выглядит окно:

- Из каких секций состоит (`partition by`).
- Как отсортированы строки внутри секции (`order by`).
- Как выглядит фрейм внутри секции (`rows between`).

```

window w as (
    partition by ...
    order by ...
    rows between ... and ...
)

```

`partition by` поддерживается всеми оконными функциями и всегда необязательно. Если не указать — будет одна секция.

`order by` поддерживается всеми оконными функциями (кроме процентилей). Для функций ранжирования и смещения оно обязательно, для агрегации — нет. Если не указать `order by` для функции агрегации — она посчитает обычный агрегат, если указать — скользящий.

Фрейм поддерживается только некоторыми функциями:

- `first_value()`, `last_value()`, `nth_value()`;
- функции агрегации.

Остальные функции фреймы не поддерживают.

Функции ранжирования

Функция	Секции	Сорт.	Фрейм	Описание
<code>row_number()</code>	необ ¹	обяз ²	n/a	порядковый номер строки
<code>dense_rank()</code>	необ	обяз	n/a	ранг строки без пропусков
<code>rank()</code>	необ	обяз	n/a	ранг строки с пропусками
<code>ntile(n)</code>	необ	обяз	n/a	номер группы

¹ необязательно

² обязательно

Функции смещения

Функция	Секции	Сорт.	Фрейм	Описание
<code>lag(v, n)</code>	необ	обяз	n/a	значение n строк назад
<code>lead(v, n)</code>	необ	обяз	n/a	значение n строк вперед
<code>first_value(v)</code>	необ	обяз	обяз	первое значение фрейма
<code>last_value(v)</code>	необ	обяз	обяз	последнее значение фрейма
<code>nth_value(v, n)</code>	необ	обяз	обяз	n-е значение фрейма

Функции агрегации

Функция	Секции	Сорт.	Фрейм	Описание
<code>min(v)</code>	необ	необ	необ	мин. значение секции/фрейма
<code>max(v)</code>	необ	необ	необ	макс. значение секции/фрейма
<code>avg(v)</code>	необ	необ	необ	среднее значение секции/фрейма
<code>count(v)</code>	необ	необ	необ	кол-во строк в секции/фрейме
<code>sum(v)</code>	необ	необ	необ	сумма по секции/фрейму

Функции статистики

Функция	Секции	Сорт.	Фрейм	Описание
<code>cume_dist()</code>	необ	обяз	n/a	кумулятивное распределение
<code>percent_rank()</code>	необ	обяз	n/a	относительный ранг
<code>percentile_disc(n)</code>	необ	w/group ¹	n/a	дискретный процентиль
<code>percentile_cont(n)</code>	необ	w/group	n/a	непрерывный процентиль

Так держать

Отличная работа! Вы освоили одну из самых сложных возможностей SQL — оконные функции. Теперь сможете составлять классные аналитические отчеты, не «допиливая» данные в Excel.

В следующей части книги детально разберем фреймы.

¹ через инструкцию `within group`

Часть 2

Фреймы

ROWS и GROUPS

В главе «Скользящие агрегаты» мы обсудили фреймы. В этой главе разберемся с ними досконально. Сначала освежим пройденное. Фрейм — это группа записей, с которой в данный момент работает оконная функция:

employees		
hr		
Дарья	70	
Борис	78	
it		
Елена	84	← начало фрейма
Ксения	90	
Леонид	104	← текущая запись
Марина	104	
Иван	120	← конец фрейма
sales		
Вероника	96	
Григорий	96	
Анна	100	

rows between frame_start **and** frame_end

Начало фрейма (frame_start) может быть:

- `current row` — фрейм начинается с текущей строки;
- `N preceding` — фрейм начинается с N-й строки перед текущей;
- `N following` — фрейм начинается с N-й строки после текущей;
- `unbounded preceding` — фрейм начинается с начала секции.

Аналогично, конец фрейма (frame_end) может быть:

- `current row` — фрейм продолжается до текущей строки;
- `N preceding` — фрейм продолжается до N-й строки перед текущей;
- `N following` — фрейм продолжается до N-й строки после текущей;
- `unbounded following` — фрейм продолжается до конца секции.

Например, фрейм на картинке выше можно описать так:

rows between 2 preceding and 1 following

rows between unbounded preceding and 1 following

Только у некоторых функций фрейм настраивается:

- функции смещения `first_value()`, `last_value()`, `nth_value()`;
- все функции агрегации: `count()`, `avg()`, `sum()`, ...

У прочих функций фрейм всегда равен секции (а если секция не задана — окну в целом).

Прежде чем двигаться дальше, решим несколько задачек. Будем использовать привычную таблицу `employees`:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

🔗 ROWS-фрейм 1

Есть таблица сотрудников employees:

#	id	name	department
1	11	Дарья	hr
2	12	Борис	hr
3	21	Елена	it
4	22	Ксения	it
5	23	Леонид	it
6	24	Марина	it
7	25	Иван	it
8	31	Вероника	sales
9	32	Григорий	sales
10	33	Анна	sales

-- текущая

В запросе задано окно:

```

window w as (
  order by id
  rows between unbounded preceding and current row
)

```

Укажите начало и конец фрейма, если текущая запись — Леонид.

Ответ в формате N-M, где N и M — номера строк. Например, если фрейм охватывает записи от Ксении до Вероники, то ответ будет 4–8.

Песочница: <https://antonz.ru/to#empdb>

☒ ROWS-фрейм 1

#	id	name	department
1	11	Дарья	hr
2	12	Борис	hr
3	21	Елена	it
4	22	Ксения	it
5	23	Леонид	it
6	24	Марина	it
7	25	Иван	it
8	31	Вероника	sales
9	32	Григорий	sales
10	33	Анна	sales

-- текущая

```

window w as (
  order by id
  rows between unbounded preceding and current row
)

```

Окно:

- секций нет,
- сортировка по id,
- фрейм от начала окна до текущей записи,
- текущая запись — № 5.

Следовательно, фрейм = 1–5

✂ ROWS-фрейм 2

Есть таблица сотрудников employees:

#	id	name	city
1	22	Ксения	Москва
2	24	Марина	Москва
3	25	Иван	Москва
4	31	Вероника	Москва
5	33	Анна	Москва
6	11	Дарья	Самара
7	12	Борис	Самара
8	21	Елена	Самара
9	23	Леонид	Самара
10	32	Григорий	Самара

-- текущая

В запросе задано окно:

```

window w as (
  order by city, id
  rows between 1 preceding and 2 following
)

```

Укажите начало и конец фрейма, если текущая запись — Елена.

Ответ в формате N-M, где N и M — номера строк. Например, если фрейм охватывает записи от Ксении до Вероники, то ответ будет 4–8.

Песочница: <https://antonz.ru/to#empdb>

☑ ROWS-фрейм 2

#	id	name	city
1	22	Ксения	Москва
2	24	Марина	Москва
3	25	Иван	Москва
4	31	Вероника	Москва
5	33	Анна	Москва
6	11	Дарья	Самара
7	12	Борис	Самара
8	21	Елена	Самара
9	23	Леонид	Самара
10	32	Григорий	Самара

-- текущая

```

window w as (
  order by city, id
  rows between 1 preceding and 2 following
)

```

Окно:

- секций нет,
- сортировка по city, id,
- фрейм от -1 до +2 записей относительно текущей,
- текущая запись — № 8.

Следовательно, фрейм = 7–10

✎ ROWS-фрейм 3

Есть таблица сотрудников `employees`. Напишите запрос, который для каждого сотрудника выведет:

- размер з/п предыдущего по зарплате сотрудника (среди коллег по департаменту);
- максимальную з/п по департаменту.

Если «предыдущего коллеги» нет (у сотрудника самая низкая зарплата в департаменте) — запрос должен возвращать зарплату самого сотрудника.

id	name	department	salary	prev_salary	max_salary
11	Дарья	hr	70	70	78
12	Борис	hr	78	70	78
21	Елена	it	84	84	120
22	Ксения	it	90	84	120
23	Леонид	it	104	90	120
24	Марина	it	104	104	120
25	Иван	it	120	104	120
31	Вероника	sales	96	96	100
32	Григорий	sales	96	96	100
33	Анна	sales	100	96	100

Напишите запрос, который это сделает. Не используйте функции `lag()` и `lead()`.

Сортировка результата: `department, salary, id`

Подсказка. Чтобы получить `prev_salary` и `max_salary`, достаточно определить одно окно. Используйте функцию `first_value()` для `prev_salary` и `last_value()` для `max_salary`.

Песочница: <https://antonz.ru/to#empdb>

☑ ROWS-фрейм 3

Окно:

- секции по департаменту;
- сортировка по з/п;
- фрейм от предыдущей строки до конца секции.

С таким окном `first_value()` вернет з/п предыдущего коллеги, а `last_value()` вернет максимальную з/п по департаменту.

```
select
    id, name, department, salary,
    first_value(salary) over w as prev_salary,
    last_value(salary) over w as max_salary
from employees
window w as (
    partition by department order by salary
    rows between 1 preceding and unbounded following
)
order by department, id;
```

id	name	department	salary	prev_salary	max_salary
11	Дарья	hr	70	70	78
12	Борис	hr	78	70	78
21	Елена	it	84	84	120
22	Ксения	it	90	84	120
23	Леонид	it	104	90	120
24	Марина	it	104	104	120
25	Иван	it	120	104	120
31	Вероника	sales	96	96	100
32	Григорий	sales	96	96	100
33	Анна	sales	100	96	100

GROUPS-фреймы

До сих пор мы пользовались таким определением фрейма:

```
rows between frame_start and frame_end
```

На самом деле, кроме фрейма по строкам (`rows`) бывают еще фреймы по группам (`groups`) и диапазону (`range`):

```
rows between frame_start and frame_end
groups between frame_start and frame_end
range between frame_start and frame_end
```

Начнем с фрейма по группам и разберем его на примере. Допустим, мы взяли окно с сортировкой по департаменту и фреймом от начала окна до текущей записи:

```
window w as (  
  order by department  
  rows between unbounded preceding and current row  
)
```

Теперь посчитаем количество записей нарастающим итогом:

```
select  
  name,  
  department,  
  count(*) over w as cnt  
from employees  
window w as (  
  order by department  
  rows between unbounded preceding and current row  
)  
order by department, id;
```

name	department	cnt
Дарья	hr	1
Борис	hr	2
Елена	it	3
Ксения	it	4
Леонид	it	5
Марина	it	6
Иван	it	7
Вероника	sales	8
Григорий	sales	9
Анна	sales	10

А теперь заменим rows на groups в определении фрейма:

```
select
  name, department,
  count(*) over w as cnt
from employees
window w as (
  order by department
  groups between unbounded preceding and current
row
)
order by department, id;
```

name	department	cnt
Дарья	hr	2
Борис	hr	2
Елена	it	7
Ксения	it	7
Леонид	it	7
Марина	it	7
Иван	it	7
Вероника	sales	10
Григорий	sales	10
Анна	sales	10

Разница в том, что rows-фрейм оперирует индивидуальными записями, а groups-фрейм — группами записей, у которых одинаковое значение столбца order by (в данном случае — одинаковый департамент):

ROWS

name	department	cnt
Дарья	hr	1
Борис	hr	2
Елена	it	3
Ксения	it	4
Марина	it	5
Леонид	it	6
Иван	it	7
Вероника	sales	8
Григорий	sales	9
Анна	sales	10

← текущая
запись

GROUPS

name	department	cnt
Дарья	hr	2
Борис	hr	2
Елена	it	7
Ксения	it	7
Марина	it	7
Леонид	it	7
Иван	it	7
Вероника	sales	10
Григорий	sales	10
Анна	sales	10

предыдущая
группа

текущая
группа

следующая
группа

То же самое в динамике:

ROWS

name	department	cnt
ROWS		

GROUPS

name	department	cnt
GROUPS		

Смотреть: <https://antonz.ru/to#w31row>

Смотреть: <https://antonz.ru/to#w31grp>

В обоих случаях запрос на каждом шаге считает количество записей нарастающим итогом: от начала до конца фрейма. Но у `rows` фрейм заканчивается на текущей записи, а у `groups` — на последней записи текущей группы.

Поскольку групп всего три (`hr` — 2 записи, `it` — 5 записей, `sales` — 3 записи), то фрейм смещается «рывками»:

1. `count(*)` для `hr` → 2 записи
2. `count(*)` для `hr + it` → 7 записей
3. `count(*)` для `hr + it + sales` → 10 записей

У групповых фреймов такой же синтаксис границ, как у строковых. Но смысл отличается:

Граница	ROWS	GROUPS
<code>current row</code>	текущая строка	текущая группа
<code>N preceding</code>	N-я строка перед текущей	N-я группа перед текущей
<code>N following</code>	N-я строка после текущей	N-я группа после текущей
<code>unbounded preceding</code>	начало секции	начало секции
<code>unbounded following</code>	конец секции	конец секции

Групповые фреймы поддерживаются в PostgreSQL, SQLite и Oracle 21c+, но не в MySQL и SQL Server.

Закрепим задачками.

✂ GROUPS-фрейм 1

Есть таблица сотрудников `employees`:

#	id	name	department
1	11	Дарья	hr
2	12	Борис	hr
3	21	Елена	it
4	22	Ксения	it
5	23	Леонид	it
6	24	Марина	it
7	25	Иван	it
8	31	Вероника	sales
9	32	Григорий	sales
10	33	Анна	sales

-- текущая

В запросе задано окно:

```
window w as (  
  order by department  
  groups between current row and current row  
)
```

Укажите начало и конец фрейма, если текущая запись — Леонид.

Ответ в формате N-M, где N и M — номера строк. Например, если фрейм охватывает записи от Ксении до Вероники, то ответ будет 4-8.

Песочница: <https://antonz.ru/to#empdb>

☑ GROUPS-фрейм 1

#	id	name	department
1	11	Дарья	hr
2	12	Борис	hr
3	21	Елена	it
4	22	Ксения	it
5	23	Леонид	it
6	24	Марина	it
7	25	Иван	it
8	31	Вероника	sales
9	32	Григорий	sales
10	33	Анна	sales

-- текущая

```
window w as (  
  order by department  
  groups between current row and current row  
)
```

Окно:

- секций нет,
- сортировка по department,
- фрейм равен текущей группе (департамент it).

Следовательно, фрейм = 3–7

✂ GROUPS-фрейм 2

Есть таблица сотрудников employees:

#	id	name	department
1	11	Дарья	hr
2	12	Борис	hr
3	21	Елена	it
4	22	Ксения	it
5	23	Леонид	it
6	24	Марина	it
7	25	Иван	it
8	31	Вероника	sales
9	32	Григорий	sales
10	33	Анна	sales

-- текущая

В запросе задано окно:

```

window w as (
  order by department
  groups between current row and 1 following
)

```

Укажите начало и конец фрейма, если текущая запись — Марина.

Ответ в формате N–M, где N и M — номера строк. Например, если фрейм охватывает записи от Ксении до Вероники, то ответ будет 4–8.

Песочница: <https://antonz.ru/to#empdb>

☑ GROUPS-фрейм 2

#	id	name	department
1	11	Дарья	hr
2	12	Борис	hr
3	21	Елена	it
4	22	Ксения	it
5	23	Леонид	it
6	24	Марина	it
7	25	Иван	it
8	31	Вероника	sales
9	32	Григорий	sales
10	33	Анна	sales

-- текущая

```

window w as (
  order by department
  groups between current row and 1 following
)

```

Окно:

- секций нет,
- сортировка по department,
- фрейм от текущей до следующей группы (департаменты it и sales).

Следовательно, фрейм = 3–10

✂ GROUPS-фрейм 3

Есть таблица сотрудников `employees`. Для каждого человека мы хотим посчитать количество сотрудников, которые получают такую же или большую зарплату (`ge_cnt`):

id	name	salary	ge_cnt
11	Дарья	70	10
12	Борис	78	9
21	Елена	84	8
22	Ксения	90	7
31	Вероника	96	6
32	Григорий	96	6
33	Анна	100	4
23	Леонид	104	3
24	Марина	104	3
25	Иван	120	1

Напишите запрос, который это сделает.

Для простоты не будем отсеивать самого человека. Например, для Ивана с максимальной зарплатой `ge_cnt = 1`, так как сам Иван тоже учтен в расчетах.

Сортировка результата: `salary, id`

Песочница: <https://antonz.ru/to#empdb>

☑ GROUPS-фрейм 3

Окно:

- сортировка по зарплате,
- фрейм от текущей группы до конца окна (сотрудники с такой же или большей зарплатой).

С таким окном `count()` возвращает искомое количество.

```
select
    id, name, salary,
    count(*) over w as ge_cnt
from employees
window w as (
    order by salary
```

groups between current row and unbounded following

```
)
order by salary, id;
```

id	name	salary	ge_cnt
11	Дарья	70	10
12	Борис	78	9
21	Елена	84	8
22	Ксения	90	7
31	Вероника	96	6
32	Григорий	96	6
33	Анна	100	4
23	Леонид	104	3
24	Марина	104	3
25	Иван	120	1

GROUPS-фрейм 4

Есть таблица сотрудников employees. Для каждого человека мы хотим увидеть ближайшую большую зарплату (next_salary):

id	name	salary	next_salary
11	Дарья	70	78
12	Борис	78	84
21	Елена	84	90
22	Ксения	90	96
31	Вероника	96	100
32	Григорий	96	100
33	Анна	100	104
23	Леонид	104	120
24	Марина	104	120
25	Иван	120	

Напишите запрос, который это сделает.

Сортировка результата: `salary, id`

Пустая ячейка у Ивана — это `null`.

Подсказка. Если испытываете проблемы с Иваном, попробуйте смотреть в окне только вперед. Фрейм не должен включать текущую строку.

Песочница: <https://antonz.ru/to#empdb>

☑ GROUPS-фрейм 4

Окно:

- сортировка по зарплате,
- фрейм равен следующей группе (сотрудники с ближайшей большей зарплатой).

С таким окном `first_value()` возвращает ближайшую большую зарплату.

Вместо `first_value` можно использовать `last_value`, `nth_value`, или функцию агрегации (`min`, `max`) — по такому фрейму все они вернут ближайшую большую зарплату относительно текущей.

```
select
  id, name, salary,
  first_value(salary) over w as next_salary
from employees
window w as (
  order by salary
  groups between 1 following and 1 following
)
order by salary, id;
```

id	name	salary	next_salary
11	Дарья	70	78
12	Борис	78	84
21	Елена	84	90
22	Ксения	90	96
31	Вероника	96	100
32	Григорий	96	100
33	Анна	100	104
23	Леонид	104	120
24	Марина	104	120
25	Иван	120	

Так держать

Мы рассмотрели строковые (`rows`) и групповые (`groups`) фреймы:

- `rows`—фрейм оперирует отдельными записями;
- `groups`—фрейм оперирует группами записей (группа включает все строки с одинаковым значением столбцов из `order by`).

Групповые фреймы есть смысл использовать, если в `order by` окна указан неуникальный набор столбцов. Например, `order by department` или `order by city` создает группы строк с одинаковым департаментом или городом — и тут групповой фрейм уместен.

А `order by department, id` задает уникальную сортировку (не может быть двух строк с одинаковым значением `id`) — так что здесь `groups` не имеет смысла.

Остался еще один тип фрейма — диапазонный (`range`). Займемся им в следующей главе!

RANGE-фреймы

Последний тип фрейма — фрейм по диапазону:

```
range between frame_start and frame_end
```

Если rows-фрейм оперирует строками, а groups — группами строк с одинаковыми значениями, то range — группами строк, которые попадают в диапазон. Например, возьмем такое окно:

```
window w as (  
    order by salary  
    range between 10 preceding and 10 following  
)
```

Фрейм для записи со значением salary = X будет включать сотрудников с диапазоном зарплат от X-10 до X+10 включительно. Например, для Григория с зарплатой 96K Р — сотрудников с з/п от 86K до 106K Р:

name	salary	
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	-- начало фрейма
Вероника	96	
Григорий	96	-- текущая запись
Анна	100	
Леонид	104	
Марина	104	
Иван	120	-- конец фрейма

Если рассчитать фрейм для каждой записи, получится так:

name	salary	-- какую з/п включает фрейм
Дарья	70	-- 60-80 Дарья, Борис
Борис	78	-- 68-88 Дарья, Борис, Елена
Елена	84	-- 74-94 Борис, Елена, Ксения
Ксения	90	-- 80-100 Еле, Ксе, Вер, Гри, Анн
Вероника	96	-- 86-106 Ксе, Вер, Гри, Анн, Лео, Мар
Григорий	96	-- 86-106 Ксе, Вер, Гри, Анн, Лео, Мар
Анна	100	-- 90-110 Ксе, Вер, Гри, Анн, Лео, Мар
Леонид	104	-- 94-114 Вер, Гри, Анн, Лео, Мар
Марина	104	-- 94-114 Вер, Гри, Анн, Лео, Мар
Иван	120	-- 110-120 Иван

Проверим и посчитаем количество сотрудников по каждому диапазону:

```
select
  name, salary, count(*) over w as cnt
from employees
window w as (
  order by salary
  range between 10 preceding and 10 following
)
order by salary, id;
```

name	sal	cnt	
Дарья	70	2	-- 60-80 Дарья, Борис
Борис	78	3	-- 68-88 Дарья, Борис, Елена
Елена	84	3	-- 74-94 Борис, Елена, Ксения
Ксения	90	5	-- 80-100 Еле, Ксе, Вер, Гри, Анн
Вероника	96	6	-- 86-106 Ксе, Вер, Гри, Анн, Лео, Мар
Григорий	96	6	-- 86-106 Ксе, Вер, Гри, Анн, Лео, Мар
Анна	100	6	-- 90-110 Ксе, Вер, Гри, Анн, Лео, Мар
Леонид	104	5	-- 94-114 Вер, Гри, Анн, Лео, Мар
Марина	104	5	-- 94-114 Вер, Гри, Анн, Лео, Мар
Иван	120	1	-- 110-120 Иван

Анимированная версия:

name	salary	cnt
RANGE		

Смотреть: <https://antonz.ru/to#w32rng>

Все сходится!

Особенности диапазонов

У диапазонных фреймов несколько особенностей:

1. Только один столбец в **order by**

Поскольку range-фрейм динамически рассчитывается по вхождению в диапазон `between .. and ..`, то в `order by` должен быть ровно один столбец. Так не получится:

```
window w as (  
  order by salary, city  
  range between 10 preceding and 10 following  
)
```

2. Только числа или даты для **N preceding / following**

Условия `N preceding` и `N following` работают только для числовых столбцов и столбцов с датами.

Например, такой фрейм лишен смысла:

```

window w as (
  order by department
  range between 10 preceding and 10 following
)

```

(о датах поговорим отдельно)

3. current row — как у groups-фрейма

Условие `current row` для `range`-фрейма работает так же, как для `groups`-фрейма — включает строки с одинаковым значением столбца из `order by`.

Условия `unbounded preceding` и `unbounded following` для всех типов фреймов работают одинаково — включает строки от начала секции (`unbounded preceding`) и до конца секции (`unbounded following`).

RANGE-фрейм 1

Есть таблица сотрудников `employees`:

#	name	salary
1	Дарья	70
2	Борис	78
3	Елена	84
4	Ксения	90
5	Вероника	96
6	Григорий	96
7	Анна	100
8	Леонид	104
9	Марина	104
10	Иван	120

-- current

В запросе задано окно:

```

window w as (
  order by salary
  range between 3 preceding and 5 following
)

```

Укажите начало и конец фрейма, если текущая запись — Вероника.

Ответ в формате N-M, где N и M — номера строк. Например, если фрейм охватывает записи от Ксении до Леонида, то ответ будет 4-8.

Песочница: <https://antonz.ru/to#empdb>

☒ RANGE-фрейм 1

#	name	salary
1	Дарья	70
2	Борис	78
3	Елена	84
4	Ксения	90
5	Вероника	96
6	Григорий	96
7	Анна	100
8	Леонид	104
9	Марина	104
10	Иван	120

-- current

```

window w as (
  order by salary
  range between 3 preceding and 5 following
)

```

Окно:

- секций нет,
- сортировка по зарплате,
- фрейм по диапазону от -3 до +5 (зарплата 93–101K),
- текущая запись — № 5.

Следовательно, фрейм = 5–7

RANGE-фрейм 2

Есть таблица сотрудников employees:

#	name	salary
1	Дарья	70
2	Борис	78
3	Елена	84
4	Ксения	90
5	Вероника	96
6	Григорий	96
7	Анна	100
8	Леонид	104
9	Марина	104
10	Иван	120

-- current

В запросе задано окно:

```

window w as (
  order by salary
  range between current row and current row
)

```

Укажите начало и конец фрейма, если текущая запись — Марина.

Ответ в формате N-M, где N и M — номера строк. Например, если фрейм охватывает записи от Ксении до Леонида, то ответ будет 4–8.

Песочница: <https://antonz.ru/to#empdb>

☑ RANGE-фрейм 2

#	name	salary
1	Дарья	70
2	Борис	78
3	Елена	84
4	Ксения	90
5	Вероника	96
6	Григорий	96
7	Анна	100
8	Леонид	104
9	Марина	104
10	Иван	120

-- current

```

window w as (
  order by salary
  range between current row and current row
)

```

Окно:

- секций нет,
- сортировка по зарплате,
- фрейм равен зарплате от текущей записи (104K),
- текущая запись — № 9.

Следовательно, фрейм = 8–9

RANGE-фрейм 3

Есть таблица сотрудников `employees`. Для каждого человека мы хотим посчитать количество сотрудников, которые получают такую же или большую зарплату, но не более чем +10 тыс. Р (`p10_cnt`):

id	name	salary	p10_cnt
11	Дарья	70	2
12	Борис	78	2
21	Елена	84	2
22	Ксения	90	4
31	Вероника	96	5
32	Григорий	96	5
33	Анна	100	3
23	Леонид	104	2
24	Марина	104	2
25	Иван	120	1

Напишите запрос, который это сделает.

Для простоты не будем отсеивать самого человека. Например, для Ивана с максимальной зарплатой `p10_cnt = 1`, так как сам Иван тоже учтен в расчетах.

Сортировка результата: `salary, id`

Песочница: <https://antonz.ru/to#empdb>

☑ RANGE-фрейм 3

Окно:

- сортировка по зарплате,
- фрейм по диапазону от текущей з/п до з/п + 10К Р.

С таким окном `count()` возвращает искомое количество сотрудников.

```
select
    id, name, salary,
    count(*) over w as p10_cnt
from employees
window w as (
    order by salary
    range between current row and 10 following
)
order by salary, id;
```

id	name	salary	p10_cnt
11	Дарья	70	2
12	Борис	78	2
21	Елена	84	2
22	Ксения	90	4
31	Вероника	96	5
32	Григорий	96	5
33	Анна	100	3
23	Леонид	104	2
24	Марина	104	2
25	Иван	120	1

✎ RANGE-фрейм 4

Есть таблица сотрудников `employees`. Для каждого человека мы хотим определить максимальную зарплату среди тех, у кого зарплата на 10–30 тыс. ₽ меньше (`lower_sal`):

id	name	salary	lower_sal
11	Дарья	70	
12	Борис	78	
21	Елена	84	70
22	Ксения	90	78
31	Вероника	96	84
32	Григорий	96	84
33	Анна	100	90
23	Леонид	104	90
24	Марина	104	90
25	Иван	120	104

Напишите запрос, который это сделает.

Сортировка результата: `salary, id`

Пустые ячейки у Дарьи и Бориса — это `null`.

Песочница: <https://antonz.ru/to#empdb>

☑ RANGE-фрейм 4

Окно:

- сортировка по зарплате,
- фрейм по диапазону от –30К до –10К относительно текущей зарплаты.

С таким окном `last_value()` возвращает максимальную зарплату из диапазона.

```
select
  id, name, salary,
  last_value(salary) over w as lower_sal
from employees
window w as (
  order by salary
  range between 30 preceding and 10 preceding
)
order by salary, id;
```

id	name	salary	lower_sal
11	Дарья	70	
12	Борис	78	
21	Елена	84	70
22	Ксения	90	78
31	Вероника	96	84
32	Григорий	96	84
33	Анна	100	90
23	Леонид	104	90
24	Марина	104	90
25	Иван	120	104

Границы фрейма

Все типы фреймов — `rows`, `groups` и `range` — используют одни и те же инструкции, чтобы задать границы:

- `unbounded preceding / following`,
- `N preceding / following`,
- `current row`.

Но трактовка этих инструкций может отличаться в зависимости от типа фрейма.

Инструкции `unbounded preceding` и `unbounded following` всегда означают границы секции:

ROWS, unbounded following

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

GROUPS, unbounded following

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

RANGE, unbounded following

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

`current row` означает:

- для строковых фреймов: текущая запись;
- для групповых фреймов: текущая запись и все равные ей;
- для диапазонных фреймов: как для групповых.

ROWS, current row

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

GROUPS, current row

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

RANGE, current row

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

`N preceding` и `N following` отличаются для каждого типа фрейма:

- для строковых фреймов: количество записей до/после текущей;
- для групповых фреймов: количество групп до/после текущей;

- для диапазонных фреймов: диапазон значений относительно текущей записи.

ROWS, 2 following

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

GROUPS, 2 following

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

RANGE, 15 following

name	salary ↑	cnt
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

Фрейм по умолчанию

Возьмем запрос, который считает зарплату по организации нарастающим итогом:

```
select
    name, salary,
    sum(salary) over w as "sum"
from employees
window w as (order by salary)
order by salary, id;
```

name	salary	sum
Дарья	70	70
Борис	78	148
Елена	84	232
Ксения	90	322
Вероника	96	514
Григорий	96	514
Анна	100	614
Леонид	104	822
Марина	104	822
Иван	120	942

Тут сразу возникает пара вопросов:

1. Мы не указали фрейм. Как тогда работает запрос?
2. Почему `sum` одинаковая для Вероники и Григория (а также Леонида и Марины)?

Ответ на оба вопроса один. Если фрейм не указан явно, используется такой:

range between unbounded preceding and current row

Умолчательный фрейм охватывает записи от начала секции до конца текущей группы (группа образована из записей с одинаковыми значениями `order by`). Вот как выглядят фреймы для Вероники и Леонида:

Вероника

name	salary ↑	sum
Дарья	70	70
Борис	78	148
Елена	84	232
Ксения	90	322
Вероника	96	514
Григорий	96	514
Анна	100	614
Леонид	104	822
Марина	104	822
Иван	120	942

Леонид

name	salary ↑	sum
Дарья	70	70
Борис	78	148
Елена	84	232
Ксения	90	322
Вероника	96	514
Григорий	96	514
Анна	100	614
Леонид	104	822
Марина	104	822
Иван	120	942

Выбор `range` вместо `rows` в качестве фрейма по умолчанию может показаться странным. Глядя на запрос с суммой нарастающим итогом, не ожидаешь, что он посчитает одинаковую сумму для разных строчек. Но такое уж решение принял комитет по разработке стандарта SQL.

Все сказанное относится только к функциям, у которых фрейм настраивается:

- функции смещения `first_value()`, `last_value()`, `nth_value()`;
- функции агрегации `count()`, `avg()`, `sum()`, ...

У прочих функций фрейм всегда равен секции.

Часто задаваемые вопросы о фреймах

1. Правильно ли я понимаю: `rows` исходит из количества строк в указании границ фрейма, а `rows` — из количества значений, входящих в диапазон?

`range` смотрит на само значение, а не на количество. А так да.

2. Как `range` работает с датами?

SQLite его не поддерживает. MySQL и PostgreSQL используют собственный синтаксис дат.

Например, PostgreSQL:

```
range between interval '1 day' preceding  
and interval '10 days' following
```

Или MySQL:

```
range between interval 1 day preceding  
and interval 10 day following
```

Чтобы работать с датами в SQLite, их хранят в `unixtime` как целое число, либо преобразуют в числа внутри запроса.

3. В главе «Скользящие агрегаты» мы затрагивали тему фрейма по умолчанию. Вы там написали: *«Фрейм по умолчанию в нашем запросе распространяется от первой до текущей записи. Поэтому результаты совпадают с запросом, где фрейм был указан явно: `rows between unbounded preceding and current row`»*

Теперь вы пишете, что фрейм по умолчанию — не `rows`, а `range`. Как так?

Фрейм по умолчанию всегда `range`. Вот что я писал в главе про скользящие агрегаты:

1. фрейм по умолчанию в запросе распространяется от первой до текущей записи (что правда);
2. результаты совпадают с `rows`-фреймом (что тоже правда).

Так держать

Мы рассмотрели все типы фреймов:

- строковые (`rows`) оперируют отдельными записями;
- групповые (`groups`) оперируют группами однокровных записей;
- диапазонные (`range`) оперируют группами значений, которые попадают в диапазон.

Теперь посмотрим, как можно их отфильтровать!

EXCLUDE

Как мы выяснили в предыдущих главах, фреймы бывают трех типов: строковые, групповые и диапазонные. Теперь определение фрейма выглядит так:

```
{ rows | groups | range } between frame_start  
and frame_end
```

Обычно этого достаточно. Но иногда хочется исключить из фрейма часть записей.

Продолжим использовать таблицу `employees`:

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Допустим, мы хотим понять, как изменится средняя зарплата по организации, если уволить того или иного сотрудника. Как посчитать среднюю зарплату, мы знаем:

```
select
  name, salary,
  round(avg(salary) over w) as "avg"
from employees
window w as ()
order by salary, id;
```

name	salary	avg
Дарья	70	94
Борис	78	94
Елена	84	94
Ксения	90	94
Вероника	96	94
Григорий	96	94
Анна	100	94
Леонид	104	94
Марина	104	94
Иван	120	94

Но как для каждого сотрудника посчитать среднее, исключая его самого?

Для начала перепишем определение окна с явным указанием фрейма:

```
window w as (  
    rows between unbounded preceding and unbounded  
following  
)
```

Теперь исключим из фрейма текущую запись с помощью `exclude`:

```
window w as (  
    rows between unbounded preceding and unbounded  
following  
    exclude current row  
)
```

Вот как поменялась ситуация на примере Вероники:

Без EXCLUDE

name	salary ↑	avg
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

EXCLUDE current row

name	salary ↑	avg
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

Теперь фрейм включает все записи, кроме текущей. А функция `avg()` считает среднюю зарплату для всех коллег сотрудника, исключая его самого. Ровно то, что нам нужно!

Запрос целиком и результат:

```
select
  name, salary,
  round(avg(salary) over w) as "avg"
from employees
window w as (
  rows between unbounded preceding and unbounded
following
  exclude current row
)
order by salary, id;
```

name	salary	avg
Дарья	70	97
Борис	78	96
Елена	84	95
Ксения	90	95
Вероника	96	94
Григорий	96	94
Анна	100	94
Леонид	104	93
Марина	104	93
Иван	120	91

`exclude` поддерживается в PostgreSQL, SQLite и Oracle 21c+, но не в MySQL и SQL Server.

Виды исключений

Стандарт SQL предусматривает четыре разновидности `exclude`:

1. `exclude no others`. Ничего не исключать. Вариант по умолчанию: если явно не указать `exclude`, сработает именно он.
2. `exclude current row`. Исключить текущую запись (как мы сделали на предыдущем шаге с сотрудником).
3. `exclude group`. Исключить текущую запись и все равные ей (по значению столбцов из `order by`).
4. `exclude ties`. Оставить текущую запись, но исключить равные ей.

На примере Вероники (окно отсортировано по зарплате):

EXCLUDE NO OTHERS

name	salary ↑	avg
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

EXCLUDE CURRENT ROW

name	salary ↑	avg
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

EXCLUDE GROUP

name	salary ↑	avg
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

EXCLUDE TIES

name	salary ↑	avg
Дарья	70	
Борис	78	
Елена	84	
Ксения	90	
Вероника	96	
Григорий	96	
Анна	100	
Леонид	104	
Марина	104	
Иван	120	

Exclude group / ties

Вернемся к запросу о средней зарплате. Вариант с `exclude current row` отвечал на вопрос:

Как изменится средняя зарплата по организации, если уволить конкретного сотрудника?

```
-- exclude current row
```

id	name	sal	avg	frame	exc
11	Дарья	70	97	12, 21-22, 31-33, 23-25	11
12	Борис	78	96	11, 21-22, 31-33, 23-25	12
21	Елена	84	95	11-12, 22, 31-33, 23-25	21
22	Ксения	90	95	11-12, 21, 31-33, 23-25	22
31	Вероника	96	94	11-12, 21-22, 32, 33, 23-25	31
32	Григорий	96	94	11-12, 21-22, 31, 33, 23-25	32
33	Анна	100	94	11-12, 21-22, 31, 32, 23-25	33
23	Леонид	104	93	11-12, 21-22, 31-33, 24, 25	23
24	Марина	104	93	11-12, 21-22, 31-33, 23, 25	24
25	Иван	120	91	11-12, 21-22, 31-33, 23, 24	25

- `frame` показывает содержимое фрейма для каждой записи;
- `exc` показывает, кого исключили.

Если использовать `exclude group` — получим ответ на вопрос:

Как изменится средняя зарплата по организации, если уволить конкретного сотрудника и всех, кто получает столько же, сколько и он?

```
-- exclude group
```

id	name	sal	avg	frame	exc
11	Дарья	70	97	12, 21-22, 31-33, 23-25	11
12	Борис	78	96	11, 21-22, 31-33, 23-25	12
21	Елена	84	95	11-12, 22, 31-33, 23-25	21
22	Ксения	90	95	11-12, 21, 31-33, 23-25	22
31	Вероник	96	94	11-12, 21-22, 33, 23-25	31-32
32	Григор	96	94	11-12, 21-22, 33, 23-25	31-32
33	Анна	100	94	11-12, 21-22, 31, 32, 23-25	33
23	Леонид	104	93	11-12, 21-22, 31-33, 25	23-24
24	Марина	104	93	11-12, 21-22, 31-33, 25	23-24
25	Иван	120	91	11-12, 21-22, 31-33, 23, 24	25

Если же использовать `exclude ties` — получим ответ на вопрос:

Как изменится средняя зарплата по организации, если уволить всех, кто получает столько же, сколько конкретный сотрудник, но не увольнять его самого?

```
-- exclude ties
```

id	name	sal	avg	frame	exc
11	Дарья	70	97	12, 21-22, 31-33, 23-25	
12	Борис	78	96	11, 21-22, 31-33, 23-25	
21	Елена	84	95	11-12, 22, 31-33, 23-25	
22	Ксения	90	95	11-12, 21, 31-33, 23-25	
31	Вероника	96	94	11-12, 21-22, 31, 33, 23-25	32
32	Григорий	96	94	11-12, 21-22, 32, 33, 23-25	31
33	Анна	100	94	11-12, 21-22, 31, 32, 23-25	
23	Леонид	104	93	11-12, 21-22, 31-33, 23, 25	24
24	Марина	104	93	11-12, 21-22, 31-33, 24, 25	23
25	Иван	120	91	11-12, 21-22, 31-33, 23, 24	

Тип фрейма и вид исключения

Возможно, вы заметили, что в `exclude` фигурируют понятия текущей записи (`current row`) и группы (`group`). Похожее разделение мы видели в типах фреймов: `rows`-фреймы оперируют записями, а `groups`-фреймы — группами.

Но! `exclude` и типы фреймов никак между собой не связаны:

1. Какой бы тип фрейма вы не использовали — `exclude current row` исключает только текущую запись, не группу.
2. Какой бы тип фрейма вы не использовали — `exclude group` исключает всю группу.

Порешаем задачки, чтобы закрепить материал.

🔪 EXCLUDE 1

Есть таблица сотрудников `employees`:

id	name	department
11	Дарья	hr
12	Борис	hr
21	Елена	it
22	Ксения	it
23	Леонид	it
24	Марина	it
25	Иван	it
31	Вероника	sales
32	Григорий	sales
33	Анна	sales

-- текущая

В запросе задано окно:

```
window w as (  
  order by department, id  
  rows between 2 preceding and 2 following  
  exclude current row  
)
```

Укажите идентификаторы записей, которые входят во фрейм, если текущая запись — Леонид.

Ответ в формате ID1, ID2, ID3, ... IDn, где IDx — идентификатор сотрудника. Например, если фрейм включает Ксению, Леонида и Марину, то ответ будет 22, 23, 24.

Песочница: <https://antonz.ru/to#empdb>

☒ EXCLUDE 1

id	name	department
11	Дарья	hr
12	Борис	hr
21	Елена	it
22	Ксения	it
23	Леонид	it
24	Марина	it
25	Иван	it
31	Вероника	sales
32	Григорий	sales
33	Анна	sales

-- текущая

```
window w as (  
  order by department, id  
  rows between 2 preceding and 2 following  
  exclude current row  
)
```

Окно:

- секций нет;
- сортировка по department, id;
- фрейм от -2 до +2 записей (21–25);
- кроме текущей записи (23).

Следовательно, фрейм = 21, 22, 24, 25

✂ EXCLUDE 2

Есть таблица сотрудников `employees`:

id	name	department
11	Дарья	hr
12	Борис	hr
21	Елена	it
22	Ксения	it
23	Леонид	it
24	Марина	it
25	Иван	it
31	Вероника	sales
32	Григорий	sales
33	Анна	sales

-- текущая

В запросе задано окно:

```
window w as (  
  partition by department  
  order by id  
  rows between unbounded preceding and current  
row  
  exclude current row  
)
```

Укажите идентификаторы записей, которые входят во фрейм, если текущая запись — Марина.

Ответ в формате `ID1, ID2, ID3, ... IDn`, где `IDx` — идентификатор сотрудника. Например, если фрейм включает Ксению, Леонида и Марину, то ответ будет `22, 23, 24`.

Песочница: <https://antonz.ru/to#empdb>

☑ EXCLUDE 2

id	name	department
11	Дарья	hr
12	Борис	hr
21	Елена	it
22	Ксения	it
23	Леонид	it
24	Марина	it
25	Иван	it
31	Вероника	sales
32	Григорий	sales
33	Анна	sales

-- текущая

```

window w as (
  partition by department
  order by id
  rows between unbounded preceding and current
row
  exclude current row
)

```

Окно:

- секция по department;
- сортировка по id;
- фрейм от начала секции до текущей записи (21–24);
- кроме текущей записи (24).

Следовательно, фрейм = 21, 22, 23

🔪 EXCLUDE 3

Есть таблица сотрудников `employees`:

id	name	salary
11	Дарья	70
12	Борис	78
21	Елена	84
22	Ксения	90
31	Вероника	96
32	Григорий	96
33	Анна	100
23	Леонид	104
24	Марина	104
25	Иван	120

-- текущая

В запросе задано окно:

```
window w as (  
  order by salary  
  range between 10 preceding and 10 following  
  exclude group  
)
```

Укажите идентификаторы записей, которые входят во фрейм, если текущая запись — Вероника.

Ответ в формате `ID1, ID2, ID3, ... IDn`, где `IDx` — идентификатор сотрудника. Например, если фрейм включает Ксению, Леонида и Марину, то ответ будет `22, 23, 24`.

Песочница: <https://antonz.ru/to#empdb>

☑ EXCLUDE 3

id	name	salary
11	Дарья	70
12	Борис	78
21	Елена	84
22	Ксения	90
31	Вероника	96
32	Григорий	96
33	Анна	100
23	Леонид	104
24	Марина	104
25	Иван	120

-- текущая

```

window w as (
  order by salary
  range between 10 preceding and 10 following
  exclude group
)

```

Окно:

- без секций;
- сортировка по salary;
- фрейм по зарплате от -10K до +10K (salary 86–106 = id 22, 31, 32, 33, 23, 24);
- кроме текущей группы (31, 32).

Следовательно, фрейм = 22, 33, 23, 24

✂ EXCLUDE 4

Есть таблица сотрудников employees:

id	name	city	department
22	Ксения	Москва	it
24	Марина	Москва	it
25	Иван	Москва	it
31	Вероника	Москва	sales
33	Анна	Москва	sales
11	Дарья	Самара	hr
12	Борис	Самара	hr
21	Елена	Самара	it
23	Леонид	Самара	it
32	Григорий	Самара	sales

-- текущая

В запросе задано окно:

```

window w as (
  partition by city
  order by department
  groups between current row and unbounded
following
  exclude ties
)

```

Укажите идентификаторы записей, которые входят во фрейм, если текущая запись — Елена.

Ответ в формате ID1, ID2, ID3, ... IDn, где IDx — идентификатор сотрудника. Например, если фрейм включает Ксению, Леонида и Марину, то ответ будет 22, 23, 24.

Песочница: <https://antonz.ru/to#empdb>

☑ EXCLUDE 4

id	name	city	department
22	Ксения	Москва	it
24	Марина	Москва	it
25	Иван	Москва	it
31	Вероника	Москва	sales
33	Анна	Москва	sales
11	Дарья	Самара	hr
12	Борис	Самара	hr
21	Елена	Самара	it
23	Леонид	Самара	it
32	Григорий	Самара	sales

-- текущая

```

window w as (
  partition by city
  order by department
  groups between current row and unbounded
following
  exclude ties
)

```

Окно:

- секция по city;
- сортировка по department;
- фрейм от текущей группы до конца секции (it и sales в Самаре = id 21, 23, 32);
- кроме равных по департаменту текущей записи (23).

Следовательно, фрейм frame = 21, 32

🔪 EXCLUDE 5

Есть таблица сотрудников `employees`. Для каждого человека мы хотим посчитать среднюю зарплату сотрудников, которые получают столько же или больше, чем он — но не более чем +20 тыс. Р (`p20_sal`). При этом зарплату самого сотрудника учитывать не следует:

id	name	salary	p20_sal
11	Дарья	70	84
12	Борис	78	92
21	Елена	84	98
22	Ксения	90	100
31	Вероника	96	101
32	Григорий	96	101
33	Анна	100	109
23	Леонид	104	112
24	Марина	104	112
25	Иван	120	

Напишите запрос, который это сделает.

Сортировка результата: `salary, id`

Пустая ячейка у Ивана — это `null`.

Песочница: <https://antonz.ru/to#empddb>

☑ EXCLUDE 5

Окно:

- сортировка по зарплате;
- фрейм по диапазону от текущей зарплаты до +20К Р;
- кроме текущего сотрудника.

```
select
  id, name, salary,
  round(avg(salary) over w) as p20_sal
from employees
window w as (
  order by salary
  range between current row and 20 following
  exclude current row
)
order by salary, id;
```

id	name	salary	p20_sal
11	Дарья	70	84
12	Борис	78	92
21	Елена	84	98
22	Ксения	90	100
31	Вероника	96	101
32	Григорий	96	101
33	Анна	100	109
23	Леонид	104	112
24	Марина	104	112
25	Иван	120	

Так держать

Теперь вы умеете отсеивать в фреймах записи текущей группы. В следующей главе посмотрим, как еще можно отфильтровать фрейм!

FILTER

Продолжим работать с таблицей `employees`:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Песочница: <https://antonz.ru/to#empdb>

Скачать: <https://antonz.ru/to#empsql>

В предыдущей главе мы задавались вопросом, как изменится средняя зарплата, если уволить того или иного сотрудника. А теперь руководство размышляет: как изменится общий фонд оплаты труда, если кого-то одного уволить, а остальным поднять зарплату на 10%?

Что ж, это несложно посчитать:

```
select
    name, department, salary,
    sum(salary) over () as "база",
    sum(salary) over w as "+0%",
    sum(salary*1.1) over w as "+10%"
from employees
window w as (
```

```

    rows between unbounded preceding and unbounded
following
    exclude current row
)
order by id;

```

- база показывает ФОТ без увольнений;
- +0% показывает ФОТ с увольнением без повышения з/п;
- +10% показывает ФОТ с увольнением и повышением остальных з/п на 10%.

name	department	salary	база	+0%	+10%
Дарья	hr	70	942	872	959.2
Борис	hr	78	942	864	950.4
Елена	it	84	942	858	943.8
Ксения	it	90	942	852	937.2
Леонид	it	104	942	838	921.8
Марина	it	104	942	838	921.8
Иван	it	120	942	822	904.2
Вероника	sales	96	942	846	930.6
Григорий	sales	96	942	846	930.6
Анна	sales	100	942	842	926.2

Пока директор думает, хватит ли денег на вариант «+10%», какая-то светлая голова предлагает — что если уволить всех айтишников, а остальным поднять аж на 50%? А ИТ можно на аутсорс отдать.

Можно, конечно, сделать отдельный запрос с условием `department <> 'it'`, а затем соединить результат с предыдущим по `id` сотрудника. Но есть вариант проще — благодаря инструкции `filter`:

```

select
...
sum(salary*1.5)
  filter(where department <> 'it')

```

```

    over () as "+50% без ИТ"
from employees
order by id;

```

filter работает как обычное условие where, но фильтрует не все записи запроса, а только фрейм для конкретной оконной функции. В нашем случае — игнорирует сотрудников ИТ-департамента при подсчете суммарной зарплаты в столбце +50% без ИТ.

Запрос целиком и результат:

```

select
    name, department, salary,
    sum(salary) over () as "база",
    sum(salary) over w as "+0%",
    sum(salary*1.1) over w as "+10%",
    sum(salary*1.5)
        filter(where department <> 'it')
        over () as "+50% без ИТ"
from employees
window w as (
    rows between unbounded preceding and unbounded
following
    exclude current row
)
order by id;

```

name	depar	salary	база	+0%	+10%	+50% б/ИТ
Дарья	hr	70	942	872	959.2	660.0
Борис	hr	78	942	864	950.4	660.0
Елена	it	84	942	858	943.8	660.0
Ксения	it	90	942	852	937.2	660.0
Леонид	it	104	942	838	921.8	660.0
Марина	it	104	942	838	921.8	660.0
Иван	it	120	942	822	904.2	660.0
Вероника	sales	96	942	846	930.6	660.0
Григорий	sales	96	942	846	930.6	660.0
Анна	sales	100	942	842	926.2	660.0

Вот это экономия!

`filter` поддерживается в PostgreSQL и SQLite, но не в Oracle, MySQL и SQL Server.

🔗 Сравнить з/п со средней по городу

Есть таблица сотрудников `employees`. Для каждого сотрудника мы посчитали, сколько процентов составляет его зарплата от средней по компании:

```
select
    id, name, salary,
    round(salary * 100 / avg(salary) over ()) as perc
from employees
order by id;
```

Теперь хотим посчитать, сколько процентов составляет зарплата сотрудника от средней по Москве (`msk`) и средней по Самаре (`sam`). Должно получиться так:

id	name	salary	perc	msk	sam
11	Дарья	70	74	69	81
12	Борис	78	83	76	90
21	Елена	84	89	82	97
22	Ксения	90	96	88	104
23	Леонид	104	110	102	120
24	Марина	104	110	102	120
25	Иван	120	127	118	139
31	Вероника	96	102	94	111
32	Григорий	96	102	94	111
33	Анна	100	106	98	116

Напишите запрос, который это сделает.

Сортировка результата: `id`

Песочница: <https://antonz.ru/to#empdb>

☑ Сравнить з/п со средней по городу

Используем окно на всю таблицу и отдельные вызовы `filter()` для Москвы и Самары.

```
select
    id, name, salary,
    round(salary * 100 / avg(salary) over ()) as
"perc",
    round(salary * 100 / avg(salary)
        filter(where city = 'Москва')
        over ()) as "msk",
    round(salary * 100 / avg(salary)
        filter(where city = 'Самара')
        over ()) as "sam"
from employees
order by id;
```

id	name	salary	perc	msk	sam
11	Дарья	70	74	69	81
12	Борис	78	83	76	90
21	Елена	84	89	82	97
22	Ксения	90	96	88	104
23	Леонид	104	110	102	120
24	Марина	104	110	102	120
25	Иван	120	127	118	139
31	Вероника	96	102	94	111
32	Григорий	96	102	94	111
33	Анна	100	106	98	116

CASE как альтернатива FILTER

Вернемся к расчетам об увольнении и повышении. Поразмыслив, директор пришел к выводу, что увольнять

весь ИТ-департамент — это чересчур. Теперь его интересует такой вариант:

- увольняем кого-то из сотрудников;
- остальным повышаем з/п на 10%;
- а айтишникам з/п не повышаем (и так большая).

Вопрос — как изменится фонд оплаты труда в этом случае?

Здесь не получится использовать `filter`, потому что он совсем отсеивает айтишников, а нам нужно учесть их зарплаты в ФОТ, но без 10% повышения. Придется воспользоваться вложенной конструкцией `case`:

```
select
  ...
  sum(
    case when department = 'it' then salary else
salary*1.1 end
  ) over w as "+10% кроме ИТ"
from employees
window w as (...)
order by id;
```

Выражение `case` внутри `sum()` выдает обычную зарплату для айтишников и +10% для всех остальных — это именно то, что просил директор.

Запрос целиком и результат:

```
select
  name, department, salary,
  sum(salary) over () as "база",
  sum(salary) over w as "+0%",
  sum(salary*1.1) over w as "+10%",
  sum(
    case when department = 'it' then salary else
salary*1.1 end
  ) over w as "+10% кроме ИТ"
```

```

from employees
window w as (
    rows between unbounded preceding and unbounded
following
    exclude current row
)
order by id;

```

name	depar	salary	база	+0%	+10%	+10% кр/ИТ
Дарья	hr	70	942	872	959.2	909.0
Борис	hr	78	942	864	950.4	900.2
Елена	it	84	942	858	943.8	902.0
Ксения	it	90	942	852	937.2	896.0
Леонид	it	104	942	838	921.8	882.0
Марина	it	104	942	838	921.8	882.0
Иван	it	120	942	822	904.2	866.0
Вероника	sales	96	942	846	930.6	880.4
Григорий	sales	96	942	846	930.6	880.4
Анна	sales	100	942	842	926.2	876.0

В общем случае конструкция case выглядит так:

```

case
    when условие1 then результат1
    when условие2 then результат2
    ...
    when условиеN then результатN
    else результат_иначе
end

```

case — универсальный и более гибкий способ фильтрации записей фрейма, чем filter. К тому же его поддерживают все СУБД (в отличие от filter). Так что можно использовать filter в PostgreSQL и SQLite для простых ситуаций, а для более сложных или в других СУБД — case.

🗂 FILTER → CASE

Есть запрос, который считает зарплатный фонд города без учета ИТ-отдела:

```
select
  name, city,
  sum(salary) over w as base,
  sum(salary) filter(where department <> 'it')
over w as no_it
from employees
window w as (partition by city)
order by city, id;
```

name	city	base	no_it
Ксения	Москва	510	196
Марина	Москва	510	196
Иван	Москва	510	196
Вероника	Москва	510	196
Анна	Москва	510	196
Дарья	Самара	432	244
Борис	Самара	432	244
Елена	Самара	432	244
Леонид	Самара	432	244
Григорий	Самара	432	244

Перепишите запрос так, чтобы он использовал case вместо filter.

Песочница: <https://antonz.ru/to#empdb>

☑ FILTER → CASE

Используем `case`, чтобы выполнить условие задачи:

Если сотрудник работает не в ИТ-отделе — берем его зарплату, иначе 0.

Используем `sum()` по таким «условным» зарплатам, чтобы посчитать искомый зарплатный фонд.

```
select
  name, city,
  sum(salary) over w as base,
  sum(
    case when department <> 'it' then salary
  else 0 end
  ) over w as no_it
from employees
window w as (partition by city)
order by city, id;
```

name	city	base	no_it
Ксения	Москва	510	196
Марина	Москва	510	196
Иван	Москва	510	196
Вероника	Москва	510	196
Анна	Москва	510	196
Дарья	Самара	432	244
Борис	Самара	432	244
Елена	Самара	432	244
Леонид	Самара	432	244
Григорий	Самара	432	244

✂ Меньше ИТ, больше HR

Есть запрос, который считает зарплатный фонд города:

```
select
    name, city,
    sum(salary) over w as base
from employees
window w as (partition by city)
order by city, id;
```

Добавьте столбец `alt` с зарплатным фондом, рассчитанным по следующим правилам:

- для сотрудников отдела `hr` умножаем зарплату на 2;
- для сотрудников отдела `it` делим зарплату на 2;
- для сотрудников отдела `sales` не меняем зарплату.

name	city	base	alt
Ксения	Москва	510	353
Марина	Москва	510	353
Иван	Москва	510	353
Вероника	Москва	510	353
Анна	Москва	510	353
Дарья	Самара	432	486
Борис	Самара	432	486
Елена	Самара	432	486
Леонид	Самара	432	486
Григорий	Самара	432	486

Напишите запрос, который это сделает.

Песочница: <https://antonz.ru/to#empdb>

☑ Меньше ИТ, больше HR

Используем `case`, чтобы выполнить условие задачи. `sum()` по этому `case` считает искомый зарплатный фонд.

```
select
  name, city,
  sum(salary) over w as base,
  sum(
    case
      when department = 'hr' then salary*2
      when department = 'it' then salary/2
      else salary
    end
  ) over w as alt
from employees
window w as (partition by city)
order by city, id;
```

name	city	base	alt
Ксения	Москва	510	353
Марина	Москва	510	353
Иван	Москва	510	353
Вероника	Москва	510	353
Анна	Москва	510	353
Дарья	Самара	432	486
Борис	Самара	432	486
Елена	Самара	432	486
Леонид	Самара	432	486
Григорий	Самара	432	486

Так держать

Теперь вы знаете все о фреймах! Давайте подведем итоги.

Резюме

В общем случае определение фрейма выглядит так:

```
{ rows | groups | range }  
between frame_start and frame_end  
[ exclude exclusion ]
```

Фрейм по умолчанию:

```
range between unbounded preceding and current row  
exclude no others
```

Только у некоторых функций фрейм настраивается:

- функции смещения `first_value()`, `last_value()`, `nth_value()`;
- функции агрегации `count()`, `avg()`, `sum()`, ...

У остальных функций фрейм всегда равен секции.

Тип фрейма

Есть три типа фреймов:

- Строковые (`rows`) оперируют отдельными записями.
- Групповые (`groups`) оперируют группами записей с одинаковыми значениями столбцов из `order by`;
- Диапазонные (`range`) оперируют группами записей, у которых значение столбца из `order by` попадает в указанный диапазон.

Границы фрейма

unbounded preceding
N preceding
current row
N following
unbounded following

unbounded preceding и unbounded following всегда означают границы секции.

current row для строковых фреймов означает текущую запись, а для групповых и диапазонных — текущую запись и все равные ей (по значениям из order by).

N preceding и N following означают:

- для строковых фреймов — количество записей до/после текущей;
- для групповых фреймов — количество групп до/после текущей;
- для диапазонных фреймов — диапазон значений относительно текущей записи.

Исключения

exclude исключает определенные записи из фрейма:

- no others. Ничего не исключать. Вариант по умолчанию.
- current row. Исключить текущую запись.
- group. Исключить текущую запись и все равные ей.
- ties. Оставить текущую запись, но исключить равные ей.

Фильтрация

Чтобы отфильтровать фрейм, используют выражение filter или case:

```
func(column) filter (where condition) over window_  
name  
func(  
    case when condition then expression else other  
end  
    ) over window_name
```

Так держать

Отличная работа! Вы разобрались в нюансах оконных рамок, и теперь сможете проверять навороченные аналитические сценарии прямо из SQL.

Часть 3

Практика

Финансы

Основные функции часто используют в продуктовой и финансовой аналитике. Поэтому в этой главе решаем задачи, связанные с продажами и тарифами.

Предположим, у вас есть облачный сервис. Вы продаете подписки на тарифные планы silver, gold и platinum. Таблица sales содержит агрегированные данные о продажах по каждому тарифу за месяц (за 2019 и 2020 годы):

year	month	quarter	plan	price	quantity	revenue
2019	1	1	silver	60	200	12000
2019	1	1	gold	240	50	12000
2019	1	1	platinum	600	10	6000
2019	2	1	silver	60	660	39600
2019	2	1	gold	240	60	14400
2019	2	1	platinum	600	15	9000
2019	3	1	silver	60	400	24000
2019	3	1	gold	240	96	23040
2019	3	1	platinum	600	25	15000
2019	4	2	silver	60	300	18000
...	...	...	...	...	...	...

- year, month, quarter — год, месяц и квартал продажи;
- plan — тарифный план;
- price — стоимость одной подписки;

- `quantity` — количество проданных подписок;
- `revenue` — стоимость, умноженная на количество.

Песочница: <https://antonz.ru/to#w4ldb>

Скачать: <https://antonz.ru/to#w4lsql>

Прежде чем приступать к задачам, разберем несколько приемов. Они пригодятся как здесь, так и в реальной жизни.

Сначала агрегация, затем окна

Чтобы составить отчет, данные часто приходится агрегировать. При этом на каком-то этапе должны отработать оконные функции. Как совместить агрегацию с окнами? Давайте разбираться.

Допустим, мы хотим посмотреть, как выручка за каждый год соотносится с выручкой за два года.

Сначала считаем агрегаты по годам:

```
select
  year, sum(revenue) as revenue
from sales
group by year;
```

year	revenue
2019	722460
2020	1244940

Теперь с помощью оконных функций считаем общую выручку на агрегированных результатах:

```
with data as (  
    select  
        year, sum(revenue) as revenue  
    from sales  
    group by year  
)  
select  
    year, revenue,  
    sum(revenue) over () as total  
from data  
order by year;
```

year	revenue	total
2019	722460	1967400
2020	1244940	1967400

```
select  
    year,  
    sum(revenue) as revenue,  
    sum(sum(revenue)) over () as total  
from sales  
group by year  
order by year;
```

year	revenue	total
2019	722460	1967400
2020	1244940	1967400

Принцип тот же самый. Сначала агрегируем данные, затем применяем «окошки» на результате.

Сначала окна, затем отсев

Допустим, мы хотим посмотреть, как соотносится месячная выручка по тарифам silver и gold.

Для начала, вытащим «серебряную» и «золотую» выручки в отдельные столбцы через case:

```
select
    month,
    (case when plan = 'silver' then revenue end)
as silver,
    (case when plan = 'gold' then revenue end) as
gold
from sales
where year = 2020 and plan in ('gold', 'silver')
order by month, plan;
```

month	silver	gold
1		14400
1	27000	
2		29040
2	61200	
3		31200
3	42000	
...	...	...

Все хорошо, только нужные нам значения оказались в разных строчках: сначала всегда идет gold, затем silver. Подтянем показатель gold в строчку к silver с помощью функции `lag()`:

```
select
    month,
    (case when plan = 'silver' then revenue end) silver,
```

```

lag(case when plan = 'gold' then revenue end)
over w as gold
from sales
where year = 2020 and plan in ('gold', 'silver')
window w as (
  partition by month
  order by plan
)
order by month, plan;

```

month	silver	gold
1		14400
1	27000	
2		29040
2	61200	
3		31200
3	42000	
...	...	...

Стало еще лучше. Теперь бы только убрать пустые строки из ответа.

Для этого поместим промежуточный запрос в табличное выражение и отфильтруем его:

```

with data as (
  select
    month,
    (case when plan = 'silver' then revenue end)
silver,
    lag(case when plan = 'gold' then revenue
end) over w as gold
  from sales
  where year = 2020 and plan in ('gold', 'silver')
)

```

```
    window w as (  
      partition by month  
      order by plan  
    )  
  )  
  
  select month, silver, gold  
  from data  
  where silver is not null;
```

Тем самым убрали строки, где в silver (а заодно gold) находятся значения null:

month	silver	gold
1	27000	14400
2	61200	29040
3	42000	31200
4	42000	43200
5	39000	34560
6	52800	44880
7	46800	40320
8	33000	28800
9	54000	28800
10	57000	36000
11	62700	51840
12	66000	28800

Вот теперь хорошо!

Агрегация и null

Рассмотрим ту же задачу: узнать, как соотносится месячная выручка по тарифам silver и gold.

Вернемся к самому первому запросу, который вытаскивает «серебряную» и «золотую» выручки в отдельные столбцы через case:

```
select
    month,
    (case when plan = 'silver' then revenue end)
as silver,
    (case when plan = 'gold' then revenue end) as
gold
from sales
where year = 2020 and plan in ('gold', 'silver')
order by month, plan;
```

month	silver	gold
1		14400
1	27000	
2		29040
2	61200	
3		31200
3	42000	
...	...	...

На предыдущем шаге мы использовали окна, чтобы подтянуть показатель gold в строку к silver. Но можно обойтись и без них.

Идея следующая:

- поместим промежуточный запрос в табличное выражение;
- сгруппируем результаты по месяцам;
- для каждого месяца возьмем максимальное значение silver и gold.

```
with data as (  
  select  
    month,  
    (case when plan = 'silver' then revenue end) as  
silver,  
    (case when plan = 'gold' then revenue end) as gold  
  from sales  
  where year = 2020 and plan in ('gold', 'silver')  
)  
  
select  
  month,  
  max(silver) as silver,  
  max(gold) as gold  
from data  
group by month  
order by month;
```

Поскольку `max()` всегда выбирает не-null значение, он схлопывает две строчки по каждому месяцу в одну:

month	silver	gold
1	27000	14400
2	61200	29040
3	42000	31200
4	42000	43200
5	39000	34560
6	52800	44880
7	46800	40320
8	33000	28800
9	54000	28800
10	57000	36000
11	62700	51840
12	66000	28800

Точно такой же результат, как на предыдущем шаге.

В простых случаях можно совместить оба запроса в один:

```
select
    month,
    max(case when plan = 'silver' then revenue
end) as silver,
    max(case when plan = 'gold' then revenue end)
as gold
from sales
where year = 2020 and plan in ('gold', 'silver')
group by month
order by month;
```

А теперь — задачки! Все они используют ту же самую таблицу sales.

Выручка по тарифу gold

Есть таблица продаж sales. Посчитайте выручку для тарифа gold по месяцам 2020 года.

year	month	revenue	prev	perc
2020	1	14400		
2020	2	29040	14400	202
2020	3	31200	29040	107
2020	4	43200	31200	138
2020	5	34560	43200	80
2020	6	44880	34560	130
2020	7	40320	44880	90
2020	8	28800	40320	71
2020	9	28800	28800	100
2020	10	36000	28800	125
2020	11	51840	36000	144
2020	12	28800	51840	56

Для каждого месяца дополнительно укажите:

- выручку за предыдущий месяц (`prev`);
- процент, который составляет выручка текущего месяца от `prev` (`perc`).
- Процент округлите до целого.

Сортировка результата: `month`

Песочница: <https://antonz.ru/to#w41db>

☒ Выручка по тарифу gold

Используем `lag()`, чтобы получить выручку за предыдущий месяц.

```
select
    year, month, revenue,
    lag(revenue) over w as prev,
    round(revenue * 100.0 / lag(revenue) over w) as perc
from sales
where year = 2020 and plan = 'gold'
window w as (order by month)
order by month;
```

year	month	revenue	prev	perc
2020	1	14400		
2020	2	29040	14400	202
2020	3	31200	29040	107
2020	4	43200	31200	138
2020	5	34560	43200	80
2020	6	44880	34560	130
2020	7	40320	44880	90
2020	8	28800	40320	71
2020	9	28800	28800	100
2020	10	36000	28800	125
2020	11	51840	36000	144
2020	12	28800	51840	56

✍ Выручка по тарифам за 1 квартал

Есть таблица продаж `sales`. Посчитайте выручку нарастающим итогом по каждому тарифному плану за первые три месяца 2020 года.

plan	year	month	revenue	total
gold	2020	1	14400	14400
gold	2020	2	29040	43440
gold	2020	3	31200	74640
platinum	2020	1	7200	7200
platinum	2020	2	13200	20400
platinum	2020	3	16800	37200
silver	2020	1	27000	27000
silver	2020	2	61200	88200
silver	2020	3	42000	130200

Сортировка результата: `plan, month`

Песочница: <https://antonz.ru/to#w41db>

☑ Выручка по тарифам за 1 квартал

Окно:

- секции по тарифам;
- сортировка по месяцам;
- `sum()` для выручки нарастающим итогом.

plan	year	month	revenue	total
gold	2020	1	14400	14400
gold	2020	2	29040	43440
gold	2020	3	31200	74640
platinum	2020	1	7200	7200
platinum	2020	2	13200	20400
platinum	2020	3	16800	37200
silver	2020	1	27000	27000
silver	2020	2	61200	88200
silver	2020	3	42000	130200

```
select
  plan, year, month, revenue,
  sum(revenue) over w as total
from sales
where
  year = 2020 and quarter = 1
window w as (
  partition by plan
  order by month
)
order by plan, month;
```

🗒 Скользящее среднее по тарифу platinum

Есть таблица продаж `sales`. Посчитайте скользящую среднюю выручку за 3 месяца для тарифа `platinum` в 2020 году.

Округлите среднюю выручку до целого.

year	month	revenue	avg3m
2020	1	7200	10200
2020	2	13200	12400
2020	3	16800	18400
2020	4	25200	22000
2020	5	24000	27200
2020	6	32400	28400
2020	7	28800	24800
2020	8	13200	18600
2020	9	13800	15000
2020	10	18000	22600
2020	11	36000	25000
2020	12	21000	28500

Сортировка результата: `month`

Песочница: <https://antonz.ru/to#w41db>

☑ Скользящее среднее по тарифу platinum

Окно:

- сортировка по месяцам;
- строковый фрейм на предыдущую, текущую и следующую записи;
- `avg()` для расчета скользящего среднего.

```
select
    year, month, revenue,
    round(avg(revenue) over w) as "avg3m"
from sales
where
    year = 2020 and plan = 'platinum'
window w as (
    order by month
    rows between 1 preceding and 1 following
)
order by month;
```

year	month	revenue	avg3m
2020	1	7200	10200
2020	2	13200	12400
2020	3	16800	18400
2020	4	25200	22000
2020	5	24000	27200
2020	6	32400	28400
2020	7	28800	24800
2020	8	13200	18600
2020	9	13800	15000
2020	10	18000	22600
2020	11	36000	25000
2020	12	21000	28500

Сравнение с декабрем

Есть таблица продаж `sales`. Посчитайте выручку по месяцам для тарифа `silver`.

Для каждого месяца дополнительно укажите:

- выручку за декабрь этого же года (`december`);
- процент, который составляет выручка текущего месяца от `december` (`perc`).

Процент округлите до целого.

year	month	revenue	december	perc
2019	1	12000	26400	45
2019	2	39600	26400	150
2019	3	24000	26400	91
...	...	...	...	...
2019	10	25200	26400	95
2019	11	29700	26400	113
2019	12	26400	26400	100
2020	1	27000	66000	41
2020	2	61200	66000	93
2020	3	42000	66000	64
...	...	...	...	...
2020	10	57000	66000	86
2020	11	62700	66000	95
2020	12	66000	66000	100

Сортировка результата: `year, month`

Песочница: <https://antonz.ru/to#w41db>

☒ Сравнение с декабрем

- Окно;
- секции по годам;
- сортировка по месяцам;

- фрейм равен секции (= год);
- `last_value()`, чтобы получить выручку за декабрь.

```

select
    year, month, revenue,
    last_value(revenue) over w as december,
    round(revenue * 100.0 / last_value(revenue)
over w) as perc
from sales
where plan = 'silver'
window w as (
    partition by year
    order by month
    rows between unbounded preceding and unbounded
following
)
order by year, month;

```

year	month	revenue	december	perc
2019	1	12000	26400	45
2019	2	39600	26400	150
2019	3	24000	26400	91
...	...	...	...	...
2019	10	25200	26400	95
2019	11	29700	26400	113
2019	12	26400	26400	100
2020	1	27000	66000	41
2020	2	61200	66000	93
2020	3	42000	66000	64
...	...	...	...	...
2020	10	57000	66000	86
2020	11	62700	66000	95
2020	12	66000	66000	100

Вклад тарифов

Есть таблица продаж `sales`. Посчитайте, какой вклад (в процентах) внес каждый из тарифов в общую выручку за год.

Процент округлите до целого.

year	plan	revenue	total	perc
2019	gold	252960	722460	35
2019	platinum	168000	722460	23
2019	silver	301500	722460	42
2020	gold	411840	1244940	33
2020	platinum	249600	1244940	20
2020	silver	583500	1244940	47

Сортировка результата: `year, plan`

Песочница: <https://antonz.ru/to#w41db>

☒ Вклад тарифов

Запрос:

- агрегируем выручку по тарифам внутри года;
- затем берем окно с секцией по годам;
- и считаем выручку за год через `sum()` по секции.

```
with data as (  
    select  
        year, plan,  
        sum(revenue) as revenue  
    from sales  
    group by year, plan  
)  
select
```

```

    year, plan, revenue,
    sum(revenue) over w as total,
    round(revenue * 100.0 / sum(revenue) over w)
as perc
from data
window w as (partition by year)
order by year, plan;

```

year	plan	revenue	total	perc
2019	gold	252960	722460	35
2019	platinum	168000	722460	23
2019	silver	301500	722460	42
2020	gold	411840	1244940	33
2020	platinum	249600	1244940	20
2020	silver	583500	1244940	47

Высокая, средняя и низкая выручка

Есть таблица продаж `sales`. Разбейте месяцы 2020 года на три группы по выручке:

year	month	revenue	tile
2020	11	150540	1
2020	6	130080	1
2020	7	115920	1
2020	12	115800	1
2020	10	111000	2
2020	4	110400	2
2020	2	103440	2
2020	5	97560	2
2020	9	96600	3
2020	3	90000	3
2020	8	75000	3
2020	1	48600	3

- tile = 1 — высокая,
- tile = 2 — средняя,
- tile = 3 — низкая.

Сортировка результата: revenue desc

Песочница: <https://antonz.ru/to#w4ldb>

☑ Высокая, средняя и низкая выручка

Запрос:

- агрегируем выручку по месяцам;
- разбиваем на группы через `ntile()`.

```
with data as (  
    select  
        year, month,  
        sum(revenue) as revenue  
    from sales  
    where year = 2020  
    group by year, month  
)  
select  
    year, month, revenue,  
    ntile(3) over w as tile  
from data  
window w as (order by revenue desc)  
order by revenue desc, month;
```

year	month	revenue	tile
2020	11	150540	1
2020	6	130080	1
2020	7	115920	1
...	...	...	...
2020	3	90000	3
2020	8	75000	3
2020	1	48600	3

2020 vs 2019

Есть таблица продаж `sales`. Посчитайте выручку по кварталам 2020 года.

Для каждого квартала дополнительно укажите:

- выручку за аналогичный квартал 2019 года (`prev`);
- процент, который составляет выручка текущего квартала от `prev` (`perc`).

Процент округлите до целого.

year	quarter	revenue	prev	perc
2020	1	242040	155040	156
2020	2	338040	162600	208
2020	3	287520	204120	141
2020	4	377340	200700	188

Подсказка. Подумайте, какая оконная функция позволит заглянуть в прошлое на фиксированное количество кварталов. Она поможет рассчитать `prev`. Чтобы убрать из результата лишние строки, используйте одну из техник, описанных в начале главы.

Сортировка результата: `quarter`

Песочница: <https://antonz.ru/to#w41db>

☑ 2020 vs 2019

Запрос:

- агрегируем выручку по кварталам;
- используем `lag()`, чтобы получить выручку аналогичного квартала в прошлом году;
- выбираем только кварталы 2020 года.

```
with data as (  
    select  
        year, quarter,  
        sum(revenue) as revenue,  
        lag(sum(revenue), 4) over w as prev,  
        round(  
            sum(revenue) * 100.0 / lag(sum(revenue),  
4) over ()  
        ) as perc  
    from sales  
    group by year, quarter  
    window w as (order by year, quarter)  
)  
select  
    year, quarter, revenue,  
    prev, perc  
from data  
where year = 2020  
order by quarter;
```

year	quarter	revenue	prev	perc
2020	1	242040	155040	156
2020	2	338040	162600	208
2020	3	287520	204120	141
2020	4	377340	200700	188

✎ Рейтинг месяцев по продажам

Есть таблица продаж `sales`. Составьте рейтинг месяцев 2020 года с точки зрения количества продаж (`quantity`) по каждому из тарифов. Чем больше подписок тарифа `P` было продано в месяц `M`, тем выше место `M` в рейтинге по тарифу `P`:

year	month	silver	gold	platinum
2020	1	12	12	12
2020	2	3	8	10
2020	3	8	7	8
2020	4	8	3	4
2020	5	10	6	5
2020	6	6	2	2
2020	7	7	4	3
2020	8	11	9	10
2020	9	5	9	9
2020	10	4	5	7
2020	11	2	1	1
2020	12	1	9	6

Например, в декабре было продано больше подписок `silver`, чем в любой другой месяц (1-е место в столбце `silver`). Для тарифа `gold` декабрь на 9-м месте, для `platinum` — на 6-м. Январь же оказался самым слабым месяцем для всех трех тарифов.

Сортировка результата: `month`

Песочница: <https://antonz.ru/to#w41db>

☑ Рейтинг месяцев по продажам

Запрос:

- секции по тарифам, чтобы считать рейтинг месяцев по тарифам;

- сортировка по убыванию количества продаж, чтобы месяцы с большим количеством продаж занимали более высокие места;
- `rank()` для расчета рейтинга;
- через `sum()` + `case` выносим рейтинг по каждому тарифу в отдельный столбец.

```
with data as (  
    select  
        year, month, plan,  
        rank() over w as qrank  
    from sales  
    where year = 2020  
    window w as (  
        partition by plan  
        order by revenue desc  
    )  
)  
select  
    year, month,  
    max(case when plan='silver' then qrank else 0  
end) as silver,  
    max(case when plan='gold' then qrank else 0  
end) as gold,  
    max(case when plan='platinum' then qrank else  
0 end) as platinum  
from data  
group by year, month  
order by month;
```

Так держать

Отличная работа! Теперь вам не придется расчехлять эксель, чтобы построить отчет по продажам.

Кластеризация

При анализе данных бывает удобно разбить датасет на группы близких значений. Частный случай такой задачи — поиск *островов* в данных (islands).

Остров — это последовательность идущих подряд значений. Например, есть числа:

Они идут с промежутками, и образуют три острова:

- (51, 52, 53)
- (55, 56)
- (59)

num
51
52
53
55
56
59

Хотелось бы автоматически найти все острова и количество значений в каждом. Это несложная задача для императивных языков программирования вроде Python, но нетривиальная для SQL. Однако, решаемая — с помощью оконных функций.

Кластеризация и поиск островов — сложная тема. Прорабатывайте каждый шаг и не переходите к следующему, пока не поймете предыдущий. Не торопитесь.

Оконные функции намного чаще используют для аналитических запросов, чем для кластеризации. Возможно, что в жизни вы вообще не столкнетесь с такими задачами. Но их иногда любят спрашивать на собеседованиях, поэтому я добавил в книгу эту главу.

Идентифицируем острова

Для начала, пронумеруем записи по возрастанию значений:

```
select
  num,
  row_number() over w as rownum
```

```

from numbers
window w as (order by num);

```

Сам по себе номер строки не слишком интересен, но у него есть полезное свойство — пока значения `num` идут подряд, разность `num-rownum` постоянна:

num	rownum
51	1
52	2
53	3
55	4
56	5
59	6

```

select
    num,
    row_number() over w as
rownum,
    num - row_number() over w as group_id
from numbers
window w as (order by num);

```

num	rownum	group_id
51	1	50
52	2	50
53	3	50
55	4	51
56	5	51
59	6	53

Раз `num-rownum` не меняется для идущих подряд значений `num` — эту разность можно считать идентификатором острова (`group_id`). Осталось только сгруппировать по нему, чтобы получить границы островов и количество записей в каждом:

```

with ngroups as (
    ...
)
select
    min(num) as n_start,
    max(num) as n_end,
    count(*) as n_count
from ngroups
group by group_id;

```

n_start	n_end	n_count
51	53	3
55	56	2
59	59	1

Получились те же самые группы, что мы раньше идентифицировали вручную:

- 51–53 (3 значения)
- 55–56 (2 значения)
- 59 (1 значение)

Числа с повторами

Допустим, есть числа:

Почти такие же, как на предыдущем шаге, только значение 52 повторяется. Как думаете, сработает в данном случае наш подход к поиску островов с помощью `row_number()`?

```

with ngroups as (
    select
        num, num - row_number() over w as
group_id

```

num
51
52
53
55
56
59

```
    from numbers
    window w as (order by num)
)
select
    min(num) as n_start,
    max(num) as n_end,
    count(*) as n_count
from ngroups
group by group_id;
```

Выберите один вариант:

- Запрос правильно находит острова.
- Запрос неправильно находит острова.
- Запрос ничего не находит.
- Запрос падает с ошибкой.

☒ Числа с повторами

Выполним наш запрос:

```
with ngroups as (
    select
        num,
        num - row_number() over w as group_id
    from numbers
    window w as (order by num)
)
select
    min(num) as n_start,
    max(num) as n_end,
    count(*) as n_count
from ngroups
group by group_id;
```

Вот что он вернул:

n_start	n_end	n_count
52	53	2
51	56	4
59	59	1

Из трех островов корректно нашелся только один (59). Со всем не то, что мы ожидали.

Поэтому правильный ответ — «Запрос неправильно находит острова».

✂ Острова с дублями

Есть те же самые числа:

Какую оконную функцию вместо `row_number()` нужно использовать, чтобы запрос корректно идентифицировал острова?

```
with ngroups as (
    select
        num,
        num - row_number() over w as
group_id
    from numbers
    window w as (order by num)
)
select
    min(num) as n_start,
    max(num) as n_end,
    count(*) as n_count
from ngroups
group by group_id;
```

num
51
52
53
55
56
59

Выберите один вариант: `dense_rank` / `ntile` / `lag` / `nth_value`

☑ Острова с дублями

Сходу непонятно, чем вообще плоха `row_number()`. Посмотрим, что она выдает:

```
select
  num,
  row_number() over w as rownum,
  num - row_number() over w as group_id
from numbers
window w as (order by num);
```

num	rownum	group_id
51	1	50
52	2	50
52	3	49
53	4	49
55	5	50
56	6	50
59	7	52

Вот оно что. В `num` значения повторяются, а `rownum` строго растёт. Из-за этого сломалось базовое свойство, на котором построен расчёт — постоянство разности `num-rownum` для идущих подряд `num`.

Чтобы исправить проблему, нужна функция, которая выдавала бы одинаковые значения `rownum` для одних и тех же значений `num`. Существует ли такая? Да, целых две — `rank()` и `dense_rank()`:

```
select
  num,
  dense_rank() over w as rownum,
  num - dense_rank() over w as group_id
from numbers
window w as (order by num);
```

num	rownum	group_id
51	1	50
52	2	50
53	3	50
55	4	51
56	5	51
59	6	53

group_id исправился, так что основной запрос должен тоже:

```
with ngroups as (...)
select
  min(num) as n_start,
  max(num) as n_end,
  count(*) as n_count
from ngroups
group by group_id;
```

n_start	n_end	n_count
51	53	4
55	56	2
59	59	1

Идеально! Дальше будем использовать именно `dense_rank()`.

Острова на датах

Представьте себе обучающую платформу, на которой люди выполняют задания и получают за это баллы. В базе данных

такой платформы может быть таблица `activity`, которая хранит баллы, набранные пользователями в конкретные дни:

adate	user_id	points
2020-02-01	51	1
2020-02-02	11	1
2020-02-02	51	3
2020-02-05	21	6
...	...	...
2020-02-25	51	1

Песочница: <https://antonz.ru/to#w42db>

Скачать: <https://antonz.ru/to#w42sql>

Допустим, нас интересуют острова по датам для пользователя 51 — то есть периоды времени, когда он занимался каждый день без перерыва.

Таблица игрушечная, поэтому для начала посчитаем вручную:

```
select adate, points
from activity
where user_id = 51
order by adate;
```

Здесь четыре острова:

- 01.02 — 05.02 (5 дней),
- 08.02 — 10.02 (3 дня),
- 20.02 — 22.02 (3 дня),
- 25.02 (1 день).

adate	points
2020-02-01	1
2020-02-02	3
2020-02-03	2
2020-02-04	6
2020-02-05	3
2020-02-08	1
2020-02-09	1
2020-02-10	1
2020-02-20	9
2020-02-21	15
2020-02-22	7
2020-02-25	1

Теперь попробуем найти острова с помощью SQL. Вспомним прием, который помог нам найти острова на числах:

```
select
    num,
    num - dense_rank() over w as group_id
from numbers
window w as (order by num);
```

Чтобы применить тот же прием к столбцу `adate`, потребуется преобразовать даты в целые числа так, чтобы соседние даты отличались на 1.

Поставим каждой дате в соответствие количество дней от некоторой общей точки отсчета — это и будет интересующим нас числом.

num	group_id
51	50
52	50
52	50
53	50
55	51
56	51
59	53

Как преобразовать дату в количество дней

К сожалению, каждая СУБД реализует работу с датами по-своему, поэтому универсального решения не существует.

В MySQL, например, есть функция `to_days()`:

```
select
    adate,
    to_days(adate) as day_id
from activity
where user_id = 51;
```

В PostgreSQL придется использовать `extract()`:

```

select
    adate,
    extract(epoch from adate)/86400 as day_id
from activity
where user_id = 51;

```

А в SQLite — unixepoch():

```

select
    adate,
    unixepoch(adate)/86400 as day_id
from activity
where user_id = 51;

```

Я буду использовать вариант SQLite.

* * *

Преобразовав дату в число, мы можем использовать dense_rank():

```

select
    adate,
    unixepoch(adate)/86400 - dense_rank() over w
as group_id
from activity
where user_id = 51
window w as (order by adate);

```

Осталось только сгруппировать по group_id:

```

with agroups as (
    select
        adate,
        unixepoch(adate)/86400 -
        dense_rank() over w as group_id
    from activity
    where user_id = 51

```

adate	group_id
2020-02-01	18292
2020-02-02	18292
2020-02-03	18292
2020-02-04	18292
2020-02-05	18292
2020-02-08	18294
2020-02-09	18294
2020-02-10	18294
2020-02-20	18303
2020-02-21	18303
2020-02-22	18303
2020-02-25	18305

```

    window w as (order by adate)
)
select
    min(adate) as day_start,
    max(adate) as day_end,
    count(*) as day_count
from agroups
group by group_id;

```

day_start	day_end	day_count
2020-02-01	2020-02-05	5
2020-02-08	2020-02-10	3
2020-02-20	2020-02-22	3
2020-02-25	2020-02-25	1

Вот и искомые острова!

Серия без перерывов

Есть таблица `activity`, которая хранит баллы, набранные пользователями в конкретные дни:

adate	user_id	points
2020-02-01	51	1
2020-02-02	11	1
2020-02-02	51	3
2020-02-05	21	6
...	...	...
2020-02-25	51	1

Посчитайте острова по дате для каждого пользователя — то есть периоды, в которые пользователь набирал хотя бы один балл каждый день без перерыва. Серия из одного дня тоже считается.

Должно получиться так:

user_id	day_start	day_end	day_count
11	2020-02-02	2020-02-11	10
11	2020-02-15	2020-02-17	3
21	2020-02-05	2020-02-05	1
...	...	...	...
21	2020-02-23	2020-02-24	2
31	2020-02-07	2020-02-07	1
31	2020-02-11	2020-02-13	3
...	...	...	...

Сортировка результата: `user_id, day_start`

Песочница: <https://antonz.ru/to#w42db>

☑ Серия без перерывов

Ранжируем даты и вычисляем идентификаторы островов:

```
select
    user_id,
    adate,
    unixepoch(adate)/86400 - dense_rank() over w
as group_id
from activity
window w as (
    partition by user_id
    order by adate
);
```

Серии у каждого пользователя независимые, поэтому окно секционировано по `user_id`.

Затем группируем и получаем острова:

```
with agroups as (  
    ...  
)  
select  
    user_id,  
    min(adate) as day_start,  
    max(adate) as day_end,  
    count(*) as day_count  
from agroups  
group by user_id, group_id;
```

Готово!

Кластеры значений

Мы начали главу с того, что при анализе данных бывает удобно разбить датасет на группы близких значений. Но «близкие» не всегда означает «идущие подряд». Иногда мы готовы включить в одну группу значения с небольшими промежутками.

Например, есть числа:

Договоримся, что соседние значения входят в одну и ту же группу, если отличаются на 1 или 2. Тогда получится три группы (или кластера):

- (51, 52, 53, 55, 56)
- (59, 61)
- (72)

Попробуем найти кластеры с помощью SQL. Сразу скажу, что подход с ранжированием строк

num
51
52
53
55
56
59
61
72

здесь не сработает, поскольку мы разрешили промежутки между соседними значениями. Поэтому поступим иначе.

Для начала, посчитаем расстояние между соседями с помощью функции `lag()`:

```
select
  num,
  num - lag(num) over w as n_lag
from numbers
window w as (order by num);
```

num	n_lag
51	
52	1
53	1
55	2
56	1
59	3
61	2
72	11

Теперь идентифицируем границы между кластерами по такому правилу:

Если расстояние между соседними значениями больше 2, то это разные кластеры.

```
— расстояния между соседями
with nlags as (
  ...
)
— границы кластеров
select
  num,
  n_lag,
  case when n_lag > 2 then 1 else 0 end as is_
boundary
from nlags;
```

num	n_lag	is_boundary
51		0
52	1	0
53	1	0
55	2	0
56	1	0
59	3	1
61	2	0
72	11	1

Границы уже явно видны. Теперь пронумеруем кластеры по порядку:

```
-- расстояния между соседями
with nlags as (
    ...
)
-- границы кластеров
select
    num,
    sum(case when n_lag > 2 then 1 else 0 end)
over w as group_id
from nlags
window w as (order by num);
```

Здесь идентификатор кластера считается как количество предшествующих ему границ (то есть порядковый номер кластера, считая с 0):

num	group_id
51	0
52	0
53	0
55	0
56	0
59	1
61	1
72	2

Наконец, агрегируем по идентификатору кластера:

```
-- расстояния между соседями
with nlags as (
  select
    num,
    num - lag(num) over w as n_lag
  from numbers
  window w as (order by num)
),
-- границы кластеров
ngroups as (
  select
    num,
    sum(case when n_lag > 2 then 1 else 0 end)
over w as group_id
  from nlags
  window w as (order by num)
)
-- сами кластеры
select
  min(num) as n_start,
  max(num) as n_end,
  count(*) as n_count
from ngroups
group by group_id;
```

n_start	n_end	n_count
51	56	5
59	61	2
72	72	1

То, что нужно!

Вот универсальный алгоритм поиска кластеров на SQL:

1. Посчитать расстояние L между соседними значениями через `lag()`.
2. Идентифицировать границы по условию `sum(case when  $L > N$  then 1 else 0 end)`, где N — максимально допустимое расстояние между соседними значениями кластера.
3. Агрегировать кластеры по идентификатору.

Острова можно было считать по этому же алгоритму. Но там мы «срезали путь» через ранжирование, потому что точно знали, что соседние значения отличаются ровно на 1.

Кластеры на датах

Вернемся к таблице `activity`, которая хранит баллы, набранные пользователями в конкретные дни:

adate	user_id	points
2020-02-01	51	1
2020-02-02	11	1
2020-02-02	51	3
2020-02-05	21	6
...	...	...
2020-02-25	51	1

Песочница: <https://antonz.ru/to#w42db>

Договоримся, что пропуск одного дня не прерывает серию. Как тогда будут выглядеть кластеры?

Рассмотрим пользователя 21:

```
select adate, points
from activity
where user_id = 21;
```

adate	points
2020-02-05	6
2020-02-07	2
2020-02-09	4
2020-02-11	6
2020-02-13	3
2020-02-15	5
2020-02-20	5
2020-02-23	3
2020-02-24	1

Вот его серии:

- 05.02 — 15.02 (6 дней)
- 20.02 (1 день)
- 23.02 — 24.02 (2 дня)

Посчитаем кластеры с помощью SQL. Вспомним универсальный алгоритм:

1. Посчитать расстояние L между соседними значениями через `lag()`.
2. Идентифицировать границы по условию `sum(case when  $L > N$  then 1 else 0 end)`, где N — максимально допустимое расстояние между соседними значениями кластера.
3. Агрегировать кластеры по идентификатору.

Шаг 1. Переводим даты в числа и считаем расстояние между соседями:

```
select
    adate,
    unixepoch(adate)/86400
    - lag(unixepoch(adate) /86400) over w as
day_lag
from activity
where user_id = 21
window w as (order by adate);
```

adate	day_lag
2020-02-05	
2020-02-07	2
2020-02-09	2
2020-02-11	2
2020-02-13	2
2020-02-15	2
2020-02-20	5
2020-02-23	3
2020-02-24	1

Шаг 2. Идентифицируем границы кластеров:

```
-- расстояния между соседями
with adays as (
    ...
)
-- границы кластеров
select
    adate, day_lag,
    sum(case when day_lag > 2 then 1 else 0 end)
over w as group_id
from adays
window w as (order by adate);
```

adate	day_lag	group_id
2020-02-05		0
2020-02-07	2	0
2020-02-09	2	0
2020-02-11	2	0
2020-02-13	2	0
2020-02-15	2	0
2020-02-20	5	1
2020-02-23	3	2
2020-02-24	1	2

Шаг 3. Агрегируем кластеры по идентификатору:

```
-- расстояния между соседями
with adays as (
  select
    adate,
    unixepoch(adate)/86400
    - lag(unixepoch(adate)/86400) over w as day_lag
  from activity
  where user_id = 21
  window w as (order by adate)
),
-- границы кластеров
agroups as (
  select
    adate, day_lag,
    sum(
      case when day_lag > 2 then 1 else 0 end
    ) over w as group_id
  from adays
  window w as (order by adate)
)
-- сами кластеры
select
  min(adate) as day_start,
  max(adate) as day_end,
  count(*) as day_count
from agroups
group by group_id;
```

day_start	day_end	day_count
2020-02-05	2020-02-15	6
2020-02-20	2020-02-20	1
2020-02-23	2020-02-24	2

Вот и искомые кластеры!

Серия с возрастанием очков

Есть таблица `activity`, которая хранит баллы, набранные пользователями в конкретные дни:

<code>adate</code>	<code>user_id</code>	<code>points</code>
2020-02-01	51	1
2020-02-02	11	1
2020-02-02	51	3
2020-02-05	21	6
...	...	...
2020-02-25	51	1

Серией в этой задаче будем считать последовательность дней, в которых количество набранных пользователем очков в каждый следующий день не меньше, чем в предыдущий. При этом между предыдущим и следующим днем может быть любой промежуток времени, это не прерывает серию. Серии из одного дня не учитываем.

Например, это все одна серия:

- 01.02 — 1 балл,
- 05.02 — 2 балла,
- 10.02 — 3 балла.

А это не считается серией, хотя дни идут без промежутков:

- 01.02 — 3 балла,
- 02.02 — 2 балла,
- 03.02 — 1 балл.

Напишите запрос, который считает серии для каждого пользователя.

Должно получиться так:

user_id	day_start	day_end	day_count	p_total
11	2020-02-02	2020-02-17	13	16
21	2020-02-07	2020-02-11	3	12
21	2020-02-13	2020-02-20	3	13
31	2020-02-11	2020-02-12	2	5
51	2020-02-01	2020-02-02	2	4
51	2020-02-03	2020-02-04	2	8
51	2020-02-08	2020-02-21	5	27

- `day_count` — количество дней в серии, в которых пользователь заработал баллы;
- `p_total` — сумма заработанных баллов за серию.

Сортировка результата: `user_id, day_start`

Песочница: <https://antonz.ru/to#w42db>

☑ Серия с возрастанием очков

Действуем по алгоритму расчета кластеров.

Шаг 1. Считаем расстояние между соседями:

```
select
  user_id, adate, points,
  points - lag(points) over w as p_diff
from activity
window w as (
  partition by user_id
  order by adate
)
```

Шаг 2. Идентифицируем границы кластеров:

```

with adays as (
    ...
),
select
    user_id, adate, points,
    sum(case when p_diff < 0 then 1 else 0 end)
over w as group_id
from adays
window w as (
    partition by user_id
    order by adate
);

```

Правило для границ:

Если расстояние между соседними значениями отрицательное, то это разные кластеры.

Шаг 3. Агрегируем кластеры по идентификатору:

```

with adays as (
    ...
),
agroups as (
    ...
)
select
    user_id,
    min(adate) as day_start,
    max(adate) as day_end,
    count(*) as day_count,
    sum(points) as p_total
from agroups
group by user_id, group_id
having count(*) > 1
order by user_id, day_start;

```

Используем `having count(*) > 1`, чтобы отсеять серии длиной в один день.

Вот и все!

Так держать

Отличная работа! Теперь вас не напугать требованием найти острова в данных.

Очистка данных

Если вы работаете с данными из надежного источника — например, внутренней ИТ-системы компании — по ним обычно можно сразу строить аналитические отчеты. Но если загрузили датасет из просторов интернета или получили от внешнего партнера — не факт, что там все хорошо. Такие данные приходится предварительно *чистить*. Оконные функции могут с этим помочь.

Очистка данных — не самое популярное применение оконных функций. Возможно, в жизни вы вообще не столкнетесь с такой задачей. Но приемы очистки данных могут пригодиться и в «обычных» отчетах, поэтому их полезно знать.

Мы будем работать с таблицей `weather` — это погода в Лондоне за 6 месяцев 2020 года:

wdate	wtemp	precip	pressure
2020-01-01	4.1	0.0	102980.0
2020-01-02	8.6	1.0	102160.0
2020-01-03	9.9	0.6	101980.0
2020-01-04	5.6	0.0	103400.0
2020-01-05	7.8	0.0	103380.0
2020-01-06	7.8	0.8	102290.0
2020-01-07	8.1	0.2	102220.0
2020-01-07	8.1	0.2	102220.0
2020-01-08	11.8	1.0	102070.0
2020-01-09	11.5	4.2	100780.0
...	...	...	...

Песочница: <https://antonz.ru/to#w43db>

Скачать: <https://antonz.ru/to#w43sql>

Столбцы:

- `wdate` — дата измерений,
- `wtemp` — средняя температура в °C за день,
- `precip` — уровень осадков в мм,
- `pressure` — атмосферное давление в Па.

Я скачал датасет из интернета и загрузил его в таблицу не глядя, так что в данных наверняка есть проблемы. Давайте разбираться.

Дубли

Каждая запись таблицы — измерения за конкретный день. Проверим, не дублируются ли эти дни:

```
select wdate, count(*)  
from weather  
group by wdate  
having count(*) > 1;
```

wdate	count(*)
2020-01-07	2
2020-02-13	2
2020-03-24	3

В данных дважды встречается 7 января и 13 февраля, а 24 марта — аж три раза.

Чтобы дубли не портили отчеты, придется их почистить. Для наглядности будем работать только с началом января (чтобы исправить весь набор данных, достаточно убрать условие `where`).

Пронумеруем дни с помощью `row_number()` и секции по дате, чтобы дубли получили номер `> 1`:

```
select
    wdate, wtemp,
    row_number() over w as rownum
from weather
where wdate between '2020-01-01' and '2020-01-08'
window w as (partition by wdate);
```

wdate	wtemp	rownum
2020-01-01	4.1	1
2020-01-02	8.6	1
2020-01-03	9.9	1
2020-01-04	5.6	1
2020-01-05	7.8	1
2020-01-06	7.8	1
2020-01-07	8.1	1
2020-01-07	8.1	2
2020-01-08	11.8	1

Теперь выберем записи с номером строки = 1, чтобы получить данные без дублей:

```
with src as (
    select
        wdate, wtemp, precip, pressure,
        row_number() over w as rownum
    from weather
    where wdate between '2020-01-01' and '2020-01-08'
    window w as (partition by wdate)
)
select
    wdate, wtemp, precip, pressure
from src
where rownum = 1;
```

wdate	wtemp	precip	pressure
2020-01-01	4.1	0.0	102980.0
2020-01-02	8.6	1.0	102160.0
2020-01-03	9.9	0.6	101980.0
2020-01-04	5.6	0.0	103400.0
2020-01-05	7.8	0.0	103380.0
2020-01-06	7.8	0.8	102290.0
2020-01-07	8.1	0.2	102220.0
2020-01-08	11.8	1.0	102070.0

Порядок!

Осадки в апреле

Напишите запрос, который выбирает апрельские данные без повторений осадков. Если в дни D1 и D2 выпало N осадков, то в результате должен попасть только один из этих дней (более поздний).

Должно получиться так:

wdate	wtemp	precip
2020-04-05	13.8	0.4
2020-04-06	14.5	0.2
2020-04-17	11.9	16.0
2020-04-18	11.3	1.0
2020-04-26	12.4	0.0
2020-04-27	12.8	3.0
2020-04-28	8.5	11.0
2020-04-29	10.7	1.6
2020-04-30	10.2	4.2

Сортировка результата: wdate

Песочница: <https://antonz.ru/to#w43db>

☑ Осадки в апреле

Секционировать дни по значению осадков и отсортировать по убыванию даты. Так самая поздняя дата получит номер 1, а более ранние дубли — номера > 1 :

```
select
  wdate, wtemp, precip,
  row_number() over w as rownum
from weather
where wdate between '2020-04-01' and '2020-04-30'
window w as (
  partition by precip
  order by wdate desc
)
```

Теперь выберем строки с номером 1, чтобы получить данные без дублей:

```
with src as (
  ...
)
select wdate, wtemp, precip
from src where rownum = 1
order by wdate;
```

Готово!

Пропуски

Бывает, что в данных заполнены не все значения. Например:

```
select wdate from weather where wtemp is null;
```

```
2020-02-05
2020-03-10
2020-03-24
2020-03-24
2020-05-06
2020-06-05
```

За полгода набралось 6 дней без температуры. Можно было бы вовсе удалить их, но мы применим другую технику: заполним пропуски предыдущим значением.

Для наглядности будем работать только с февралем:

```
select wdate, wtemp
from weather
where wdate between '2020-02-01' and '2020-02-08';
```

wdate	wtemp
2020-02-01	11.2
2020-02-02	9.9
2020-02-03	10.3
2020-02-04	6.7
2020-02-05	
2020-02-06	4.8
2020-02-07	5.0
2020-02-08	9.5

5 февраля должно получить температуру от предыдущего дня — 6,7 °C.

Для начала, с помощью функции `lag()` вынесем температуру предыдущего дня в отдельный столбец `prev`:

```
select
  wdate, wtemp,
  lag(wtemp, 1) over w as prev
from weather
where wdate between '2020-02-01' and '2020-02-08'
window w as (order by wdate);
```

wdate	wtemp	prev
2020-02-01	11.2	
2020-02-02	9.9	11.2
2020-02-03	10.3	9.9
2020-02-04	6.7	10.3
2020-02-05		6.7
2020-02-06	4.8	
2020-02-07	5.0	4.8
2020-02-08	9.5	5.0

Затем выберем значение `prev` в тех случаях, когда `wtemp` пустое. В этом поможет стандартная функция `coalesce()`, которая возвращает первое не-`null` значение:

```
select
  wdate, wtemp,
  coalesce(wtemp, lag(wtemp, 1) over w) as fixed
from weather
where wdate between '2020-02-01' and '2020-02-08'
window w as (order by wdate);
```

wdate	wtemp	fixed
2020-02-01	11.2	11.2
2020-02-02	9.9	9.9
2020-02-03	10.3	10.3
2020-02-04	6.7	6.7
2020-02-05		6.7
2020-02-06	4.8	4.8
2020-02-07	5.0	5.0
2020-02-08	9.5	9.5

Теперь в столбце `fixed` получилась температура без пропусков.

Предыдущее непустое значение

Снова посмотрим на февраль:

```
select wdate, wtemp
from weather
where wdate between '2020-02-01' and '2020-02-08';
```

wdate	wtemp
2020-02-01	11.2
2020-02-02	9.9
2020-02-03	10.3
2020-02-04	6.7
2020-02-05	
2020-02-06	4.8
2020-02-07	5.0
2020-02-08	9.5

Здесь сработает подход «заполнить предыдущим значением». Но что делать, если и предыдущее значение пустое?

```
update weather set wtemp = null
where wdate = '2020-02-04';
```

```
select
    wdate, wtemp,
    coalesce(wtemp, lag(wtemp, 1) over w) as fixed
from weather
where wdate between '2020-02-01' and '2020-02-08'
window w as (order by wdate);
```

wdate	wtemp	fixed
2020-02-01	11.2	11.2
2020-02-02	9.9	9.9
2020-02-03	10.3	10.3
2020-02-04		10.3
2020-02-05		
2020-02-06	4.8	4.8
2020-02-07	5.0	5.0
2020-02-08	9.5	9.5

Теперь установить температуру за 5 февраля не получится — ведь значение за 4 февраля тоже пустое.

Можно применить другой подход: брать не просто предыдущее, а предыдущее непустое значение. Тогда и 4, и 5 февраля получат температуру от 3 февраля — 10,3 °С.

Для начала, определим группы дней, как мы это делали в главе «Кластеризация». Будем считать, что номер группы увеличивается каждый раз, когда мы встречаем день с заполненной температурой:

```
select
  wdate, wtemp,
  sum(
    case when wtemp is not null then 1 else 0 end
  ) over w as group_id
from weather
where wdate between '2020-02-01' and '2020-02-08'
window w as (order by wdate);
```

wdate	wtemp	group_id
2020-02-01	11.2	1
2020-02-02	9.9	2
2020-02-03	10.3	3
2020-02-04		3
2020-02-05		3
2020-02-06	4.8	4
2020-02-07	5.0	5
2020-02-08	9.5	6

Благодаря такому подходу, проблемные дни 4 и 5 февраля попали в одну группу с «эталонным» 3 февраля. Осталось подставить температуру от 3 февраля с помощью функции `first_value()` и секции по `group_id`:


```
-- группы по температуре
with src as (
  select
    wdate, wtemp,
    sum(
      case when wtemp is not null then 1 else 0 end
    ) over w as group_id
  from weather
  where wdate between '2020-02-01' and '2020-02-08'
  window w as (order by wdate)
)
-- предыдущее непустое значение температуры
select
  wdate, wtemp,
  first_value(wtemp) over w as fixed
from src
window w as (
  partition by group_id
  order by wdate
);
```

wdate	wtemp	fixed
2020-02-01	11.2	11.2
2020-02-02	9.9	9.9
2020-02-03	10.3	10.3
2020-02-04		10.3
2020-02-05		10.3
2020-02-06	4.8	4.8
2020-02-07	5.0	5.0
2020-02-08	9.5	9.5

Идеально!

✎ Пропущенные осадки

Для некоторых дней не указаны осадки (`precip`). Заполните их как среднее арифметическое значение осадков от предыдущего и следующего дней.

Включите в ответ дни с 6 по 11 марта и с 1 по 6 июня. Должно получиться так:

wdate	precip	fixed
2020-03-06	0.0	0.0
2020-03-07	0.4	0.4
2020-03-08	2.0	2.0
2020-03-09	7.0	7.0
2020-03-10		4.5
2020-03-11	2.0	2.0
2020-06-01	0.0	0.0
2020-06-02	0.0	0.0
2020-06-03	0.4	0.4
2020-06-04		0.7
2020-06-05	1.0	1.0
2020-06-06	10.0	10.0

Сортировка результата: `wdate`

Песочница: <https://antonz.ru/to#w43db>

☑ Пропущенные осадки

Запрос:

- Считаем осадки предыдущего дня через `lag()`.
- Считаем осадки следующего дня через `lead()`.
- Берем их среднее арифметическое.

```
select
  wdate, precip,
  coalesce(
    precip,
    (lag(precip, 1) over w + lead(precip, 1)
over w)/2
  ) as fixed
from weather
where
  wdate between '2020-03-06' and '2020-03-11'
  or wdate between '2020-06-01' and '2020-06-06'
window w as (order by wdate);
```

Выбросы

Часто набор данных удобно характеризовать одним значением, чтобы не закапываться в детали. Например, посчитать среднюю температуру по каждому месяцу:

```
select
  strftime('%Y-%m', wdate) as wmonth,
  round(avg(wtemp), 2) as avg_temp
from weather
group by strftime('%Y-%m', wdate)
order by wmonth;
```

wmonth	avg_temp
2020-01	7.57
2020-02	8.29
2020-03	7.97
2020-04	12.35
2020-05	15.26
2020-06	17.72

Но со средним значением есть проблема: оно подвержено влиянию выбросов (экстремально больших или малых значений).

В баре сидят 9 работяг, доход каждого составляет 45 тыс. Средний доход посетителя бара равен 45 тыс.

В бар заходит вице-президент корпорации, который зарабатывает 1.7 млн. Оп! Средний доход посетителя бара вырос до $(45 \cdot 9 + 1700) / 10 = 210$ тыс.

Хорошо ли среднее арифметическое отражает реальность? Не слишком.

Чтобы сгладить влияние выбросов, можно вместо среднего использовать медиану.

Медиана — это срединное значение набора данных. 50% значений в датасете меньше медианы, 50% значений — больше. В нашем примере с баром медиана равна 45 тыс., что намного лучше отражает реальный доход посетителя бара.

Медиану можно рассчитать через функцию `percentile_cont()`, знакомую нам по главе «Статистика»:

```
select
  to_char(wdate, 'YYYY-MM') as wmonth,
  round(avg(wtemp)::decimal, 2) as avg_temp,
  round(
    percentile_cont(0.5) within group (order by
wtemp)::decimal, 2
  ) as med_temp
from weather
group by to_char(wdate, 'YYYY-MM')
order by wmonth;
```

Здесь и далее запросы для PostgreSQL, так как SQLite не поддерживает процентиля. Стоило бы сначала удалить дубли по датам, как мы это делали на одном из предыдущих шагов. Но для простоты не будем на это отвлекаться.

Есть и другой путь. Можно использовать среднее арифметическое, но предварительно отсеять выбросы. Например:

- выкинуть 5% самых маленьких значений (то есть значения меньше 5-го перцентиля);
- выкинуть 5% самых больших значений (то есть значения больше 95-го перцентиля);
- посчитать среднее арифметическое от оставшихся.

Считаем процентиля:

wmonth	avg_temp	med_temp
2020-01	7.57	7.90
2020-02	8.29	8.00
2020-03	7.97	7.65
2020-04	12.35	12.20
2020-05	15.26	16.05
2020-06	17.72	17.50

```
select
    to_char(wdate, 'YYYY-MM') as wmonth,
    min(wtemp) as temp_min,
    percentile_cont(0.05) within group (order by
wtemp) as temp_5,
    percentile_cont(0.95) within group (order by
wtemp) as temp_95,
    max(wtemp) as temp_max
from weather
group by to_char(wdate, 'YYYY-MM')
order by wmonth;
```

wmonth	temp_min	temp_5	temp_95	temp_max
2020-01	3.0	3.92	11.06	11.8
2020-02	4.8	5.04	11.88	12.5
2020-03	4.2	5.74	11.81	12.5
2020-04	4.2	8.39	16.79	17.2
2020-05	8.6	9.08	20.16	20.5
2020-06	10.9	13.44	25.02	25.9

Считаем среднее по температурам, попадающим в интервал от 5 до 95 перцентилей:

```
-- границы
with avgs as (
  select
    to_char(wdate, 'YYYY-MM') as wmonth,
    percentile_cont(0.05) within group (order by
wtemp) as p5,
    percentile_cont(0.95) within group (order by
wtemp) as p95
  from weather
  group by to_char(wdate, 'YYYY-MM')
)
-- средние значения
select
  to_char(wdate, 'YYYY-MM') as wmonth,
  round(avg(wtemp)::decimal, 2) as temp_avg,
  round(
    avg(wtemp) filter (
      where wtemp between avgs.p5 and avgs.p95
    )::decimal, 2
  ) as adjusted
from weather
join avgs on to_char(wdate, 'YYYY-MM') = avgs.
wmonth
group by to_char(wdate, 'YYYY-MM')
order by wmonth;
```

wmonth	avg_temp	adjusted
2020-01	7.57	7.58
2020-02	8.29	8.18
2020-03	7.97	7.88
2020-04	12.35	12.45
2020-05	15.26	15.36
2020-06	17.72	17.53

Среднее с поправкой на выбросы готово!

✂ Подавленные даты

Для каждого месяца посчитайте количество дат, которые попадают в верхние 10% по значению `pressure` для этого месяца. Например, для января:

```
select pressure
from weather
where wdate between '2020-01-01' and '2020-01-31'
order by pressure;
```

pressure
99490.0
99610.0
100130.0
100450.0
100510.0
...
103400.0
103810.0
104260.0
104280.0
104820.0

Здесь 90 перцентиль равен 103769. В верхние 10% попадают даты, у которых значение `pressure` > 103769 — таких дат четыре. Значит, искомое количество для января — 4.

Посчитайте искомое количество дат для каждого месяца от января до июня. Для расчета перцентилей используйте функцию `percentile_cont()`.

Песочница: <https://antonz.ru/to#w43pg>

☑ Подавленные даты

Считаем 90-й перцентиль давления для каждого месяца:

```
select
    extract(month from wdate) as wmonth,
    percentile_cont(0.90) within group (order by
pressure) as p90
from weather
group by extract(month from wdate);
```

Теперь для каждого месяца считаем количество дней, в которые значение осадков превысило 90-й перцентиль:

```
with boundary as (
    ...
)
select
    extract(month from wdate) as wmonth,
    count(*) filter (where pressure > boundary.p90)
as over_count
from weather
    join boundary on extract(month from wdate) =
boundary.wmonth
group by extract(month from wdate)
order by wmonth;
```

wmonth	over_count
2020-01	4
2020-02	3
2020-03	4
2020-04	2
2020-05	2
2020-06	3

Готово!

Так держать

Отличная работа! Теперь вы умеете обращаться с наборами данных из ненадежных источников.

Заключение

Оконные функции — не просто функции. Это целый мини-язык внутри обычного SQL. И язык, прямо скажем, не самый простой.

Вы совершили то, что мало кому удастся: разобрались в нюансах «оконного языка» и научились решать самые разные задачи с его помощью. Вот что вы теперь умеете:

- Ранжировать записи и разбивать их на группы.
- Сравнивать записи друг с другом и с границами секций.
- Считать общие и скользящие агрегаты.
- Вычислять сводные показатели.
- Запускать функции на отдельных записях, группах и диапазонах.
- Отсеивать часть записей при агрегации.
- Строить финансовые отчеты, находить кластеры и чистить данные.

Поздравляю!

Если книга вам понравилась — пожалуйста, рекомендуйте ее друзьям и коллегам. Сарафанное радио — основной способ для книги найти своего читателя.

Чем больше в отрасли людей, которые понимают продвинутые возможности SQL — тем лучше для всех. Очень рад, что вы теперь в их числе!

Антон

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
----------------	---

О книге	3
Зачем нужны оконные функции	5
Песочница	15

Часть 1

ОКНА И ФУНКЦИИ	16
----------------------	----

Ранжирование	16
Сравнение со смещением	41
Агрегация	61
Скользящие агрегаты	76
Статистика	97
Резюме	121

Часть 2

ФРЕМЫ	125
-------------	-----

ROWS и GROUPS	125
RANGE-фреймы	144
EXCLUDE	159
FILTER	176
Резюме	187

Часть 3

ПРАКТИКА	190
----------------	-----

Финансы	190
---------------	-----

Кластеризация	212
---------------------	-----

Очистка данных	235
----------------------	-----

ЗАКЛЮЧЕНИЕ	252
------------------	-----

Научно-популярное издание

*Танымал ғылыми басылым
Серия «Программирование для всех»*

12+

Антон Жиянов

**ОКОННЫЕ ФУНКЦИИ SQL.
Анализ данных на практике**

Редактор *А. Ходякова*
Менеджер проекта *В. Живина*
Дизайнер *А. Шмулий*
Верстка *А. Грених*
Технический редактор *Н. Чернышева*

Подписано в печать 10.07.2024. Формат 70х100/16. Усл. печ. л. 14.

Печать офсетная. Гарнитура SonetSerif. Бумага офсетная.

Тираж 2000 экз. Заказ №

Произведено в Российской Федерации

Изготовлено в 2024 г.

Оригинал–макет подготовлен редакцией «Времена», импринт «Альфа»

Изготовитель: ООО «Издательство АСТ»

129085, Российская Федерация, г. Москва, Звездный бульвар, д. 21, стр. 1,
комн. 705, пом. I, этаж 7

Наш сайт: WWW.AST.RU

E-mail: ask@ast.ru

Общероссийский классификатор продукции ОК–034–2014 (КПЕС 2008);

58.11.1 — книги, брошюры печатные

«Баспа Аста» деген ООО
129085, г. Мәскеу, Жұлдызды гүлзар, д. 21, 1 кұрылым, 705 бөлме,
пом. 1, 7-қабат
Біздің электрондық мекенжайымыз : www.ast.ru
E-mail: ask@ast.ru
Интернет-магазин: www.book24.kz Интернет-дүкен: www.book24.kz
Импортёр в Республику Казахстан и Представитель по приему претензий
в Республике Казахстан — ТОО РДЦ Алматы, г. Алматы.
Қазақстан Республикасына импорттаушы және
Қазақстан Республикасында наразылықтарды қабылдау
бойынша өкіл — «РДЦ-Алматы» ЖШС, Алматы
к., Домбровский көш., 3«а», Б литері офис 1. Тел.: 8(727) 2 51 59 90, 91 ,
факс: 8 (727) 251 59 92 ішкі 107; E-mail: RDC-Almaty@eksmo.kz ,
www.book24.kz Тауар белгісі: «АСТ» Өндірілген жылы: 2024
Өнімнің жарамдылық; мерзімі шектелмеген.
Сертификация қарастырылмаған

АНТОН ЖИЯНОВ — разработчик Go и энтузиаст SQLite, специалист по работе с исходным кодом.

«ОКОННЫЕ ФУНКЦИИ SQL» — книга о мощном инструменте для анализа данных, который позволяет выполнять сложные вычисления и получать информацию о группах строк или результатах окон, но если вкратце — как делать классные аналитические отчеты без участия «экслея».

Вместе с Антоном Жияновым разберемся в основах SQL:

- что такое «окно» в SQL и как оно работает;
- про фреймы и как с ними работать;
- и конечно же выполним практические задания в песочнице.

Книга будет полезна как начинающим разработчикам, так и опытным специалистам, желающим расширить и закрепить свои знания в области оконных функций SQL.

книги для любого настроения здесь



ИЗДАТЕЛЬСКАЯ ГРУППА АСТ

www.ast.ru | www.book24.ru

 vk.com/izdatelstvoast

 ok.ru/izdatelstvoast

ISBN 978-5-17-158845-8



9 785171 588458